ENGLISH FOR WORK / ENGLISH FOR SOFTWARE ENGINEERS AND DEVELOPMENT TEAMS

# Software Development English

Make the behavior clear. Make the limits explicit.



## Phil Stilwell

© 2026 Phil Stilwell. All rights reserved.
No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior written permission from the author, except as permitted by law.

Eight practical lessons. Eleven extended conversations.
Vocabulary, phrase banks, guided speaking, and explained answers.

Revised 2026 edition / Upper-intermediate to advanced

# Your route through the book


## 01 Clarifying interface contracts, page 4

Distinguish omission, an empty string, and null while requesting an explicit interface decision before implementation.

## 02 Debugging from reproducible evidence, page 14

Report a reproducible failure, expose a confounded comparison, and request a controlled test without overstating causation.

## 03 Giving actionable code-review feedback, page 26

Anchor a review comment in a changed call location, its observable effect, and a specific correction and test.

## 04 Requesting dependency evidence, page 36

Request the resolved dependency version, affected call sites, and migration evidence before claiming upgrade compatibility.

## 05 Explaining what a test actually exercises, page 49

Explain the limits of a success-only test double and distinguish a failure-path unit test from real-adapter integration coverage.

## 06 Comparing performance tradeoffs fairly, page 59

Compare measured latency and memory under matched conditions while separating median improvement from budget compliance and production claims.

## 07 Discussing retries and shared state, page 71

Explain an uncertain request outcome and request a retry contract before risking a second reservation.

## 08 Handing over a release with explicit limits, page 81

Hand over environment, flag state, migration limits, and decision ownership without conflating deployment with production readiness.

Professional communication notes, page 3

Three additional workplace cases, page 94

Answers and explanations, page 108

Phrase finder, page 166

Vocabulary index, page 168

References and publication notes, page 173


Suggested study range: B2-C1. This is an editorial study range, not a certified proficiency assessment.

# Describe behavior before confidence.

A confident engineering statement names its evidence and its boundary. A test double is not the real dependency; a staging deployment is not a production release; a timeout does not establish that a request failed.

## Separate contract from convention

An omitted field, an empty string, and null can mean different things. State what this interface actually promises rather than relying on how another system behaves. A proposed interpretation needs agreement before it becomes a contract.
"Omission leaves the nickname unchanged; the meaning of null is not specified."

## Make disagreement actionable

Anchor review comments in a changed line, observable behavior, and a concrete request. Keep naming preferences separate from defects. A focused failure test can make the concern easier to discuss without turning the review into a judgment of its author.
"Moving this call outside the success branch also sends completion notices after failure."

## Attach the measurement conditions

A median describes the middle of observed results, not the slowest users or production behavior. Compare implementations under the same stated conditions, and include resource budgets. A faster measured response does not settle an overall tradeoff.
"B lowers the median to twenty-five milliseconds but exceeds the memory budget."

## Keep release stages distinct

Code can be deployed but disabled, tested in staging but not production, or approved for review but not release. Name the environment, flag state, unresolved compatibility check, and actual decision owner in the handoff.
"R18 is in staging with search disabled; Lee still owns the release decision."

### Professional boundaries

All products, people, measurements, interfaces, and release processes are fictional. These scenarios teach professional English, not production operating instructions. Actual contracts, security requirements, engineering review, test environments, and change authorization take precedence. No exercise authorizes production changes or guarantees correctness, compatibility, reliability, or performance. Code-review and testing terminology is used without requiring a particular language or framework.

# A Read the situation

**LESSON 01 / CLARIFYING INTERFACE CONTRACTS**

Distinguish omission, an empty string, and null while requesting an explicit interface decision before implementation.

> **Three nickname states, only two defined meanings**
>
> A fictional profile interface accepts an optional nickname. Its current contract says an omitted nickname leaves the stored value unchanged, while an empty string clears it. A proposed client sends null. The meaning of null has not been specified. Client developer Maya and service developer Arun review the mismatch before either side implements a new interpretation. Optional describes whether the field may be absent; it does not settle what an explicit null means.

People: Maya, Client developer; Arun, Service developer.

## Read the record

Choose one answer per question using the facts above.

**1. What does omitting nickname do under the current contract?**

**A.** Rejects every update
**B.** Leaves the stored value unchanged
**C.** Clears it
**D.** Writes the word null

**2. What does an empty string do?**

**A.** Leaves it unchanged
**B.** Sets an unspecified default
**C.** Proves null is accepted
**D.** Clears the nickname

**3. What remains undefined?**

**A.** The meaning of an explicit null
**B.** Whether omission is possible
**C.** Whether an empty string clears
**D.** Whether nickname is a field

Answers and explanations, page 108

# B Vocabulary in use

**CORE TERMS**

### interface contract
Agreed rules for inputs, outputs, and observable behavior.

*clarify the interface contract*

### application programming interface
Defined way software components communicate; abbreviated API.

*document the API*

### request payload
Data carried by a request.

*inspect the request payload*

### optional field
Field that may be absent under the contract.

*omit an optional field*

### nullable field
Field whose allowed values include null under the contract.

*declare a nullable field*

### omitted field
Field not included in the submitted data.

*distinguish an omitted field*

### empty string
Text value containing zero characters.

*send an empty string*

### null value
Explicit marker whose meaning depends on the contract.

*define the null value*

### stored state
Value or condition currently retained by the system.

*preserve stored state*

### partial update
Change affecting only specified parts of a record.

*apply a partial update*

### default value
Value supplied when the governing rules call for one.

*document the default value*

### validation rule
Condition an input must satisfy.

*agree on a validation rule*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**interface contract** INTERFACE CONTRACT

clarify the interface contract

Keep the meaning precise: Agreed rules for inputs, outputs, and observable behavior.

**omitted field** OMITTED FIELD

distinguish an omitted field

Keep the meaning precise: Field not included in the submitted data.

# B Precision and professional shorthand

**EXTEND YOUR RANGE**

### serialization

Conversion of data into a transportable representation.

*inspect serialization behavior*

### deserialization

Conversion from transported data into usable values.

*verify deserialization behavior*

### schema

Formal description of data structure and constraints.

*revise the schema*

### field presence

Whether a particular field exists in the input.

*check field presence*

### strong ETag

Representation validator compared strongly in an If-Match condition.

*compare the strong ETag*

### backward compatibility

Ability to preserve supported behavior for earlier consumers.

*assess backward compatibility*

### consumer

Component using another component's interface.

*identify the consumer*

### producer

Component providing data or behavior through an interface.

*align producer expectations*

### decimal arithmetic

Base-ten calculation with specified precision and rounding behavior.

*use decimal arithmetic*

### contract test

Test checking agreed interactions across an interface boundary.

*add a contract test*

### unspecified interface behavior

Input effect not defined by this interface contract.

*resolve unspecified interface behavior*

### acceptance criterion

Specific condition used to judge whether work meets requirements.

*state an acceptance criterion*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**validation rule** VALIDATION RULE

agree on a validation rule

Keep the meaning precise: Condition an input must satisfy.

**strong ETag** STRONG ETAG

compare the strong ETag

Keep the meaning precise: Representation validator compared strongly in an If-Match condition.

# C Phrases that move the work forward

**Name the boundary**
I want to confirm the profile interface contract.

**Describe clearing**
An empty string clears the nickname.

**Expose the gap**
The contract does not specify what null means.

**Ask for the effect**
What should the stored value be after that input?

**Inspect the actual data**
Can we check the request payload?

**Keep a proposal provisional**
Treating null as clear would be a new decision.

**Request documentation**
Please record that rule in the contract.

**Avoid premature completion**
The null case is still unresolved.

**Describe absence**
An omitted nickname leaves the stored value unchanged.

**Identify the proposal**
The proposed client sends null.

**Separate terms**
Optional does not automatically mean nullable.

**Avoid borrowing a rule**
I would not infer this from another API.

**State a condition**
If the field is absent, the current value stays.

**Request agreement**
Can we agree on the intended behavior first?

**Request a check**
We need a test for each defined input state.

**Close precisely**
We agree on omission and empty string, but not yet on null.

## Precision in real exchanges

**Purpose:** Identify the proposal
**Useful wording:** The proposed client sends null.

**Purpose:** Expose the gap
**Useful wording:** The contract does not specify what null means.

# D Choose the response that fits

Choose one response for each situation. Preserve the facts, the decision boundary, and the purpose of the exchange.

**1. Which comment identifies the real gap?**

**A.** Empty string and omission are identical
**B.** Every API interprets null alike
**C.** The null case needs an agreed meaning
**D.** Optional proves null clears it

**2. Which pair already has different defined effects in this contract?**

**A.** Null and a documented clearing command
**B.** Omission and an empty string
**C.** Two spellings of the same field name
**D.** Two confirmed clearing commands

**3. A colleague needs the term for this meaning: Field that may be absent under the contract. Which term fits?**

**A.** optional field
**B.** decimal arithmetic
**C.** stored state
**D.** schema

# D Choose the response that fits / continued

**4. Rebuild the exchange from "Three nickname states, only two defined meanings". Maya: "For the existing clearing action, then, I'd send a string with zero characters. Leaving nickname out would preserve the old value instead." Arun: [missing response] Maya: "I'll stop calling null an alternative clearing command. We haven't agreed on what it means here, even if another API uses it that way." Which response belongs in the gap?**

**A.** We should separate that from an omitted field. If nickname is absent from the request, the stored value stays unchanged. That part is already defined.

**B.** That will make field presence visible. For the first two, write the resulting stored value beside the input rather than using the vague label no nickname.

**C.** Exactly. An optional field may be absent. It does not tell us whether null is accepted or what effect null should have if it is accepted.

**D.** Yes, an empty string clears the nickname. We should preserve that distinction in the examples so nobody silently changes the current behavior while updating the client.

Answers and explanations, page 108

### One useful distinction

Optional and nullable answer different questions. Optional concerns presence; nullable concerns allowed values. The stated contract defines omission and an empty string, but it does not authorize treating null as either one.

# E Conversation

## Three nickname states, only two defined meanings

Three nickname states, only two defined meanings

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

omitted field / request payload / acceptance criterion / field presence / backward compatibility / optional field / validation rule / interface contract / empty string / contract test

01 **Maya:** The form currently sends null when a user removes a nickname. Before I ship that, which input does this service actually define as clearing?

02 **Arun:** We should separate that from an [1] _____. If nickname is absent from the request, the stored value stays unchanged. That part is already defined.

03 **Maya:** I treated optional as permission to send null. But a missing property and a present property with a null value aren't the same payload.

04 **Arun:** Exactly. An [2] _____ may be absent. It does not tell us whether null is accepted or what effect null should have if it is accepted.

05 **Maya:** For the existing clearing action, then, I'd send a string with zero characters. Leaving nickname out would preserve the old value instead.

06 **Arun:** Yes, an [3] _____ clears the nickname. We should preserve that distinction in the examples so nobody silently changes the current behavior while updating the client.

07 **Maya:** I'll stop calling null an alternative clearing command. We haven't agreed on what it means here, even if another API uses it that way.

08 **Arun:** Correct. The [4] _____ needs a decision for that input. We could discuss a change, but discussion alone does not make the proposed meaning an existing guarantee.

09 **Maya:** The client object shows null, but I haven't captured what its serializer sends. Could that conversion drop the property before the request leaves?

10 **Arun:** Let us check the [5] _____ in a controlled example. We need to distinguish the client's internal value from the representation delivered to the service.

Conversation continues on the next page.

# Three nickname states, only two defined meanings

**CONVERSATION / CONTINUED**

11 **Maya:** I'll show the actual three wire examples separately: absent, empty string, and explicit null. Only the first two have defined outcomes so far.

12 **Arun:** That will make [6] _____ visible. For the first two, write the resulting stored value beside the input rather than using the vague label no nickname.

13 **Maya:** Would allowing null in the schema get ahead of the decision? I don't want that change to imply a stored-state rule nobody has approved.

14 **Arun:** Hold that change until the [7] _____ and intended effect are agreed. Allowing a value and defining its effect need to be consistent.

15 **Maya:** Once the rule is settled, I'll test the client-service interaction, not just whether the form looks empty on screen.

16 **Arun:** A [8] _____ can support that. Its expected result must come from the agreement, though; the test must not invent the missing null rule.

17 **Maya:** And I want the omission case protected. Solving the null question shouldn't silently turn an unchanged nickname into a cleared one for older clients.

18 **Arun:** That is a [9] _____ concern worth recording. We have not demonstrated compatibility merely by agreeing that the new client looks correct.

19 **Maya:** Then I'll report omission preserves, empty string clears, and null unresolved. My proposed null implementation stays pending until the interface decision.

20 **Arun:** Good. That gives us an explicit [10] _____ once the decision is made, instead of treating an ambiguous word as enough to approve the change.

**CHECK THE GAPS**

**1.** omitted field / **2.** optional field / **3.** empty string / **4.** interface contract / **5.** request payload / **6.** field presence / **7.** validation rule / **8.** contract test / **9.** backward compatibility / **10.** acceptance criterion

Why these answers fit, page 109

# F Check accuracy and scope

## Three nickname states, only two defined meanings

### Return to the case

A fictional profile interface accepts an optional nickname. Its current contract says an omitted nickname leaves the stored value unchanged, while an empty string clears it. A proposed client sends null. The meaning of null has not been specified. Client developer Maya and service developer Arun review the mismatch before either side implements a new interpretation. Optional describes whether the field may be absent; it does not settle what an explicit null means.

Choose one answer per question. Use the case facts and keep any uncertainty or limits intact.

**1. What would justify calling null a clearing command?**

**A.** The developer's preference alone
**B.** The word optional alone
**C.** An unrelated API example
**D.** An explicit agreed contract change

**2. Which test plan matches the discussion?**

**A.** Check each defined input state and resolve null before asserting its effect
**B.** Test omission and assume all other inputs match
**C.** Test only the client's display
**D.** Treat null as proven compatible

Answers and explanations, page 111

### Listen for the distinction

Swap roles and repeat turns 9-16, stressing actual payload and agreed validation rule.

### Ask for the missing rule, not a guess

Developers may use the same familiar word for different behaviors. Repeat the exact input and resulting state before agreeing that an interface is clear. An explicit question about null avoids hiding a product decision inside an implementation detail.

# F Say it / transfer to a new case

## Three inputs, three separate statements

Complete the interface summary using only the stated contract and the unresolved proposal.

Complete the four gaps. With a partner, read alternate roles, then switch. Studying alone, read both roles and pause before each response.

**WORD AND PHRASE BANK**

unspecified / unchanged / clears / nullable

01 **Developer:** An omitted nickname leaves the value [1] _____.

02 **Reviewer:** An empty string [2] _____ the nickname.

03 **Developer:** The meaning of null remains [3] _____.

04 **Reviewer:** Optional does not automatically mean [4] _____.

**CHECK THE GAPS**

**1.** unchanged / **2.** clears / **3.** unspecified / **4.** nullable

Why these answers fit, page 111

### Rebuild the exchange

Choose the missing response. Read the completed three-turn exchange aloud, changing roles on the second reading.

**1. Rebuild the exchange from "Three nickname states, only two defined meanings". Maya: "Would allowing null in the schema get ahead of the decision? I don't want that change to imply a stored-state rule nobody has approved." Arun: [missing response] Maya: "Once the rule is settled, I'll test the client-service interaction, not just whether the form looks empty on screen." Which response belongs in the gap?**

**A.** Hold that change until the validation rule and intended effect are agreed. Allowing a value and defining its effect need to be consistent.

**B.** That is a backward compatibility concern worth recording. We have not demonstrated compatibility merely by agreeing that the new client looks correct.

**C.** We should separate that from an omitted field. If nickname is absent from the request, the stored value stays unchanged. That part is already defined.

**D.** A contract test can support that. Its expected result must come from the agreement, though; the test must not invent the missing null rule.

Answers and explanations, page 112

# A Read the situation

**LESSON 02 / DEBUGGING FROM REPRODUCIBLE EVIDENCE**

Report a reproducible failure, expose a confounded comparison, and request a controlled test without overstating causation.

> **The empty date is a suspect, not yet the cause**
>
> Developer Nia has a three-row import file containing one empty date. It fails, and the stack trace points to date parsing. A two-row file without that row succeeds. Colleague Ellis reviews the evidence. No controlled test has isolated the empty date as the cause: the successful file differs in both row count and content. They need a test in a suitable nonproduction environment that changes one relevant condition while preserving the others.

People: Nia, Developer investigating the import; Ellis, Developer reviewing the evidence.

## Read the record

Choose one answer per question using the facts above.

**1. Which input fails?**

**A.** Every file with any date
**B.** A three-row file containing an empty date
**C.** Every three-row file
**D.** A two-row file with all dates empty

**2. What does the successful comparison change?**

**A.** Nothing relevant
**B.** Both row count and content
**C.** Only the date format
**D.** Only the environment

**3. What does the stack trace establish?**

**A.** The empty field is conclusively the root cause
**B.** The database is corrupt
**C.** Production is safe to test
**D.** The observed failure path reaches date parsing

Answers and explanations, page 113

# B Vocabulary in use

**CORE TERMS**

### reproduction
Repeatable demonstration of an observed failure.

*obtain a reproduction*

### reproducible example
Input and conditions that consistently show the behavior.

*share a reproducible example*

### stack trace
Recorded chain of calls associated with an error.

*inspect the stack trace*

### parser
Component interpreting structured input.

*inspect the parser*

### date parsing
Conversion of date text into a usable representation.

*investigate date parsing*

### input fixture
Prepared input used in a test.

*preserve the input fixture*

### hypothesis
Proposed explanation requiring evidence.

*test a hypothesis*

### root cause
Underlying cause established through investigation.

*isolate the root cause*

### confounding variable
Additional changed factor obscuring a comparison.

*remove a confounding variable*

### controlled comparison
Comparison holding relevant conditions fixed except the tested change.

*run a controlled comparison*

### optimistic concurrency
Detecting intervening changes when a conditional update is attempted.

*enforce optimistic concurrency*

### rounding mode
Rule for selecting a representable value when a result must be rounded.

*specify the rounding mode*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**reproduction** REPRODUCTION

obtain a reproduction

Keep the meaning precise: Repeatable demonstration of an observed failure.

**input fixture** INPUT FIXTURE

preserve the input fixture

Keep the meaning precise: Prepared input used in a test.

# B Precision and professional shorthand

EXTEND YOUR RANGE

### failure signature
Identifying pattern of an observed failure.
*compare the failure signature*

### batched lookup
Retrieving data for several keys in one grouped operation.
*replace repeated calls with a batched lookup*

### empty field
Present field containing no supplied value.
*isolate the empty field*

### deterministic behavior
Behavior producing the same result under the same relevant conditions.
*check deterministic behavior*

### regression
Previously working behavior broken by a later change.
*investigate a regression*

### causal claim
Statement that one condition produces an effect.
*qualify a causal claim*

### exception
Signal that interrupts normal execution under specified conditions.
*capture the exception*

### N+1 query pattern
One initial query followed by one related query per loaded item.
*identify an N+1 query pattern*

### malformed input
Data not conforming to the required structure or rules.
*identify malformed input*

### environment parity
Relevant similarity between execution environments.
*establish environment parity*

### instrumentation
Added observation mechanisms such as logs or measurements.
*improve instrumentation*

### nonproduction environment
Environment not serving the live production workload.
*use a nonproduction environment*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**rounding mode** ROUNDING MODE

specify the rounding mode

Keep the meaning precise: Rule for selecting a representable value when a result must be rounded.

**empty field** EMPTY FIELD

isolate the empty field

Keep the meaning precise: Present field containing no supplied value.

# C Phrases that move the work forward

**Report the failing input**
The three-row file fails during import.

**Report the distinguishing detail**
One row contains an empty date.

**Name the observation**
The stack trace reaches date parsing.

**Report the comparison**
The two-row file succeeds.

**Qualify the conclusion**
I suspect the empty date, but have not isolated it.

**Expose the confound**
We changed both row count and content.

**Ask for control**
Can we hold the other fields constant?

**Propose a narrow change**
Change only the suspect date condition.

**Preserve evidence**
Keep the original failing fixture.

**Avoid a universal claim**
We have not shown that every empty date fails.

**Ask for repeatability**
Does the same fixture fail consistently?

**Compare the outcome**
Check whether the failure signature stays the same.

**Name the setting**
Run the comparison in a suitable nonproduction environment.

**Separate location and cause**
The trace points to parsing, not yet to a proven cause.

**Report a limit**
We do not have the controlled result yet.

**Close the handoff**
Share the fixture, conditions, trace, and next comparison.

## Precision in real exchanges

**Purpose:** Report the comparison
**Useful wording:** The two-row file succeeds.

**Purpose:** Qualify the conclusion
**Useful wording:** I suspect the empty date, but have not isolated it.

# D Choose the response that fits

Choose one response for each situation. Preserve the facts, the decision boundary, and the purpose of the exchange.

**1. Which conclusion is supported?**

**A.** The root cause is proven
**B.** The observed failing path reaches date parsing
**C.** All empty dates always fail
**D.** Row count has no effect

**2. Which proposed comparison is strongest?**

**A.** Change date, encoding, and parser version together
**B.** Keep three rows and other content fixed while varying the suspect date
**C.** Delete two more rows and change the environment
**D.** Compare unrelated files

**3. A colleague needs the term for this meaning: Component interpreting structured input. Which term fits?**

**A.** parser
**B.** confounding variable
**C.** regression
**D.** batched lookup

# D Choose the response that fits / continued

**4. Rebuild the exchange from "The empty date is a suspect, not yet the cause". Nia: "I wrote that the empty date causes the error. You're asking me to narrow that to what the comparison actually shows." Ellis: [missing response] Nia: "Then I'll keep three rows and change only that date to a valid value, with the other data and settings unchanged." Which response belongs in the gap?**

**A.** Let's make that a reproducible example before calling the cause settled. Can another run with the same input and conditions produce the same failure?

**B.** Yes. Row count is a confounding variable in this comparison. We don't know whether it matters, but we haven't held it fixed.

**C.** That gives us a controlled comparison. Keep both files so we can inspect exactly what changed rather than rely on a description.

**D.** It could. The stack trace records the observed call path; it doesn't establish which input condition first caused the problem.

Answers and explanations, page 113

> **One useful distinction**
>
> The two-row success is useful evidence, but it changes two things at once. It does not distinguish an empty-date effect from other differences associated with the removed row or the changed row count.

# E Conversation

## The empty date is a suspect, not yet the cause

The empty date is a suspect, not yet the cause

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

failure signature / reproducible example / confounding variable / stack trace / input fixture / controlled comparison / deterministic behavior / nonproduction environment / hypothesis / causal claim

01 **Nia:** The three-row import failed, and the trace reaches date parsing. I've attached the input, including the empty date in row two.

02 **Ellis:** Let's make that a [1] _____ before calling the cause settled. Can another run with the same input and conditions produce the same failure?

03 **Nia:** I haven't established that yet. I removed row two and the import passed, which made me suspect the empty date.

04 **Ellis:** That's a useful [2] _____, not a conclusion. Removing the row changed its contents and the number of rows together.

05 **Nia:** I wrote that the empty date causes the error. You're asking me to narrow that to what the comparison actually shows.

06 **Ellis:** Yes. Row count is a [3] _____ in this comparison. We don't know whether it matters, but we haven't held it fixed.

07 **Nia:** Then I'll keep three rows and change only that date to a valid value, with the other data and settings unchanged.

08 **Ellis:** That gives us a [4] _____. Keep both files so we can inspect exactly what changed rather than rely on a description.

09 **Nia:** Should I reduce the file further first? Two rows would be easier to send around, but that version currently passes.

10 **Ellis:** Keep the failing [5] _____ intact. A smaller passing file isn't a smaller reproduction of the failure we're investigating.

Conversation continues on the next page.

# The empty date is a suspect, not yet the cause

**CONVERSATION / CONTINUED**

11 **Nia:** The trace points into the parser. Could that still be downstream of a problem elsewhere in the import?

12 **Ellis:** It could. The [6] _____ records the observed call path; it doesn't establish which input condition first caused the problem.

13 **Nia:** I'll preserve the error type and location with the run details. If a later run fails differently, we shouldn't count it as the same result.

14 **Ellis:** Exactly. Compare the [7] _____ as well as pass or fail. Otherwise a changed exception could hide a different defect.

15 **Nia:** And I'll rerun each controlled input. A single failure followed by a single success doesn't establish repeatability.

16 **Ellis:** Right. Check [8] _____ rather than assuming it. Record variations if the same input doesn't reliably produce the same result.

17 **Nia:** I'll use the approved test environment, not try these files against a live customer import to get a quick answer.

18 **Ellis:** Use the suitable [9] _____ and keep its configuration in the record. We need evidence without changing production data.

19 **Nia:** My update will say the empty date is suspected, the successful comparison changed two factors, and a controlled repeat test is pending.

20 **Ellis:** That keeps the [10] _____ at the right strength. Bring back the actual results; don't mark the hypothesis confirmed before the comparison is run.

**CHECK THE GAPS**

**1.** reproducible example / **2.** hypothesis / **3.** confounding variable / **4.** controlled comparison / **5.** input fixture / **6.** stack trace / **7.** failure signature / **8.** deterministic behavior / **9.** nonproduction environment / **10.** causal claim

Why these answers fit, page 114

# F Check accuracy and scope

## The empty date is a suspect, not yet the cause

### Return to the case

Developer Nia has a three-row import file containing one empty date. It fails, and the stack trace points to date parsing. A two-row file without that row succeeds. Colleague Ellis reviews the evidence. No controlled test has isolated the empty date as the cause: the successful file differs in both row count and content. They need a test in a suitable nonproduction environment that changes one relevant condition while preserving the others.

Choose one answer per question. Use the case facts and keep any uncertainty or limits intact.

**1. Which phrase avoids overstating evidence?**

**A.** The empty date is conclusively responsible
**B.** The trace proves every input rule
**C.** The successful file eliminates all other causes
**D.** The empty date is a working hypothesis

**2. What should the handoff preserve?**

**A.** The failing fixture, conditions, and trace
**B.** Only the developer's confidence
**C.** A rewritten file without its original data
**D.** A production change without authorization

Answers and explanations, page 116

### Listen for the distinction

Swap roles and repeat turns 5-10, keeping row count constant in the proposed test.

### Treat a hypothesis as useful, not embarrassing

Naming a likely cause can help investigation, provided its status remains clear. Distinguish I suspect from I have isolated. A colleague can challenge a comparison without dismissing the observation or the person who found it.

# F Say it / transfer to a new case

## Report the evidence without upgrading the conclusion

Complete the debugging handoff. The controlled test has not yet produced a result.

Complete the four gaps. With a partner, read alternate roles, then switch. Studying alone, read both roles and pause before each response.

**WORD AND PHRASE BANK**

three / hypothesis / constant / two

01 **Developer:** The failing file contains [1] _____ rows.

02 **Reviewer:** The successful file contains [2] _____ rows.

03 **Developer:** The empty date remains a [3] _____.

04 **Reviewer:** Keep the other conditions [4] _____ in the next comparison.

**CHECK THE GAPS**

**1.** three / **2.** two / **3.** hypothesis / **4.** constant

Why these answers fit, page 116

### Rebuild the exchange

Choose the missing response. Read the completed three-turn exchange aloud, changing roles on the second reading.

**1. Rebuild the exchange from "The empty date is a suspect, not yet the cause". Nia: "I'll preserve the error type and location with the run details. If a later run fails differently, we shouldn't count it as the same result." Ellis: [missing response] Nia: "And I'll rerun each controlled input. A single failure followed by a single success doesn't establish repeatability." Which response belongs in the gap?**

**A.** Exactly. Compare the failure signature as well as pass or fail. Otherwise a changed exception could hide a different defect.

**B.** It could. The stack trace records the observed call path; it doesn't establish which input condition first caused the problem.

**C.** That's a useful hypothesis, not a conclusion. Removing the row changed its contents and the number of rows together.

**D.** Right. Check deterministic behavior rather than assuming it. Record variations if the same input doesn't reliably produce the same result.

Answers and explanations, page 117

# Review 1 / Meaning and context

Use language from the lessons you have completed. Choose one answer per question.

> **Case file**
>
> Revisit "Clarifying interface contracts" and "Debugging from reproducible evidence". Choose precise terminology, then reconstruct the missing responses using both neighboring turns. The final two questions also revisit earlier conversations.

**1. A colleague needs the term for this meaning: Data carried by a request. Which term fits?**

**A.** null value
**B.** request payload
**C.** deserialization
**D.** producer

**2. A colleague needs the term for this meaning: Underlying cause established through investigation. Which term fits?**

**A.** reproduction
**B.** deterministic behavior
**C.** root cause
**D.** failure signature

**3. Rebuild the exchange from "Three nickname states, only two defined meanings". Maya: "The client object shows null, but I haven't captured what its serializer sends. Could that conversion drop the property before the request leaves?" Arun: [missing response] Maya: "I'll show the actual three wire examples separately: absent, empty string, and explicit null. Only the first two have defined outcomes so far." Which response belongs in the gap?**

**A.** Exactly. An optional field may be absent. It does not tell us whether null is accepted or what effect null should have if it is accepted.
**B.** Let us check the request payload in a controlled example. We need to distinguish the client's internal value from the representation delivered to the service.
**C.** We should separate that from an omitted field. If nickname is absent from the request, the stored value stays unchanged. That part is already defined.
**D.** Good. That gives us an explicit acceptance criterion once the decision is made, instead of treating an ambiguous word as enough to approve the change.

# Review 1 / Meaning and context / continued

**4. Rebuild the exchange from "The empty date is a suspect, not yet the cause". Nia: "I'll use the approved test environment, not try these files against a live customer import to get a quick answer." Ellis: [missing response] Nia: "My update will say the empty date is suspected, the successful comparison changed two factors, and a controlled repeat test is pending." Which response belongs in the gap?**

**A.** Keep the failing input fixture intact. A smaller passing file isn't a smaller reproduction of the failure we're investigating.

**B.** Use the suitable nonproduction environment and keep its configuration in the record. We need evidence without changing production data.

**C.** That's a useful hypothesis, not a conclusion. Removing the row changed its contents and the number of rows together.

**D.** It could. The stack trace records the observed call path; it doesn't establish which input condition first caused the problem.

Answers and explanations, page 148

# A Read the situation

**LESSON 03 / GIVING ACTIONABLE CODE-REVIEW FEEDBACK**

Anchor a review comment in a changed call location, its observable effect, and a specific correction and test.

> **A rename that changed the failure path**
>
> A pull request is titled Rename export helper. The diff also moves the completion-notification call outside the success branch. Failed exports therefore notify users as though export completed. Reviewer Priya accepts the naming change but raises the behavior change with author Tomas. The review should request restoration of success-only notification and a failure-path test. The acceptable rename does not justify approving the unintended notification behavior.

People: Priya, Code reviewer; Tomas, Pull-request author.

## Read the record

Choose one answer per question using the facts above.

**1. What is the pull request titled?**

**A.** Release the importer
**B.** Rename export helper
**C.** Rewrite notification policy
**D.** Remove all export tests

**2. What additional change appears in the diff?**

**A.** Notifications are removed entirely
**B.** Notification moves outside the success branch
**C.** The export always succeeds
**D.** The failure branch is proven unreachable

**3. What part does Priya accept?**

**A.** The false success message
**B.** Missing failure coverage
**C.** Automatic production release
**D.** The naming change

Answers and explanations, page 118

# B Vocabulary in use

**CORE TERMS**

### pull request
Proposed code change submitted for review; abbreviated PR.

*review the pull request*

### review comment
Feedback attached to a proposed change.

*write a review comment*

### failure path
Execution path taken when an operation fails.

*cover the failure path*

### control flow
Order and conditions under which code executes.

*trace the control flow*

### completion notification
Message indicating an operation has finished successfully here.

*gate the completion notification*

### refactor
Internal restructuring intended to preserve observable behavior.

*separate a refactor from a behavior change*

### diff
Display of changes between versions.

*inspect the diff*

### success branch
Execution path entered when the relevant operation succeeds.

*keep the call in the success branch*

### call site
Location where a function or operation is invoked.

*identify the call site*

### side effect
Observable effect beyond a function's returned value.

*examine the side effect*

### behavior change
Alteration in what the system observably does.

*flag the behavior change*

### rename
Change to an identifier's name.

*approve the rename*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**pull request** PULL REQUEST

review the pull request

Keep the meaning precise: Proposed code change submitted for review; abbreviated PR.

**call site** CALL SITE

identify the call site

Keep the meaning precise: Location where a function or operation is invoked.

# B Precision and professional shorthand

**EXTEND YOUR RANGE**

### regression test

Test guarding against recurrence of a known defect.

*add a regression test*

### assertion

Test statement checking an expected condition.

*strengthen the assertion*

### blocking concern

Issue that must be resolved before approval under the review process.

*explain the blocking concern*

### nonblocking suggestion

Improvement that does not prevent approval under the process.

*label a nonblocking suggestion*

### review scope

Set of changes and concerns covered by a review.

*clarify review scope*

### intent

Purpose the author means a change to achieve.

*confirm the intent*

### observable behavior

Effect visible to a user or another component.

*preserve observable behavior*

### false success

Indication of success when the operation did not succeed.

*prevent false success*

### guard condition

Condition controlling whether an action may occur.

*restore the guard condition*

### patch revision

Updated version of a proposed change.

*request a patch revision*

### approval

Explicit acceptance at the relevant review stage.

*withhold approval pending correction*

### verification evidence

Results supporting a claim that a change behaves as intended.

*provide verification evidence*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**rename** RENAME

approve the rename

Keep the meaning precise: Change to an identifier's name.

**review scope** REVIEW SCOPE

clarify review scope

Keep the meaning precise: Set of changes and concerns covered by a review.

# C Phrases that move the work forward

**Acknowledge the acceptable part**

The helper name is clear.

**Describe the consequence**

A failed export can now send a completion notice.

**Ask about intent**

Was this control-flow change intentional?

**Make a concrete request**

Please restore the success-only condition.

**Specify the assertion**

The failure case should not send a completion notification.

**Clarify review status**

I cannot approve this behavior yet.

**Distinguish scope**

The diff does more than the title suggests.

**Keep verification separate**

I will review the revised behavior and test.

**Locate the concern**

This notification call has moved outside the success branch.

**Separate preference and defect**

My concern is behavior, not the naming choice.

**State the expected boundary**

The completion notice belongs on the success path.

**Request evidence**

Add a test for a failed export.

**Avoid personal criticism**

This change introduces a false-success path.

**Limit the objection**

I am not asking you to undo the rename.

**Request a revision**

Please update the patch and its description.

**Close constructively**

The rename can stay once the notification issue is resolved.

## Precision in real exchanges

**Purpose:** Separate preference and defect
**Useful wording:** My concern is behavior, not the naming choice.

**Purpose:** Ask about intent
**Useful wording:** Was this control-flow change intentional?

# D Choose the response that fits

Choose one response for each situation. Preserve the facts, the decision boundary, and the purpose of the exchange.

**1. Which comment is actionable?**

**A.** Restore the success-only notification and test that failure sends no completion notice

**B.** Your code is careless

**C.** Make this nicer

**D.** The name is fine, so everything is approved

**2. Why is this not only a refactor?**

**A.** Observable notification behavior changes

**B.** The helper has a new name

**C.** Review comments are present

**D.** The title is short

**3. A colleague needs the term for this meaning: Execution path entered when the relevant operation succeeds. Which term fits?**

**A.** guard condition

**B.** completion notification

**C.** success branch

**D.** blocking concern

# D Choose the response that fits / continued

**4. Rebuild the exchange from "A rename that changed the failure path". Priya: "Keep the clearer name. Restore the guard around the notification; that's the behavior I'm blocking on, not my naming preference." Tomas: [missing response] Priya: "Please add a failing-export case too. It should verify that no completion notice is sent, not merely that export was attempted." Which response belongs in the gap?**

**A.** Understood. The notification is a side effect we need to preserve correctly. I'll keep it dependent on a successful export.

**B.** Yes. That false success is a defect, not a style disagreement. I won't describe the current patch as an approved refactor.

**C.** I'll clarify the review scope. A small diff and a rename title don't establish that the actual behavior stayed the same.

**D.** I'll submit a patch revision for another review. Your acceptance of the name isn't approval of the whole change.

Answers and explanations, page 118

> **One useful distinction**
>
> A rename can be acceptable while the same diff contains a blocking defect. Moving the notification outside the success branch changes observable behavior, so it is not merely a naming preference or a behavior-preserving refactor.

# E Conversation

## A rename that changed the failure path

A rename that changed the failure path

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

rename / review scope / verification evidence / failure path / control flow / regression test / patch revision / review comment / false success / side effect

01 **Priya:** The helper name is clearer. Before I approve, though, can we look at the notification call you moved below the success branch?

02 **Tomas:** I titled this as a [1] _____. Are you concerned about the identifier or something the new position changes?

03 **Priya:** The position. A failed export can now reach that call, so the user could receive a completion notice for an unsuccessful export.

04 **Tomas:** I see the [2] _____ change. I pulled the call out while cleaning up the method, but that removes the success condition.

05 **Priya:** Keep the clearer name. Restore the guard around the notification; that's the behavior I'm blocking on, not my naming preference.

06 **Tomas:** Understood. The notification is a [3] _____ we need to preserve correctly. I'll keep it dependent on a successful export.

07 **Priya:** Please add a failing-export case too. It should verify that no completion notice is sent, not merely that export was attempted.

08 **Tomas:** So the [4] _____ needs a no-notification expectation. A test that only checks the helper call could pass with this defect still present.

09 **Priya:** Exactly. I'll point to the moved call and explain that consequence in the review so you don't have to infer the requested change.

10 **Tomas:** That makes the [5] _____ actionable. I can address the changed behavior without reopening the whole naming discussion.

Conversation continues on the next page.

# A rename that changed the failure path

**CONVERSATION / CONTINUED**

11 **Priya:** Also check the description against the final patch. Right now the title makes this sound like a purely mechanical change.

12 **Tomas:** I'll clarify the [6] _____. A small diff and a rename title don't establish that the actual behavior stayed the same.

13 **Priya:** Would you agree the current version isn't behavior-preserving? The failed export and the completion notice contradict each other.

14 **Tomas:** Yes. That [7] _____ is a defect, not a style disagreement. I won't describe the current patch as an approved refactor.

15 **Priya:** Once the guard and failure assertion are in place, send the updated change back with the relevant test result.

16 **Tomas:** I'll submit a [8] _____ for another review. Your acceptance of the name isn't approval of the whole change.

17 **Priya:** Make sure the new check would fail if that notification moved outside the success condition again. Otherwise it won't protect this behavior.

18 **Tomas:** I'll verify the [9] _____ catches that specific recurrence. Simply executing the failure branch isn't enough if the assertion misses the notice.

19 **Priya:** Good. My review remains blocked on corrected behavior and its check. I'm happy with the name; those are separate decisions.

20 **Tomas:** I'll return with [10] _____ for the correction. We can resolve the blocking comment after you've reviewed that, not from this conversation alone.

**CHECK THE GAPS**

**1.** rename / **2.** control flow / **3.** side effect / **4.** failure path / **5.** review comment / **6.** review scope / **7.** false success / **8.** patch revision / **9.** regression test / **10.** verification evidence

Why these answers fit, page 119

# F Check accuracy and scope

## A rename that changed the failure path

### Return to the case

A pull request is titled Rename export helper. The diff also moves the completion-notification call outside the success branch. Failed exports therefore notify users as though export completed. Reviewer Priya accepts the naming change but raises the behavior change with author Tomas. The review should request restoration of success-only notification and a failure-path test. The acceptable rename does not justify approving the unintended notification behavior.

Choose one answer per question. Use the case facts and keep any uncertainty or limits intact.

**1. What should the failure test assert?**

**A.** Production release is approved
**B.** No completion notice is sent
**C.** A completion notice always appears
**D.** Only the helper spelling is correct

**2. Which statement preserves Priya's position?**

**A.** The rename is acceptable; the notification behavior needs correction
**B.** Every line must be reverted
**C.** The author cannot write software
**D.** The test can be omitted because the rename is small

Answers and explanations, page 121

### Listen for the distinction

Swap roles for turns 5-10. Stress success-only notification and no notification after failure.

### Critique the change, not the author

A useful review points to behavior and asks for a verifiable change. Acknowledge acceptable parts without letting praise obscure a defect. Separating naming from functionality lets the author respond to a concrete concern instead of defending personal competence.

# F Say it / transfer to a new case

## Turn a vague objection into a review request

Complete the review summary while preserving the accepted rename and unresolved behavior issue.

Complete the four gaps. With a partner, read alternate roles, then switch. Studying alone, read both roles and pause before each response.

**WORD AND PHRASE BANK**

approval / failure / name / success

01 **Reviewer:** The helper [1] _____ is acceptable.

02 **Author:** The notification must remain on the [2] _____ path.

03 **Reviewer:** Add a test for the [3] _____ path.

04 **Author:** The current feedback is not yet [4] _____.

**CHECK THE GAPS**

**1.** name / **2.** success / **3.** failure / **4.** approval

Why these answers fit, page 121

### Rebuild the exchange

Choose the missing response. Read the completed three-turn exchange aloud, changing roles on the second reading.

**1. Rebuild the exchange from "A rename that changed the failure path". Priya: "Would you agree the current version isn't behavior-preserving? The failed export and the completion notice contradict each other." Tomas: [missing response] Priya: "Once the guard and failure assertion are in place, send the updated change back with the relevant test result." Which response belongs in the gap?**

**A.** Understood. The notification is a side effect we need to preserve correctly. I'll keep it dependent on a successful export.

**B.** I'll clarify the review scope. A small diff and a rename title don't establish that the actual behavior stayed the same.

**C.** Yes. That false success is a defect, not a style disagreement. I won't describe the current patch as an approved refactor.

**D.** I see the control flow change. I pulled the call out while cleaning up the method, but that removes the success condition.

Answers and explanations, page 122

# A Read the situation

**LESSON 04 / REQUESTING DEPENDENCY EVIDENCE**

Request the resolved dependency version, affected call sites, and migration evidence before claiming upgrade compatibility.

> **A direct upgrade with an indirect change**
>
> A fictional application uses ParcelKit directly. ParcelKit in turn uses TextCore. The proposed ParcelKit upgrade changes the resolved TextCore version. A change note marks a parsing entry point as deprecated. Developers Leila and Ben have not established whether the application or its dependencies use that entry point, whether it has been removed, or whether the replacement is compatible. The packages are not assumed to follow any particular versioning convention.

People: Leila, Developer proposing the upgrade; Ben, Developer checking dependency impact.

## Read the record

Choose one answer per question using the facts above.

**1. Which dependency does the application use directly?**

**A.** Neither
**B.** ParcelKit
**C.** TextCore only
**D.** Both are proven direct

**2. What changes indirectly?**

**A.** Every public interface
**B.** A confirmed production failure
**C.** The resolved TextCore version
**D.** The application name

**3. What does the note establish?**

**A.** A parsing entry point is deprecated
**B.** The replacement is fully compatible
**C.** The application definitely uses it
**D.** The entry point has definitely been removed

Answers and explanations, page 123

# B Vocabulary in use

**CORE TERMS**

### direct dependency
Package explicitly used or declared by the application here.

*upgrade a direct dependency*

### transitive dependency
Package brought in through another dependency.

*inspect a transitive dependency*

### dependency tree
Structure showing how packages depend on one another.

*inspect the dependency tree*

### resolved version
Specific version selected after dependency resolution.

*record the resolved version*

### lockfile
File recording resolved dependency selections for reproducibility.

*inspect the lockfile*

### version constraint
Rule limiting which dependency versions may be selected.

*check a version constraint*

### release note
Published description of changes in a release.

*read the release note*

### deprecation
Notice that an interface is discouraged or scheduled for replacement.

*investigate a deprecation*

### removal
Elimination of a previously available interface or behavior.

*verify removal separately*

### migration guide
Instructions describing transition to a changed interface.

*consult the migration guide*

### entry point
Named interface through which functionality is invoked.

*locate the parsing entry point*

### call site
Location that invokes a particular interface.

*search for affected call sites*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**direct dependency** DIRECT DEPENDENCY

upgrade a direct dependency

Keep the meaning precise: Package explicitly used or declared by the application here.

**version constraint** VERSION CONSTRAINT

check a version constraint

Keep the meaning precise: Rule limiting which dependency versions may be selected.

# B Precision and professional shorthand

**EXTEND YOUR RANGE**

### public interface

Supported surface exposed for other components to use.

*compare the public interface*

### breaking change

Change that can invalidate previously supported use.

*identify a breaking change*

### compatibility

Ability to work with the relevant existing consumers and behavior.

*verify compatibility*

### semantic versioning

Version convention relating version numbers to public-interface changes.

*confirm semantic versioning policy*

### major version

Leading version component under a stated convention.

*interpret the major version*

### minor version

Version component whose meaning depends on the adopted convention.

*check minor-version guarantees*

### patch version

Version component commonly used for fixes under a stated convention.

*verify patch-version policy*

### pin

Fixed dependency selection restricting version movement.

*review the version pin*

### changelog

Maintained record of notable changes between versions.

*compare the changelog*

### replacement interface

Interface intended to supersede another.

*assess the replacement interface*

### resolution

Process selecting concrete dependency versions.

*reproduce dependency resolution*

### upgrade evidence

Records and checks supporting an upgrade assessment.

*assemble upgrade evidence*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**call site** CALL SITE

search for affected call sites

Keep the meaning precise: Location that invokes a particular interface.

**major version** MAJOR VERSION

interpret the major version

Keep the meaning precise: Leading version component under a stated convention.

# C Phrases that move the work forward

**Identify the direct dependency**
The application uses ParcelKit directly.

**Identify the indirect dependency**
TextCore comes through ParcelKit.

**Name the additional change**
The resolved TextCore version also changes.

**Request the actual selection**
Which version does the lockfile record?

**Ask for the relationship**
Can you show the dependency tree?

**Quote the narrow finding**
The note marks the parsing entry point as deprecated.

**Avoid an unsupported upgrade**
Deprecated does not by itself mean removed.

**Ask about usage**
Do we have any affected call sites?

**Include indirect callers**
Check whether ParcelKit invokes it as well.

**Request migration detail**
What does the migration guide say about the replacement?

**Separate availability and behavior**
An available replacement is not automatically compatible.

**Qualify the version inference**
We need the package's actual versioning policy.

**Request verification**
Which checks exercise the affected parsing behavior?

**Keep the decision open**
We do not have enough evidence to claim compatibility.

**State the pending work**
Usage and replacement behavior still need review.

**Close the request**
Please attach the resolved versions and the relevant findings.

## Precision in real exchanges

**Purpose:** Request the actual selection
**Useful wording:** Which version does the lockfile record?

**Purpose:** Ask for the relationship
**Useful wording:** Can you show the dependency tree?

# D Choose the response that fits

Choose one response for each situation. Preserve the facts, the decision boundary, and the purpose of the exchange.

**1. Which statement is accurate?**

**A.** TextCore is transitive through ParcelKit
**B.** TextCore cannot affect the application
**C.** ParcelKit is only transitive
**D.** All transitive dependencies are unused

**2. What should be checked first about the selected version?**

**A.** The latest version mentioned in a blog
**B.** The resolved dependency record
**C.** Only the pull-request title
**D.** An unrelated package's policy

**3. A colleague needs the term for this meaning: Specific version selected after dependency resolution. Which term fits?**

**A.** compatibility
**B.** removal
**C.** resolved version
**D.** changelog

# D Choose the response that fits / continued

**4. Rebuild the exchange from "A direct upgrade with an indirect change". Leila: "The release notes mark a parsing entry point deprecated. I don't yet know whether our code or ParcelKit's internals call it." Ben: [missing response] Leila: "I'll remove my claim that the upgrade deletes a parser we use. Deprecation, removal, and our usage are three different questions." Which response belongs in the gap?**

**A.** Then do not assume semantic versioning guarantees. We can check the actual policy, but a familiar number pattern is not evidence that its promised compatibility rules apply.

**B.** Then we need to review the transitive dependency change as well. The application uses ParcelKit directly, but that does not make TextCore irrelevant to its behavior.

**C.** Please include the lockfile and the relevant dependency relationship. A constraint can allow several versions, while the resolved record tells us which one this change actually selects.

**D.** Keep that as a deprecation finding, not a removal finding. The note deserves investigation, but its label alone does not tell us whether this application is affected.

Answers and explanations, page 123

> **One useful distinction**
>
> Direct describes the application-to-ParcelKit relationship. Transitive describes the application-to-TextCore relationship through ParcelKit. The dependency actually selected matters, not just the version written in the top-level upgrade request.

# E Conversation

## A direct upgrade with an indirect change

A direct upgrade with an indirect change

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

call site / public interface / dependency tree / deprecation / compatibility / semantic versioning / lockfile / upgrade evidence / migration guide / transitive dependency

01 **Leila:** The ParcelKit upgrade also selects a different TextCore version. The request title doesn't show that indirect change, but the resolved list does.

02 **Ben:** Then we need to review the [1] _____ change as well. The application uses ParcelKit directly, but that does not make TextCore irrelevant to its behavior.

03 **Leila:** I can attach the package constraints and actual selected versions. Which record do you need to see what this build will really use?

04 **Ben:** Please include the [2] _____ and the relevant dependency relationship. A constraint can allow several versions, while the resolved record tells us which one this change actually selects.

05 **Leila:** The release notes mark a parsing entry point deprecated. I don't yet know whether our code or ParcelKit's internals call it.

06 **Ben:** Keep that as a [3] _____ finding, not a removal finding. The note deserves investigation, but its label alone does not tell us whether this application is affected.

# E Conversation / continued

07 **Leila:** I'll remove my claim that the upgrade deletes a parser we use. Deprecation, removal, and our usage are three different questions.

08 **Ben:** Correct. Start by locating any relevant [4] _____, including indirect calls through ParcelKit. An absence of direct application calls would not automatically rule out an indirect dependency on that behavior.

09 **Leila:** The replacement has a similar name. Before I switch anything, where should we check whether its inputs and error behavior actually match?

10 **Ben:** Consult the [5] _____ first. Similar names do not establish identical inputs, outputs, error behavior, or handling of edge cases. We need the relevant documented differences.

Conversation continues on the next page.

# A direct upgrade with an indirect change

**CONVERSATION / CONTINUED**

11 **Leila:** The version looks familiar, but we haven't checked either package's versioning policy. I shouldn't infer its compatibility promise from the number alone.

12 **Ben:** Then do not assume [6] _____ guarantees. We can check the actual policy, but a familiar number pattern is not evidence that its promised compatibility rules apply.

13 **Leila:** I'll trace ParcelKit to TextCore in the review notes. That should explain why a package we don't call directly still matters to this upgrade.

14 **Ben:** Good. The [7] _____ will help explain why TextCore belongs in the discussion even though the request is named after ParcelKit.

15 **Leila:** The existing suite passes, but I haven't shown that it exercises the affected parsing entry point. What evidence should the review request?

16 **Ben:** Ask which checks exercise the relevant [8] _____. A passing suite cannot establish coverage of a path it never executes.

17 **Leila:** I'll name that coverage gap instead of saying all tests passed therefore compatible. A green suite could simply miss the changed path.

18 **Ben:** Exactly. We need [9] _____ tied to this change. It should show what was checked and what remains unknown, not just that an upgrade command succeeded.

19 **Leila:** My revised summary will separate resolved versions, deprecation status, usage, and replacement behavior. The last two still need investigation.

20 **Ben:** That is accurate. Keep [10] _____ unverified for now. Once those findings are available, we can discuss the upgrade on evidence rather than infer its impact from the headline.

**CHECK THE GAPS**

**1.** transitive dependency / **2.** lockfile / **3.** deprecation / **4.** call site / **5.** migration guide / **6.** semantic versioning / **7.** dependency tree / **8.** public interface / **9.** upgrade evidence / **10.** compatibility

Why these answers fit, page 124

# F Check accuracy and scope

## A direct upgrade with an indirect change

### Return to the case

A fictional application uses ParcelKit directly. ParcelKit in turn uses TextCore. The proposed ParcelKit upgrade changes the resolved TextCore version. A change note marks a parsing entry point as deprecated. Developers Leila and Ben have not established whether the application or its dependencies use that entry point, whether it has been removed, or whether the replacement is compatible. The packages are not assumed to follow any particular versioning convention.

Choose one answer per question. Use the case facts and keep any uncertainty or limits intact.

**1. What does deprecated not establish?**

**A.** That the interface deserves attention
**B.** That the entry point has been removed
**C.** That a transition may need review
**D.** That a note exists

**2. Which compatibility claim is justified now?**

**A.** The replacement is guaranteed identical
**B.** Every minor upgrade is safe
**C.** No call-site review is needed
**D.** Compatibility remains unverified

Answers and explanations, page 126

### Listen for the distinction

Swap roles and repeat turns 5-12. Keep deprecated, removed, used, and compatible separate.

### Ask what changed below the headline

An upgrade can affect more than the package named in the request. Asking for the resolved dependency tree and affected calls is a practical evidence request, not resistance to maintenance. Avoid translating deprecated into removed or compatible without checking the actual contract.

# F Say it / transfer to a new case

## Keep the dependency chain and uncertainty intact

Complete the upgrade handoff without turning deprecation into confirmed removal or compatibility.

Complete the four gaps. With a partner, read alternate roles, then switch. Studying alone, read both roles and pause before each response.

**WORD AND PHRASE BANK**

deprecated / transitive / unverified / direct

01 **Developer:** ParcelKit is the [1] _____ dependency.

02 **Reviewer:** TextCore is the [2] _____ dependency.

03 **Developer:** The parsing entry point is marked [3] _____.

04 **Reviewer:** Replacement compatibility remains [4] _____.

**CHECK THE GAPS**

**1.** direct / **2.** transitive / **3.** deprecated / **4.** unverified

Why these answers fit, page 126

### Rebuild the exchange

Choose the missing response. Read the completed three-turn exchange aloud, changing roles on the second reading.

**1. Rebuild the exchange from "A direct upgrade with an indirect change". Leila: "I'll trace ParcelKit to TextCore in the review notes. That should explain why a package we don't call directly still matters to this upgrade." Ben: [missing response] Leila: "The existing suite passes, but I haven't shown that it exercises the affected parsing entry point. What evidence should the review request?" Which response belongs in the gap?**

**A.** Good. The dependency tree will help explain why TextCore belongs in the discussion even though the request is named after ParcelKit.

**B.** Then we need to review the transitive dependency change as well. The application uses ParcelKit directly, but that does not make TextCore irrelevant to its behavior.

**C.** Ask which checks exercise the relevant public interface. A passing suite cannot establish coverage of a path it never executes.

**D.** Consult the migration guide first. Similar names do not establish identical inputs, outputs, error behavior, or handling of edge cases. We need the relevant documented differences.

Answers and explanations, page 127

# Review 2 / Meaning and context

Use language from the lessons you have completed. Choose one answer per question.

> **Case file**
>
> Revisit "Giving actionable code-review feedback" and "Requesting dependency evidence". Choose precise terminology, then reconstruct the missing responses using both neighboring turns. The final two questions also revisit earlier conversations.

**1. A colleague needs the term for this meaning: Feedback attached to a proposed change. Which term fits?**

**A.** false success
**B.** review comment
**C.** assertion
**D.** side effect

**2. A colleague needs the term for this meaning: Notice that an interface is discouraged or scheduled for replacement. Which term fits?**

**A.** patch version
**B.** deprecation
**C.** public interface
**D.** direct dependency

**3. Rebuild the exchange from "The empty date is a suspect, not yet the cause". Nia: "Should I reduce the file further first? Two rows would be easier to send around, but that version currently passes." Ellis: [missing response] Nia: "The trace points into the parser. Could that still be downstream of a problem elsewhere in the import?" Which response belongs in the gap?**

**A.** It could. The stack trace records the observed call path; it doesn't establish which input condition first caused the problem.
**B.** Keep the failing input fixture intact. A smaller passing file isn't a smaller reproduction of the failure we're investigating.
**C.** Use the suitable nonproduction environment and keep its configuration in the record. We need evidence without changing production data.
**D.** That's a useful hypothesis, not a conclusion. Removing the row changed its contents and the number of rows together.

# Review 2 / Meaning and context / continued

**4. Rebuild the exchange from "A direct upgrade with an indirect change". Leila: "I'll name that coverage gap instead of saying all tests passed therefore compatible. A green suite could simply miss the changed path." Ben: [missing response] Leila: "My revised summary will separate resolved versions, deprecation status, usage, and replacement behavior. The last two still need investigation." Which response belongs in the gap?**

**A.** Consult the migration guide first. Similar names do not establish identical inputs, outputs, error behavior, or handling of edge cases. We need the relevant documented differences.

**B.** Exactly. We need upgrade evidence tied to this change. It should show what was checked and what remains unknown, not just that an upgrade command succeeded.

**C.** Please include the lockfile and the relevant dependency relationship. A constraint can allow several versions, while the resolved record tells us which one this change actually selects.

**D.** Then we need to review the transitive dependency change as well. The application uses ParcelKit directly, but that does not make TextCore irrelevant to its behavior.

Answers and explanations, page 151

# A Read the situation

**LESSON 05 / EXPLAINING WHAT A TEST ACTUALLY EXERCISES**

Explain the limits of a success-only test double and distinguish a failure-path unit test from real-adapter integration coverage.

### A green test that never sees rejection

A save-operation unit test uses a storage double that always succeeds and checks the success message. A previous bug showed success after storage rejected the save. The current test never produces rejection, and the real storage adapter is untested. Developers Owen and Farah need to configure a rejection case, assert that false success does not appear, and keep real-adapter integration coverage as a separate unresolved concern.

People: Owen, Developer maintaining the test; Farah, Developer reviewing coverage.

## Read the record

Choose one answer per question using the facts above.

**1. What does the storage double currently do?**

**A.** Always succeeds
**B.** Rejects every request
**C.** Uses the real adapter
**D.** Alternates unpredictably

**2. What did the previous bug show?**

**A.** A missing helper name
**B.** A proven adapter timeout
**C.** Success after storage rejected the save
**D.** Failure after a confirmed save

**3. What remains separately untested?**

**A.** The existence of the double
**B.** Every application feature
**C.** The real storage adapter
**D.** Whether the test shows a success message

Answers and explanations, page 128

# B Vocabulary in use

**CORE TERMS**

### unit test
Focused test of a small behavior or component.
*extend the unit test*

### test double
Substitute for a real dependency during testing.
*configure the test double*

### stub
Test substitute supplying predetermined responses.
*use a success stub*

### mock
Test substitute that can record or check interactions.
*inspect mock interactions*

### fixture
Prepared state or data used by a test.
*arrange the fixture*

### success response
Result indicating an operation succeeded.
*return a success response*

### rejection
Failed outcome reported by the storage operation here.
*simulate a rejection*

### exception
Error signal raised during execution under the relevant model.
*configure an exception*

### assertion
Explicit check of an expected test condition.
*add a negative assertion*

### negative assertion
Check that an unwanted event or state does not occur.
*verify a negative assertion*

### test isolation
Separation of a tested unit from unrelated dependencies.
*preserve test isolation*

### integration test
Test of interaction between connected components.
*add an integration test*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**unit test** UNIT TEST

extend the unit test

Keep the meaning precise: Focused test of a small behavior or component.

**success response** SUCCESS RESPONSE

return a success response

Keep the meaning precise: Result indicating an operation succeeded.

# B Precision and professional shorthand

**EXTEND YOUR RANGE**

### adapter
Component translating between an application and an external interface.
*test the storage adapter*

### branch coverage
Measure of which decision outcomes a test set executes.
*examine branch coverage*

### happy path
Expected successful execution path.
*cover the happy path*

### error path
Execution path handling an unsuccessful operation.
*exercise the error path*

### false negative
Test result that misses a defect which is present.
*avoid a false negative*

### test oracle
Rule or reference used to judge the expected result.
*define the test oracle*

### arrange
Prepare the conditions for a test.
*arrange a rejection case*

### act
Execute the behavior being tested.
*act on the save request*

### assert
Check the observed result against the expectation.
*assert no false success*

### interaction verification
Check of how a component called its dependencies.
*separate interaction verification*

### behavioral verification
Check of the externally observable result.
*perform behavioral verification*

### coverage gap
Relevant behavior not exercised by the current tests.
*report the coverage gap*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**integration test** INTEGRATION TEST

add an integration test

Keep the meaning precise: Test of interaction between connected components.

**false negative** FALSE NEGATIVE

avoid a false negative

Keep the meaning precise: Test result that misses a defect which is present.

# C Phrases that move the work forward

**Describe the current setup**
The storage double always succeeds.

**Name the tested path**
This test exercises the happy path.

**Recall the defect**
The bug showed success after a rejected save.

**Identify the gap**
The current test never triggers rejection.

**Request a configured outcome**
Make the double reject in a separate case.

**Specify the check**
Assert that no success message appears after rejection.

**Preserve existing coverage**
Keep the successful-save case too.

**Separate call and result**
A storage call alone does not prove the right user message.

**Name the substitute**
This is a double, not the real adapter.

**Limit the conclusion**
The result applies to the simulated rejection case.

**Request another test level**
The real adapter needs separate integration coverage.

**Avoid a blanket claim**
Green tests do not establish all storage behavior.

**State the open question**
We have not verified the real adapter.

**Ask about expectations**
What observable result should this case check?

**Request evidence**
Please include the configured outcome and assertion.

**Close accurately**
The new unit case can close one gap, not every storage risk.

## Precision in real exchanges

**Purpose:** Identify the gap
**Useful wording:** The current test never triggers rejection.

**Purpose:** Request a configured outcome
**Useful wording:** Make the double reject in a separate case.

# D Choose the response that fits

Choose one response for each situation. Preserve the facts, the decision boundary, and the purpose of the exchange.

**1. Why does the current test miss the prior bug?**

**A.** Every unit test uses production storage
**B.** Success messages cannot be tested
**C.** Its double never rejects
**D.** Its name contains save

**2. Which addition targets the defect?**

**A.** Check only that storage is called
**B.** Remove all assertions
**C.** Configure rejection and assert no success message
**D.** Rename the success test only

**3. A colleague needs the term for this meaning: Test substitute that can record or check interactions. Which term fits?**

**A.** mock
**B.** assertion
**C.** assert
**D.** happy path

# D Choose the response that fits / continued

**4. Rebuild the exchange from "A green test that never sees rejection". Owen: "I could assert that storage was called once. Would that distinguish the corrected application from the old one?" Farah: [missing response] Owen: "Right, the old version called it but treated the rejected save as completed. The call count wouldn't catch that." Which response belongs in the gap?**

**A.** Exactly. Interaction verification answers whether the dependency was called as expected; it doesn't replace checking what the user is told afterward.

**B.** Not by itself. Add a negative assertion that the success message is absent after rejection. Both versions might call storage once.

**C.** Then this checks the happy path. Where does the test make storage reject the save, which is what exposed the old bug?

**D.** Configure the test double for that outcome in a separate case. Keep the successful-save case; it still checks useful behavior.

Answers and explanations, page 128

### One useful distinction

A double that always succeeds can support a success-path assertion, but it cannot exercise rejection handling. Configure a failing response and check the user-visible consequence relevant to the known bug.

# E Conversation

## A green test that never sees rejection

A green test that never sees rejection

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

happy path / integration test / behavioral verification / negative assertion / Interaction verification / test double / coverage gap / arrange / unit test / success response

01 **Owen:** The save test passes. I was about to mark the old false-success bug covered, but the storage substitute always returns success.

02 **Farah:** Then this checks the [1] _____. Where does the test make storage reject the save, which is what exposed the old bug?

03 **Owen:** It doesn't. I have no rejection case in this fixture, so the green result can't tell us whether we still ignore rejection.

04 **Farah:** Configure the [2] _____ for that outcome in a separate case. Keep the successful-save case; it still checks useful behavior.

05 **Owen:** I could assert that storage was called once. Would that distinguish the corrected application from the old one?

06 **Farah:** Not by itself. Add a [3] _____ that the success message is absent after rejection. Both versions might call storage once.

07 **Owen:** Right, the old version called it but treated the rejected save as completed. The call count wouldn't catch that.

08 **Farah:** Exactly. [4] _____ answers whether the dependency was called as expected; it doesn't replace checking what the user is told afterward.

09 **Owen:** This still crosses the storage interface. Could I label it an integration check once the failure case is in place?

10 **Farah:** Not as evidence for the real adapter. An [5] _____ of that interaction needs the relevant actual components; your adapter remains replaced.

Conversation continues on the next page.

# A green test that never sees rejection

**CONVERSATION / CONTINUED**

11 **Owen:** Then the description should say simulated storage rejection and application response. We haven't exercised the actual adapter at all.

12 **Farah:** Keep that [6] _____ explicit. Better simulated coverage doesn't silently close a different untested boundary.

13 **Owen:** I'll prepare the rejected result, execute the save, then inspect the displayed message. The fixture should make the unsuccessful outcome obvious.

14 **Farah:** Yes: [7] _____ the rejection, act on the save, and assert no success indication. Reviewers should see that setup without hunting through defaults.

15 **Owen:** I'll retain the original check separately. It confirms that the application shows the intended message when the substitute reports a successful save.

16 **Farah:** That's a bounded claim about the [8] _____ case. The misleading part was describing it as proof of the complete storage workflow.

17 **Owen:** After adding rejection, I'll report exactly which outcome was simulated and which visible behavior was checked, alongside the real-adapter gap.

18 **Farah:** That is useful [9] _____. Report the assertion result, not a blanket statement that storage is fully tested.

19 **Owen:** The patch is still pending review. I'll attach both test results when they're available, without inventing a successful run in the handoff.

20 **Farah:** Good. A targeted [10] _____ can catch this application defect while leaving the real integration question clearly open.

**CHECK THE GAPS**

**1.** happy path / **2.** test double / **3.** negative assertion / **4.** Interaction verification / **5.** integration test / **6.** coverage gap / **7.** arrange / **8.** success response / **9.** behavioral verification / **10.** unit test

Why these answers fit, page 129

# F Check accuracy and scope

## A green test that never sees rejection

### Return to the case

A save-operation unit test uses a storage double that always succeeds and checks the success message. A previous bug showed success after storage rejected the save. The current test never produces rejection, and the real storage adapter is untested. Developers Owen and Farah need to configure a rejection case, assert that false success does not appear, and keep real-adapter integration coverage as a separate unresolved concern.

Choose one answer per question. Use the case facts and keep any uncertainty or limits intact.

**1. What does the rejection unit test not establish?**

**A.** Correct behavior of the real storage adapter
**B.** Behavior under the configured double response
**C.** Whether the assertion passes in that case
**D.** Whether the fixture rejects

**2. Which report is precise?**

**A.** Rejection cannot be simulated
**B.** Success-path coverage exists; rejection and real-adapter coverage need work
**C.** All storage behavior is verified
**D.** A green test proves no bugs remain

Answers and explanations, page 131

### Listen for the distinction

Swap roles and repeat turns 5-12. Distinguish a no-success-message assertion from checking a dependency call.

### A passing result needs a scope statement

Green is a useful status, but it is not a complete explanation. Name the behavior the test actually exercises and the dependencies it replaces. This allows a colleague to request missing coverage without denying the value of existing tests.

# F Say it / transfer to a new case

## Name the test boundary

Complete the review note about the current test and the proposed rejection case.

Complete the four gaps. With a partner, read alternate roles, then switch. Studying alone, read both roles and pause before each response.

**WORD AND PHRASE BANK**

succeeds / success / rejection / untested

01 **Developer:** The existing double always [1] _____.

02 **Reviewer:** The new case must trigger [2] _____.

03 **Developer:** The assertion should prevent a false [3] _____ message.

04 **Reviewer:** The real adapter remains [4] _____.

**CHECK THE GAPS**

**1.** succeeds / **2.** rejection / **3.** success / **4.** untested

Why these answers fit, page 131

### Rebuild the exchange

Choose the missing response. Read the completed three-turn exchange aloud, changing roles on the second reading.

**1. Rebuild the exchange from "A green test that never sees rejection". Owen: "I'll prepare the rejected result, execute the save, then inspect the displayed message. The fixture should make the unsuccessful outcome obvious." Farah: [missing response] Owen: "I'll retain the original check separately. It confirms that the application shows the intended message when the substitute reports a successful save." Which response belongs in the gap?**

**A.** Not as evidence for the real adapter. An integration test of that interaction needs the relevant actual components; your adapter remains replaced.

**B.** Yes: arrange the rejection, act on the save, and assert no success indication. Reviewers should see that setup without hunting through defaults.

**C.** That's a bounded claim about the success response case. The misleading part was describing it as proof of the complete storage workflow.

**D.** Then this checks the happy path. Where does the test make storage reject the save, which is what exposed the old bug?

Answers and explanations, page 132

# A Read the situation

**LESSON 06 / COMPARING PERFORMANCE TRADEOFFS FAIRLY**

Compare measured latency and memory under matched conditions while separating median improvement from budget compliance and production claims.

### Fifteen milliseconds faster, forty megabytes over budget

In a controlled benchmark using the same input and machine, implementation A has median latency of 40 milliseconds and memory use of 80 megabytes. B has median latency of 25 milliseconds and memory use of 160 megabytes. The memory budget is 120 megabytes. Developers Sora and Malik discuss the tradeoff. Tail latency and production behavior have not been measured. B reduces the measured median by 15 milliseconds, or 37.5 percent relative to A, but exceeds the memory budget by 40 megabytes.

People: Sora, Developer presenting the benchmark; Malik, Developer reviewing the tradeoff.

## Read the record

Choose one answer per question using the facts above.

**1. Which conditions are matched?**

**A.** Tail-latency percentiles
**B.** Input and machine
**C.** Production traffic and every user
**D.** All future workloads

**2. How far is B over the memory budget?**

**A.** 80 megabytes
**B.** It is under budget
**C.** 40 megabytes
**D.** 15 megabytes

**3. What is unmeasured?**

**A.** The budget difference
**B.** Tail latency and production behavior
**C.** The stated median values
**D.** The stated memory values

Answers and explanations, page 133

# B Vocabulary in use

**CORE TERMS**

### benchmark
Structured measurement under specified conditions.

*run a benchmark*

### median
Middle ordered value, or mean of the two middle values for an even sample.

*report median latency*

### percentile
Value below which a stated percentage of observations falls.

*report a latency percentile*

### throughput
Amount of work completed per unit of time.

*measure throughput separately*

### resource budget
Allowed limit for a specified resource.

*respect the resource budget*

### overrun
Amount by which usage exceeds a limit.

*quantify the budget overrun*

### latency
Time taken for a request or operation.

*measure latency*

### tail latency
Slower-end behavior in a latency distribution.

*measure tail latency*

### p95
Ninety-fifth percentile of a measured distribution.

*distinguish p95 from the median*

### memory footprint
Amount of memory used under the stated conditions.

*compare the memory footprint*

### headroom
Remaining capacity below a stated limit.

*calculate memory headroom*

### baseline
Reference implementation or measurement for comparison.

*use A as the baseline*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**benchmark** BENCHMARK

run a benchmark

Keep the meaning precise: Structured measurement under specified conditions.

**p95** P95

distinguish p95 from the median

Keep the meaning precise: Ninety-fifth percentile of a measured distribution.

# B Precision and professional shorthand

EXTEND YOUR RANGE

### relative reduction

Decrease expressed as a fraction of the original value.

*calculate the relative reduction*

### absolute difference

Direct numerical subtraction between comparable measurements.

*report the absolute difference*

### tradeoff

Gain in one property accompanied by a cost in another.

*explain the tradeoff*

### workload

Type and amount of work applied to a system.

*describe the workload*

### sample

Set of observations used in an analysis.

*document the sample*

### distribution

Pattern of observed values across a range.

*inspect the latency distribution*

### outlier

Observation unusually distant from most other values.

*investigate an outlier*

### measurement noise

Variation not attributable to the intended comparison alone.

*account for measurement noise*

### controlled conditions

Specified conditions held consistent during comparison.

*preserve controlled conditions*

### production behavior

Behavior under actual live operating conditions.

*avoid assuming production behavior*

### bottleneck

Constraint limiting overall system performance.

*identify the bottleneck*

### budget compliance

Whether measured usage satisfies the stated limit.

*check budget compliance*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**baseline** BASELINE

use A as the baseline

Keep the meaning precise: Reference implementation or measurement for comparison.

**sample** SAMPLE

document the sample

Keep the meaning precise: Set of observations used in an analysis.

# C Phrases that move the work forward

**Name the conditions**

Both runs use the same input and machine.

**State A**

A has a forty-millisecond median and uses eighty megabytes.

**State B**

B has a twenty-five-millisecond median and uses one hundred sixty megabytes.

**Report the absolute gain**

B lowers the median by fifteen milliseconds.

**Name the baseline**

That is a thirty-seven-point-five percent reduction relative to A.

**Report the cost**

B uses twice A's measured memory.

**State the constraint**

The memory budget is one hundred twenty megabytes.

**Calculate the overrun**

B exceeds that budget by forty megabytes.

**Calculate the headroom**

A remains forty megabytes below the limit.

**Avoid a vague verdict**

B is faster on this median measure, not automatically better overall.

**Limit the metric**

The median does not tell us the tail latency.

**Name the missing evidence**

We have not measured production behavior.

**Separate throughput**

These figures do not establish throughput.

**Request a balanced summary**

Put the latency gain and memory cost together.

**Keep the decision bounded**

The measured gain does not remove the budget constraint.

**Close without overclaiming**

We have a documented tradeoff, not a production guarantee.

## Precision in real exchanges

**Purpose:** Report the absolute gain
**Useful wording:** B lowers the median by fifteen milliseconds.

**Purpose:** Name the baseline
**Useful wording:** That is a thirty-seven-point-five percent reduction relative to A.

# D Choose the response that fits

Choose one response for each situation. Preserve the facts, the decision boundary, and the purpose of the exchange.

**1. What is B's median reduction relative to A?**

**A.** 100 percent
**B.** 37.5 percent
**C.** 60 percent
**D.** 15 percent

**2. Which budget statement is correct?**

**A.** B is 80 below budget
**B.** A is 40 above budget
**C.** A is 40 below and B is 40 above
**D.** Both are within budget

**3. A colleague needs the term for this meaning: Slower-end behavior in a latency distribution. Which term fits?**

**A.** controlled conditions
**B.** tail latency
**C.** resource budget
**D.** tradeoff

# D Choose the response that fits / continued

**4. Rebuild the exchange from "Fifteen milliseconds faster, forty megabytes over budget". Sora: "For the percentage, I subtracted twenty-five from forty and divided the fifteen-millisecond decrease by forty. That gives thirty-seven-point-five percent." Malik: [missing response] Sora: "Would faster be too loose in the headline? I measured latency, not how many requests per second either version handles." Which response belongs in the gap?**

**A.** That is useful benchmark evidence. Put the memory figures beside it before we call B the better option. The resource budget is part of this decision.

**B.** That is forty megabytes of headroom under the stated budget. It is a different claim from saying A has more memory available under all possible workloads.

**C.** Yes, that is the relative reduction relative to A. Keep the baseline explicit so nobody divides by twenty-five and reports a different quantity under the same label.

**D.** Correct. Tail latency remains unmeasured. Do not turn a better middle value into a claim about every request or a specific percentile we did not record.

Answers and explanations, page 133

### One useful distinction

B reduces the median from 40 to 25 milliseconds: a 15-millisecond difference and a 37.5 percent reduction relative to A. It uses twice A's measured memory, but doubling memory is not the same as doubling speed.

# E Conversation

## Fifteen milliseconds faster, forty megabytes over budget

Fifteen milliseconds faster, forty megabytes over budget

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

relative reduction / controlled conditions / absolute difference / Production behavior / budget compliance / tradeoff / Tail latency / memory footprint / headroom / benchmark

01 **Sora:** B's measured median is twenty-five milliseconds versus A's forty on the same input and machine. I'd like to propose B, but its memory figure complicates that.

02 **Malik:** That is useful [1] _____ evidence. Put the memory figures beside it before we call B the better option. The resource budget is part of this decision.

03 **Sora:** A uses eighty megabytes; B uses a hundred sixty. The budget is a hundred twenty, so B is forty over even in this measured case.

04 **Malik:** Right. Its [2] _____ is twice A's and forty megabytes over budget. The latency improvement does not cancel that constraint.

05 **Sora:** For the percentage, I subtracted twenty-five from forty and divided the fifteen-millisecond decrease by forty. That gives thirty-seven-point-five percent.

06 **Malik:** Yes, that is the [3] _____ relative to A. Keep the baseline explicit so nobody divides by twenty-five and reports a different quantity under the same label.

07 **Sora:** Would faster be too loose in the headline? I measured latency, not how many requests per second either version handles.

08 **Malik:** Prefer the precise [4] _____ and percentage latency reduction. Those statements tell readers exactly which measurement changed without suggesting an unmeasured throughput result.

09 **Sora:** We didn't collect tail percentiles. A lower median could coexist with worse slow requests, so I can't claim improvement for every request.

10 **Malik:** Correct. [5] _____ remains unmeasured. Do not turn a better middle value into a claim about every request or a specific percentile we did not record.

Conversation continues on the next page.

# Fifteen milliseconds faster, forty megabytes over budget

**CONVERSATION / CONTINUED**

11 **Sora:** A is forty megabytes below the same budget. I'll keep that separate from B's forty-megabyte excess rather than call both numbers spare memory.

12 **Malik:** That is forty megabytes of [6] _____ under the stated budget. It is a different claim from saying A has more memory available under all possible workloads.

13 **Sora:** These are benchmark runs, not live measurements. Should I put that boundary alongside the numbers instead of burying it in a note?

14 **Malik:** Yes. [7] _____ is not established by these figures. Name that limit instead of relying on readers to infer it from the word benchmark.

15 **Sora:** Then the proposal needs both sides: fifteen milliseconds lower median, twice the memory, and B above the present limit.

16 **Malik:** That presents the [8] _____ fairly. It acknowledges a real improvement without treating one favorable number as a complete engineering decision.

17 **Sora:** I could request a changed memory budget, but that wouldn't make this run compliant with the current one. It would be a separate decision.

18 **Malik:** Exactly. Current [9] _____ is clear: A is below the limit and B is above it. A hypothetical future budget should not rewrite today's comparison.

19 **Sora:** I'll report the matched conditions, latency decrease, memory figures, and unmeasured tail and live behavior. We still need to decide what to investigate next.

20 **Malik:** Good. Keep the [10] _____ visible alongside that summary. We can make a reasoned next decision without pretending the benchmark answers questions it never measured.

**CHECK THE GAPS**

**1.** benchmark / **2.** memory footprint / **3.** relative reduction / **4.** absolute difference / **5.** Tail latency / **6.** headroom / **7.** Production behavior / **8.** tradeoff / **9.** budget compliance / **10.** controlled conditions

Why these answers fit, page 134

# F Check accuracy and scope

## Fifteen milliseconds faster, forty megabytes over budget

### Return to the case

In a controlled benchmark using the same input and machine, implementation A has median latency of 40 milliseconds and memory use of 80 megabytes. B has median latency of 25 milliseconds and memory use of 160 megabytes. The memory budget is 120 megabytes. Developers Sora and Malik discuss the tradeoff. Tail latency and production behavior have not been measured. B reduces the measured median by 15 milliseconds, or 37.5 percent relative to A, but exceeds the memory budget by 40 megabytes.

Choose one answer per question. Use the case facts and keep any uncertainty or limits intact.

**1. Which claim exceeds the evidence?**

**A.** B has a lower measured median

**B.** B uses more measured memory

**C.** The benchmark used the same machine

**D.** B improves every production request

**2. Which summary is balanced?**

**A.** B is universally superior

**B.** A is faster in the supplied test

**C.** Memory no longer matters because latency improved

**D.** B lowers median latency but exceeds the memory budget; tail and production remain unmeasured

Answers and explanations, page 136

### Listen for the distinction

Swap roles for turns 5-12. State the fifteen-millisecond decrease, 37.5 percent reduction, and forty-megabyte headroom accurately.

### Do not hide a constraint inside a headline

Faster is incomplete when a resource budget matters. State the metric, conditions, comparison baseline, and constraint together. A colleague can acknowledge a genuine latency improvement while rejecting the claim that it settles the overall implementation decision.

# F Say it / transfer to a new case

## Report both sides of the comparison

Use the supplied numbers to complete the benchmark summary. Units are printed in each sentence.

Complete the four gaps. With a partner, read alternate roles, then switch. Studying alone, read both roles and pause before each response.

**WORD AND PHRASE BANK**

25 / 160 / 40 / 15

01 **Developer:** B's median is [1] _____ milliseconds.

02 **Reviewer:** The median decreases by [2] _____ milliseconds relative to A.

03 **Developer:** B uses [3] _____ megabytes of memory.

04 **Reviewer:** B exceeds the budget by [4] _____ megabytes.

**CHECK THE GAPS**

**1.** 25 / **2.** 15 / **3.** 160 / **4.** 40

Why these answers fit, page 136

### Rebuild the exchange

Choose the missing response. Read the completed three-turn exchange aloud, changing roles on the second reading.

**1. Rebuild the exchange from "Fifteen milliseconds faster, forty megabytes over budget". Sora: "These are benchmark runs, not live measurements. Should I put that boundary alongside the numbers instead of burying it in a note?" Malik: [missing response] Sora: "Then the proposal needs both sides: fifteen milliseconds lower median, twice the memory, and B above the present limit." Which response belongs in the gap?**

**A.** Yes. Production behavior is not established by these figures. Name that limit instead of relying on readers to infer it from the word benchmark.

**B.** Good. Keep the controlled conditions visible alongside that summary. We can make a reasoned next decision without pretending the benchmark answers questions it never measured.

**C.** Exactly. Current budget compliance is clear: A is below the limit and B is above it. A hypothetical future budget should not rewrite today's comparison.

**D.** Prefer the precise absolute difference and percentage latency reduction. Those statements tell readers exactly which measurement changed without suggesting an unmeasured throughput result.

Answers and explanations, page 137

# Review 3 / Meaning and context

Use language from the lessons you have completed. Choose one answer per question.

> **Case file**
>
> Revisit "Explaining what a test actually exercises" and "Comparing performance tradeoffs fairly". Choose precise terminology, then reconstruct the missing responses using both neighboring turns. The final two questions also revisit earlier conversations.

**1. A colleague needs the term for this meaning: Test substitute supplying predetermined responses. Which term fits?**

**A.** act

**B.** exception

**C.** stub

**D.** branch coverage

**2. A colleague needs the term for this meaning: Amount of memory used under the stated conditions. Which term fits?**

**A.** memory footprint

**B.** outlier

**C.** benchmark

**D.** relative reduction

**3. Rebuild the exchange from "A direct upgrade with an indirect change". Leila: "The replacement has a similar name. Before I switch anything, where should we check whether its inputs and error behavior actually match?" Ben: [missing response] Leila: "The version looks familiar, but we haven't checked either package's versioning policy. I shouldn't infer its compatibility promise from the number alone." Which response belongs in the gap?**

**A.** Exactly. We need upgrade evidence tied to this change. It should show what was checked and what remains unknown, not just that an upgrade command succeeded.

**B.** Then we need to review the transitive dependency change as well. The application uses ParcelKit directly, but that does not make TextCore irrelevant to its behavior.

**C.** Please include the lockfile and the relevant dependency relationship. A constraint can allow several versions, while the resolved record tells us which one this change actually selects.

**D.** Consult the migration guide first. Similar names do not establish identical inputs, outputs, error behavior, or handling of edge cases. We need the relevant documented differences.

# Review 3 / Meaning and context / continued

**4. Rebuild the exchange from "Fifteen milliseconds faster, forty megabytes over budget". Sora: "I could request a changed memory budget, but that wouldn't make this run compliant with the current one. It would be a separate decision." Malik: [missing response] Sora: "I'll report the matched conditions, latency decrease, memory figures, and unmeasured tail and live behavior. We still need to decide what to investigate next." Which response belongs in the gap?**

**A.** Exactly. Current budget compliance is clear: A is below the limit and B is above it. A hypothetical future budget should not rewrite today's comparison.

**B.** Correct. Tail latency remains unmeasured. Do not turn a better middle value into a claim about every request or a specific percentile we did not record.

**C.** That is useful benchmark evidence. Put the memory figures beside it before we call B the better option. The resource budget is part of this decision.

**D.** Good. Keep the controlled conditions visible alongside that summary. We can make a reasoned next decision without pretending the benchmark answers questions it never measured.

Answers and explanations, page 154

# A Read the situation

**LESSON 07 / DISCUSSING RETRIES AND SHARED STATE**

Explain an uncertain request outcome and request a retry contract before risking a second reservation.

> **The timeout did not answer the business question**
>
> A fictional reservation client times out before receiving a reply. The service may already have created the reservation. A proposed automatic retry would submit a new reservation request, and the service's duplicate-handling behavior is unspecified. Developers Imani and Leo discuss the risk. No production change is authorized. They need to clarify outcome reconciliation and retry semantics, then test agreed behavior in a suitable controlled environment.

People: Imani, Client developer; Leo, Service developer.

## Read the record

Choose one answer per question using the facts above.

**1. What did the client observe?**

**A.** A verified cancellation
**B.** No reply before its timeout
**C.** Confirmed rejection before processing
**D.** Confirmed creation twice

**2. What might the service have done?**

**A.** Created the reservation already
**B.** Definitely done nothing
**C.** Guaranteed cancellation
**D.** Automatically merged duplicates

**3. What is unspecified?**

**A.** Whether the client timed out
**B.** Whether the retry is proposed as new
**C.** Whether production authorization exists
**D.** Duplicate-handling behavior for the proposed retry

Answers and explanations, page 138

# B Vocabulary in use

**CORE TERMS**

### timeout
End of a waiting period without the required response.

*observe a timeout*

### retry
Repeated attempt after an earlier uncertain or unsuccessful attempt.

*define retry behavior*

### request identity
Identifier or meaning used to distinguish one operation from another.

*preserve request identity*

### idempotency
Property allowing repeated equivalent requests without additional intended effects.

*specify idempotency semantics*

### idempotency key
Identifier used under an agreed contract to recognize repeated intent.

*define an idempotency key*

### duplicate request
Repeated request that may represent the same intended operation.

*detect a duplicate request*

### duplicate side effect
Additional effect caused by repeating an operation.

*prevent a duplicate side effect*

### shared state
Data or condition used by multiple participants or operations.

*reconcile shared state*

### uncertain outcome
Situation where the operation's final result is not known.

*report an uncertain outcome*

### acknowledgment
Response indicating receipt or completion as defined by the protocol.

*distinguish an acknowledgment*

### reconciliation
Checking records to establish and align the actual outcome.

*request outcome reconciliation*

### at-least-once delivery
Delivery model allowing one or more deliveries of a message.

*discuss at-least-once delivery*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**timeout** TIMEOUT

observe a timeout

Keep the meaning precise: End of a waiting period without the required response.

**duplicate request** DUPLICATE REQUEST

detect a duplicate request

Keep the meaning precise: Repeated request that may represent the same intended operation.

# B Precision and professional shorthand

**EXTEND YOUR RANGE**

### at-most-once delivery

Delivery model allowing no more than one delivery, with possible loss.

*distinguish at-most-once delivery*

### race condition

Behavior depending on timing among concurrent operations.

*investigate a race condition*

### atomic operation

Operation treated as an indivisible state change under its model.

*define the atomic operation*

### backoff

Delay strategy between repeated attempts.

*configure retry backoff*

### correlation identifier

Identifier linking related events across components.

*retain the correlation identifier*

### sandbox

Controlled environment separated from live operations.

*test in a sandbox*

### deduplication

Recognition and handling of repeated requests or records.

*define deduplication behavior*

### concurrency

Overlapping progress of multiple operations.

*account for concurrency*

### transaction boundary

Scope within which specified changes are coordinated.

*clarify the transaction boundary*

### retry budget

Limit on permitted repeated attempts under a policy.

*set a retry budget*

### status lookup

Query intended to retrieve an operation's recorded state.

*define a status lookup*

### authorization

Permission for a specified action or change.

*obtain production authorization*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**at-least-once delivery** AT-LEAST-ONCE DELIVERY

discuss at-least-once delivery

Keep the meaning precise: Delivery model allowing one or more deliveries of a message.

**atomic operation** ATOMIC OPERATION

define the atomic operation

Keep the meaning precise: Operation treated as an indivisible state change under its model.

# C Phrases that move the work forward

**State the observation**
The client timed out before receiving a reply.

**Preserve uncertainty**
The service may already have created the reservation.

**Avoid a false conclusion**
No reply does not prove no reservation.

**Identify the proposal**
The retry would submit a new reservation request.

**Name the risk**
That could create a duplicate business effect.

**Ask for the contract**
How does the service recognize repeated intent?

**Avoid assuming support**
Duplicate handling is not specified.

**Clarify identity**
Would this carry the same request identity or a new one?

**Request reconciliation**
Can we establish the outcome of the first request?

**Separate timing and semantics**
Backoff does not itself prevent duplicates.

**Qualify the key**
An idempotency key needs agreed service behavior.

**State the permission boundary**
No production change is authorized.

**Request a controlled check**
Test the agreed behavior in a suitable sandbox.

**Name the missing evidence**
We do not know whether the first reservation exists.

**Keep the design provisional**
Automatic retry remains a proposal.

**Close with the dependency**
Define the contract before claiming retry safety.

## Precision in real exchanges

**Purpose:** Identify the proposal
**Useful wording:** The retry would submit a new reservation request.

**Purpose:** Name the risk
**Useful wording:** That could create a duplicate business effect.

# D Choose the response that fits

Choose one response for each situation. Preserve the facts, the decision boundary, and the purpose of the exchange.

**1. Which statement follows from the timeout?**

**A.** Two reservations definitely exist
**B.** The service canceled the request
**C.** The client did not receive a timely reply
**D.** No reservation exists

**2. Why is a new reservation request risky?**

**A.** Backoff guarantees a second failure
**B.** The first may already have created a reservation
**C.** Every retry always duplicates
**D.** The client has confirmed cancellation

**3. A colleague needs the term for this meaning: Property allowing repeated equivalent requests without additional intended effects. Which term fits?**

**A.** uncertain outcome
**B.** idempotency
**C.** correlation identifier
**D.** race condition

# D Choose the response that fits / continued

**4. Rebuild the exchange from "The timeout did not answer the business question". Imani: "If the service completed the first request, our new request could create another reservation. We don't have a documented rule that combines them." Leo: [missing response] Imani: "Could we use a key to repeat the original intent? I don't want to add a field and assume the service knows how to handle it." Which response belongs in the gap?**

**A.** Exactly. A timeout is an observation at the client. We need separate evidence about the service state before turning that into a business conclusion.

**B.** We need to preserve the uncertain outcome of the first attempt. The service may already have created it, so the timeout is not proof that nothing happened.

**C.** Then deduplication must remain unspecified in the proposal. We cannot promise the second request will be merged or ignored just because the input looks similar.

**D.** Request an agreed status lookup or another supported reconciliation process. Its availability and meaning need confirmation; we have not established either in this discussion.

Answers and explanations, page 138

> **One useful distinction**
>
> A timeout describes the client's wait, not proof that the service did nothing. Submitting a new reservation could create another business effect if the first request already succeeded and duplicate handling is not defined.

# E Conversation

## The timeout did not answer the business question

The timeout did not answer the business question

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

status lookup / Backoff / idempotency key / request identity / reconciliation / deduplication / shared state / authorization / uncertain outcome / timeout

01 **Imani:** The reservation client stopped waiting before a reply arrived. My retry proposal creates a new request, and I'm worried the first reservation may already exist.

02 **Leo:** We need to preserve the [1] _____ of the first attempt. The service may already have created it, so the timeout is not proof that nothing happened.

03 **Imani:** I wrote request failed in the incident note. That's too strong: the only confirmed fact is that the client didn't get a response in time.

04 **Leo:** Exactly. A [2] _____ is an observation at the client. We need separate evidence about the service state before turning that into a business conclusion.

05 **Imani:** If the service completed the first request, our new request could create another reservation. We don't have a documented rule that combines them.

06 **Leo:** Then [3] _____ must remain unspecified in the proposal. We cannot promise the second request will be merged or ignored just because the input looks similar.

07 **Imani:** Could we use a key to repeat the original intent? I don't want to add a field and assume the service knows how to handle it.

08 **Leo:** An [4] _____ needs agreed semantics on the service side. A new field alone does not tell us its scope, reuse rules, or what response a repeat receives.

09 **Imani:** Then we need to specify whether this is another attempt at one booking or a genuinely new booking. The identifiers must reflect that distinction.

10 **Leo:** Yes. Define the [5] _____ and how the service interprets it. That distinction belongs in the contract, not in an assumption hidden inside the client's retry loop.

Conversation continues on the next page.

# The timeout did not answer the business question

**CONVERSATION / CONTINUED**

11 **Imani:** Is there a supported way to check the first outcome? I can ask about a status operation, but I haven't established that one exists.

12 **Leo:** Request an agreed [6] _____ or another supported reconciliation process. Its availability and meaning need confirmation; we have not established either in this discussion.

13 **Imani:** Someone suggested waiting longer before retrying. That might reduce traffic, but wouldn't it leave the duplicate-reservation question exactly where it is?

14 **Leo:** Correct. [7] _____ is a timing policy, not a substitute for outcome and duplicate-handling semantics. A slower duplicate is still a possible duplicate.

15 **Imani:** I'll keep automatic retry disabled while those rules are unresolved. This investigation isn't permission to change live reservation behavior.

16 **Leo:** Keep that [8] _____ boundary visible. Once the contract is defined, we can plan a controlled check without changing live reservation behavior.

17 **Imani:** For the controlled test, I want the service to complete while its reply is delayed. Testing only an immediate rejection misses the uncertain-outcome case.

18 **Leo:** That will address the relevant [9] _____ uncertainty. Use a suitable sandbox and define the expected outcome before interpreting the test as evidence.

19 **Imani:** My revised note will say no timely reply, first outcome unknown, retry currently new, duplicates unspecified, and no production change approved.

20 **Leo:** Good. We need [10] _____ and an explicit retry contract before claiming safety. That is the concrete next discussion, not a guarantee that an extra attempt will be harmless.

**CHECK THE GAPS**

**1.** uncertain outcome / **2.** timeout / **3.** deduplication / **4.** idempotency key / **5.** request identity / **6.** status lookup / **7.** Backoff / **8.** authorization / **9.** shared state / **10.** reconciliation

Why these answers fit, page 139

# F Check accuracy and scope

## The timeout did not answer the business question

### Return to the case

A fictional reservation client times out before receiving a reply. The service may already have created the reservation. A proposed automatic retry would submit a new reservation request, and the service's duplicate-handling behavior is unspecified. Developers Imani and Leo discuss the risk. No production change is authorized. They need to clarify outcome reconciliation and retry semantics, then test agreed behavior in a suitable controlled environment.

Choose one answer per question. Use the case facts and keep any uncertainty or limits intact.

**1. What does an idempotency key require?**

**A.** No server participation
**B.** Automatic universal guarantees
**C.** Agreed service semantics
**D.** Only a convenient label

**2. Which next step is appropriate?**

**A.** Enable the retry in production immediately
**B.** Assume timeout means rejection
**C.** Promise exactly one reservation without evidence
**D.** Clarify reconciliation and retry behavior, then test it outside production

Answers and explanations, page 141

### Listen for the distinction

Swap roles and repeat turns 5-14. Distinguish request identity, outcome lookup, and retry timing.

### Distinguish not knowing from knowing failure

Timeout language can silently collapse uncertainty into failure. Say what the client observed and what the service may have done. This makes a request for an explicit retry contract sound like necessary coordination rather than unnecessary caution.

# F Say it / transfer to a new case

## Do not turn silence into failure

Complete the retry discussion using the stated uncertainty and permission boundary.

Complete the four gaps. With a partner, read alternate roles, then switch. Studying alone, read both roles and pause before each response.

**WORD AND PHRASE BANK**

authorized / unspecified / timeout / exist

01 **Developer:** The client observed a [1] _____.

02 **Reviewer:** The first reservation may already [2] _____.

03 **Developer:** Duplicate handling remains [3] _____.

04 **Reviewer:** No production change is [4] _____.

**CHECK THE GAPS**

**1.** timeout / **2.** exist / **3.** unspecified / **4.** authorized

Why these answers fit, page 141

### Rebuild the exchange

Choose the missing response. Read the completed three-turn exchange aloud, changing roles on the second reading.

**1. Rebuild the exchange from "The timeout did not answer the business question". Imani: "Someone suggested waiting longer before retrying. That might reduce traffic, but wouldn't it leave the duplicate-reservation question exactly where it is?" Leo: [missing response] Imani: "I'll keep automatic retry disabled while those rules are unresolved. This investigation isn't permission to change live reservation behavior." Which response belongs in the gap?**

**A.** Exactly. A timeout is an observation at the client. We need separate evidence about the service state before turning that into a business conclusion.

**B.** That will address the relevant shared state uncertainty. Use a suitable sandbox and define the expected outcome before interpreting the test as evidence.

**C.** Keep that authorization boundary visible. Once the contract is defined, we can plan a controlled check without changing live reservation behavior.

**D.** Correct. Backoff is a timing policy, not a substitute for outcome and duplicate-handling semantics. A slower duplicate is still a possible duplicate.

Answers and explanations, page 142

# A Read the situation

**LESSON 08 / HANDING OVER A RELEASE WITH EXPLICIT LIMITS**

Hand over environment, flag state, migration limits, and decision ownership without conflating deployment with production readiness.

> **R18 is deployed, but the release decision is not made**
>
> Release R18 is deployed in staging. Its new search feature is disabled by a feature flag. A migration adds an optional field, but compatibility with the previous application version has not been tested. Developer Chen hands the status to release reviewer Lee. The production decision remains with Lee and has not been made. Neither the optional field nor the staging deployment establishes that an older application will safely work with the migrated data.

People: Chen, Developer handing over R18; Lee, Release reviewer.

## Read the record

Choose one answer per question using the facts above.

**1. Where is R18 deployed?**

**A.** Production
**B.** Every customer environment
**C.** Nowhere
**D.** Staging

**2. What is the new search feature's state?**

**A.** Enabled for all users
**B.** Deleted from the release
**C.** Proven compatible with every client
**D.** Disabled by a feature flag

**3. Which check is unresolved?**

**A.** Compatibility with the previous application version
**B.** Whether the staging deployment occurred
**C.** Whether the field is optional
**D.** Who owns the production review

Answers and explanations, page 143

# B Vocabulary in use

**CORE TERMS**

### release candidate
Version proposed for release pending the relevant checks.

*review the release candidate*

### deployment
Placement of software into a specified environment.

*report the deployment*

### staging
Preproduction environment used for relevant checks.

*deploy to staging*

### production
Live environment serving actual operational users or workloads.

*authorize production release*

### feature flag
Control that enables or disables specified behavior.

*verify the feature flag*

### disabled state
Condition in which a feature is not enabled.

*preserve the disabled state*

### rollout
Process of making a change available to intended users.

*plan the rollout*

### migration
Change to stored data or its structure.

*review the migration*

### optional field
Field that may be absent under the applicable rules.

*add an optional field*

### schema compatibility
Ability of components to work with the relevant data structure.

*verify schema compatibility*

### previous version
Earlier application version relevant to the comparison.

*test the previous version*

### rollback
Return to an earlier application or system state under a defined plan.

*assess rollback readiness*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**release candidate** RELEASE CANDIDATE

review the release candidate

Keep the meaning precise: Version proposed for release pending the relevant checks.

**disabled state** DISABLED STATE

preserve the disabled state

Keep the meaning precise: Condition in which a feature is not enabled.

# B Precision and professional shorthand

**EXTEND YOUR RANGE**

### forward compatibility

Ability to work with relevant newer data or interfaces.

*check forward compatibility*

### backward compatibility

Ability to preserve supported use by earlier consumers.

*verify backward compatibility*

### release gate

Required condition before advancing a release.

*satisfy the release gate*

### sign-off

Explicit approval at a stated decision point.

*obtain release sign-off*

### go/no-go decision

Decision whether to proceed under the relevant release process.

*record the go/no-go decision*

### handoff

Transfer of relevant status, evidence, and responsibility.

*prepare the release handoff*

### known limitation

Identified restriction or unresolved boundary.

*list known limitations*

### verification matrix

Organized record of combinations and checks to be verified.

*update the verification matrix*

### environment boundary

Limit separating claims about different execution settings.

*preserve the environment boundary*

### data compatibility

Ability to interpret and use the relevant stored data correctly.

*test data compatibility*

### release owner

Person responsible for the specified release decision.

*name the release owner*

### readiness claim

Statement that required conditions for proceeding are met.

*qualify the readiness claim*

## Say it in context

Stress the highlighted expression. Keep the surrounding words connected; pause at the end of the phrase.

**rollback** ROLLBACK

assess rollback readiness

Keep the meaning precise: Return to an earlier application or system state under a defined plan.

**go/no-go decision** GO/NO-GO DECISION

record the go/no-go decision

Keep the meaning precise: Decision whether to proceed under the relevant release process.

# C Phrases that move the work forward

**Identify the release**
This handoff concerns R18.

**Name the environment**
R18 is deployed in staging.

**Name the feature state**
New search remains disabled by its flag.

**Separate deployment and use**
Deployed does not mean enabled.

**Describe the data change**
The migration adds an optional field.

**State the missing check**
Compatibility with the previous application version is untested.

**Avoid a shortcut**
Optional does not prove compatibility.

**Name the decision owner**
Lee owns the production review.

**Keep the decision open**
The production decision has not been made.

**Limit the readiness claim**
This is a staging status, not production sign-off.

**Request the exact evidence**
Check the previous application against the migrated state.

**Separate rollback assumptions**
An older build alone does not establish rollback readiness.

**Identify the pending item**
The compatibility check remains open.

**Ask for a traceable decision**
Record the review outcome explicitly.

**Preserve the boundary**
Do not describe search as live.

**Close with status**
Staging deployed, search disabled, compatibility pending, decision with Lee.

## Precision in real exchanges

**Purpose:** Separate deployment and use
**Useful wording:** Deployed does not mean enabled.

**Purpose:** Describe the data change
**Useful wording:** The migration adds an optional field.

# D Choose the response that fits

Choose one response for each situation. Preserve the facts, the decision boundary, and the purpose of the exchange.

**1. Which status is accurate?**

**A.** R18 has full production sign-off
**B.** The old application is verified compatible
**C.** R18 is in staging with search disabled
**D.** Search is live in production

**2. Why does optional not settle compatibility?**

**A.** Optional fields are always incompatible
**B.** The migration never ran anywhere
**C.** Lee has already rejected release
**D.** The previous application's behavior with migrated data is untested

**3. A colleague needs the term for this meaning: Live environment serving actual operational users or workloads. Which term fits?**

**A.** production
**B.** optional field
**C.** environment boundary
**D.** release gate

# D Choose the response that fits / continued

**4. Rebuild the exchange from "R18 is deployed, but the release decision is not made". Chen: "The migration adds an optional field. I called it backward compatible in the draft, but we haven't tested the prior application against the migrated data." Lee: [missing response] Chen: "So I'll replace the claim with prior-version compatibility untested. I don't have evidence that the older reader tolerates the changed data." Which response belongs in the gap?**

**A.** Good. A release gate isn't met simply because another step completed. We'll review the missing evidence before deciding whether to advance.

**B.** Keep that environment boundary in the first sentence. Someone reading only the headline mustn't mistake staging deployment for a production launch.

**C.** Correct. I own the go/no-go decision, and it hasn't been made. Please don't phrase my review as approval that's already expected.

**D.** Make that a known limitation instead. Optionality is a property of the field, not a result from running the previous application.

Answers and explanations, page 143

> **One useful distinction**
>
> R18 is deployed in staging, and search is disabled. Those are two separate facts. They do not imply production deployment, enabled search, completed compatibility testing, or Lee's approval to proceed.

# E Conversation

## R18 is deployed, but the release decision is not made

R18 is deployed, but the release decision is not made

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

readiness claim / go/no-go decision / rollback / release gate / environment boundary / known limitation / feature flag / handoff / data compatibility / Schema compatibility

01 **Chen:** R18 is deployed in staging. Search is still disabled, and I need your release decision after we address the compatibility gap.

02 **Lee:** Keep that [1] _____ in the first sentence. Someone reading only the headline mustn't mistake staging deployment for a production launch.

03 **Chen:** I'll put the flag status next to the environment. The code being present doesn't mean users can use the new search.

04 **Lee:** Right. The [2] _____ remains off. Installed code, enabled behavior, and production approval are separate facts.

05 **Chen:** The migration adds an optional field. I called it backward compatible in the draft, but we haven't tested the prior application against the migrated data.

06 **Lee:** Make that a [3] _____ instead. Optionality is a property of the field, not a result from running the previous application.

07 **Chen:** So I'll replace the claim with prior-version compatibility untested. I don't have evidence that the older reader tolerates the changed data.

08 **Lee:** Correct. [4] _____ needs the relevant combination checked. Successful deployment of the new build doesn't answer how the old build handles it.

09 **Chen:** We do have the old build available. Is that enough to say we can revert if the release causes a problem?

10 **Lee:** No. [5] _____ readiness depends on more than having the executable. The data state after migration matters to that path.

Conversation continues on the next page.

# R18 is deployed, but the release decision is not made

**CONVERSATION / CONTINUED**

11 **Chen:** Then the open check is the previous application with the migrated state, not simply confirming that its build artifact can be downloaded.

12 **Lee:** Exactly. Request [6] _____ evidence for that combination. Don't substitute an artifact-availability check for an operational result.

13 **Chen:** I'll leave production pending. Sending you the handoff doesn't authorize me to turn search on or change the production environment.

14 **Lee:** Correct. I own the [7] _____, and it hasn't been made. Please don't phrase my review as approval that's already expected.

15 **Chen:** The handoff will list R18, staging, search disabled, the optional-field change, and the missing prior-version check with you as decision owner.

16 **Lee:** That's a usable [8] _____. Concrete states tell me what's complete and what I still need; ready on its own doesn't.

17 **Chen:** When we get the compatibility result, I'll attach the actual evidence. Until then the successful staging step stays separate from that open check.

18 **Lee:** Good. A [9] _____ isn't met simply because another step completed. We'll review the missing evidence before deciding whether to advance.

19 **Chen:** Final status: staging deployed, search disabled, compatibility pending, production decision with Lee. No statement that search is live.

20 **Lee:** That keeps the [10] _____ defensible. Now the next person can act on the unresolved check without having to undo an implied release approval.

**CHECK THE GAPS**

**1.** environment boundary / **2.** feature flag / **3.** known limitation / **4.** Schema compatibility / **5.** rollback / **6.** data compatibility / **7.** go/no-go decision / **8.** handoff / **9.** release gate / **10.** readiness claim

Why these answers fit, page 144

# F Check accuracy and scope

## R18 is deployed, but the release decision is not made

### Return to the case

Release R18 is deployed in staging. Its new search feature is disabled by a feature flag. A migration adds an optional field, but compatibility with the previous application version has not been tested. Developer Chen hands the status to release reviewer Lee. The production decision remains with Lee and has not been made. Neither the optional field nor the staging deployment establishes that an older application will safely work with the migrated data.

Choose one answer per question. Use the case facts and keep any uncertainty or limits intact.

**1. Which rollback claim is justified?**

**A.** An older build guarantees safe rollback
**B.** Staging deployment proves data reversibility
**C.** A disabled flag automatically undoes the migration
**D.** Readiness remains unverified without the relevant compatibility evidence

**2. Who owns the unresolved production decision?**

**A.** The feature flag itself
**B.** The migration tool
**C.** Lee
**D.** Every staging user

Answers and explanations, page 146

### Listen for the distinction

Swap roles for turns 5-12. Stress untested prior-version behavior rather than assuming optional means compatible.

### Say done with an object attached

Deployed, enabled, tested, and approved describe different milestones. Attach each status to an environment, feature, or specific check. This reduces pressure on the next person to infer readiness from a single broad word such as ready or complete.

# F Say it / transfer to a new case

## Four facts for the release handoff

Complete the R18 handoff. Do not upgrade any pending check or decision to completed status.

Complete the four gaps. With a partner, read alternate roles, then switch. Studying alone, read both roles and pause before each response.

**WORD AND PHRASE BANK**

untested / disabled / staging / Lee

01 **Developer:** R18 is deployed in [1] _____.

02 **Reviewer:** New search remains [2] _____.

03 **Developer:** Prior-version compatibility is [3] _____.

04 **Reviewer:** The production decision remains with [4] _____.

**CHECK THE GAPS**

**1.** staging / **2.** disabled / **3.** untested / **4.** Lee

Why these answers fit, page 146

### Rebuild the exchange

Choose the missing response. Read the completed three-turn exchange aloud, changing roles on the second reading.

**1. Rebuild the exchange from "R18 is deployed, but the release decision is not made". Chen: "I'll leave production pending. Sending you the handoff doesn't authorize me to turn search on or change the production environment." Lee: [missing response] Chen: "The handoff will list R18, staging, search disabled, the optional-field change, and the missing prior-version check with you as decision owner." Which response belongs in the gap?**

**A.** Keep that environment boundary in the first sentence. Someone reading only the headline mustn't mistake staging deployment for a production launch.

**B.** Make that a known limitation instead. Optionality is a property of the field, not a result from running the previous application.

**C.** Good. A release gate isn't met simply because another step completed. We'll review the missing evidence before deciding whether to advance.

**D.** Correct. I own the go/no-go decision, and it hasn't been made. Please don't phrase my review as approval that's already expected.

Answers and explanations, page 147

# Review 4 / Meaning and context

Use language from the lessons you have completed. Choose one answer per question.

> **Case file**
>
> Revisit "Discussing retries and shared state" and "Handing over a release with explicit limits". Choose precise terminology, then reconstruct the missing responses using both neighboring turns. The final two questions also revisit earlier conversations.

**1. A colleague needs the term for this meaning: Identifier or meaning used to distinguish one operation from another. Which term fits?**

**A.** shared state
**B.** deduplication
**C.** retry budget
**D.** request identity

**2. A colleague needs the term for this meaning: Change to stored data or its structure. Which term fits?**

**A.** known limitation
**B.** release candidate
**C.** forward compatibility
**D.** migration

**3. Rebuild the exchange from "Fifteen milliseconds faster, forty megabytes over budget". Sora: "We didn't collect tail percentiles. A lower median could coexist with worse slow requests, so I can't claim improvement for every request." Malik: [missing response] Sora: "A is forty megabytes below the same budget. I'll keep that separate from B's forty-megabyte excess rather than call both numbers spare memory." Which response belongs in the gap?**

**A.** Yes, that is the relative reduction relative to A. Keep the baseline explicit so nobody divides by twenty-five and reports a different quantity under the same label.
**B.** Correct. Tail latency remains unmeasured. Do not turn a better middle value into a claim about every request or a specific percentile we did not record.
**C.** That is useful benchmark evidence. Put the memory figures beside it before we call B the better option. The resource budget is part of this decision.
**D.** That is forty megabytes of headroom under the stated budget. It is a different claim from saying A has more memory available under all possible workloads.

# Review 4 / Meaning and context / continued

**4. Rebuild the exchange from "R18 is deployed, but the release decision is not made". Chen: "When we get the compatibility result, I'll attach the actual evidence. Until then the successful staging step stays separate from that open check." Lee: [missing response] Chen: "Final status: staging deployed, search disabled, compatibility pending, production decision with Lee. No statement that search is live." Which response belongs in the gap?**

**A.** Keep that environment boundary in the first sentence. Someone reading only the headline mustn't mistake staging deployment for a production launch.

**B.** Make that a known limitation instead. Optionality is a property of the field, not a result from running the previous application.

**C.** Good. A release gate isn't met simply because another step completed. We'll review the missing evidence before deciding whether to advance.

**D.** Correct. I own the go/no-go decision, and it hasn't been made. Please don't phrase my review as approval that's already expected.

Answers and explanations, page 157

# Language for the next case

## The newer tag cannot repair an older payload

### stale representation

B edited the older v7 representation containing eighteen dollars; the current representation is v8 with twenty dollars, so the submitted full record is stale.

*Marco: That is the stale representation we need to discuss. A has already changed the price to twenty; B hasn't seen that change.*

### If-Match precondition

If-Match requires the supplied entity tag to match the current representation using strong comparison; v7 does not match the current v8 here.

*Marco: Yes. The If-Match precondition fails here. The response is 412 with no update, not evidence that B's description was saved.*

### lost update

Sending the stale eighteen-dollar price with a current tag could overwrite A's twenty-dollar change; replacing only the tag does not reconcile the old body.

*Marco: It could cause a lost update. The body still carries eighteen. Passing a version check wouldn't make that overwrite intentional.*

### strong ETag

A strong ETag is the representation validator used by this conditional request; it is not permission to submit stale field values or proof of user intent.

*Marco: Exactly. A strong ETag identifies the representation you're checking against. It does not tell us which conflicting fields the user meant to replace.*

### HTTP 412

HTTP 412 is the specified precondition-failure response in this case; it does not establish a server outage, a successful save, or a generic authorization failure.

*Marco: Keep the HTTP 412 meaning precise. The price remains twenty and the description remains old after this rejected request.*

### conflict reconciliation

Conflict reconciliation compares the current record with the intended edit before constructing another request; copying v8 onto the old body would bypass that reasoning.

*Marco: That's conflict reconciliation, not a blind retry. Another update would still need the current condition checked when it reaches the service.*

Read the phrases aloud. Then complete the conversation on the following pages using the word bank.

# Conversation

## The newer tag cannot repair an older payload

Fictional whole-record PUT replaces price and description. Both clients read $18, "old", strong ETag "v7". A changes price to $20, creating strong tag "v8". B submits $18, "new", If-Match: "v7". B's change has not previously succeeded. This service returns 412 without changing data when the condition fails. No later update is approved.

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

stale representation / strong ETag / lost update / HTTP 412 / conflict reconciliation / If-Match precondition

01 **Dina:** B got a 412 saving the description. Its payload still contains eighteen dollars, because both clients started from the same version.

02 **Marco:** That is the [1] _____ we need to discuss. A has already changed the price to twenty; B hasn't seen that change.

03 **Dina:** The request includes v7. Is the server comparing that with the current version, rather than comparing our description text?

04 **Marco:** Yes. The [2] _____ fails here. The response is 412 with no update, not evidence that B's description was saved.

05 **Dina:** Could I fetch just the new tag and resubmit exactly the same body? That would get past the version check.

06 **Marco:** It could cause a [3] _____. The body still carries eighteen. Passing a version check wouldn't make that overwrite intentional.

# Conversation / continued

07 **Dina:** So we need the current record as well as its tag. We should show the price change before preparing another full replacement.

08 **Marco:** Exactly. A [4] _____ identifies the representation you're checking against. It does not tell us which conflicting fields the user meant to replace.

09 **Dina:** The status text says Precondition Failed. Someone called it a server outage, but the service is rejecting our condition as designed.

10 **Marco:** Keep the [5] _____ meaning precise. The price remains twenty and the description remains old after this rejected request.

Conversation continues on the next page.

# The newer tag cannot repair an older payload

**CONVERSATION / CONTINUED**

11 **Dina:** For the next attempt, we'll retrieve the current record, review the difference, and preserve twenty if the user only wants to change the description.

12 **Marco:** That's [6] _____, not a blind retry. Another update would still need the current condition checked when it reaches the service.

13 **Dina:** Meaning someone else could change the record between our fresh read and our next submission. Fetching once doesn't lock it.

14 **Marco:** Right. Optimistic concurrency detects a changed version at the conditional write. It doesn't reserve the record while the user is reading it.

15 **Dina:** Our error message should preserve the user's unsaved description rather than discard it when we fetch the newer record.

16 **Marco:** Agreed as a proposed interface behavior. Show the fresh record and pending edit separately so the user can review what will actually be sent.

17 **Dina:** For the test, I'll interleave two clients: A saves twenty first, then B sends the stale condition and gets the specified rejection.

18 **Marco:** Check the stored fields afterward too. A status assertion alone wouldn't catch a defective implementation that rejects the request but still changes the data.

19 **Dina:** Then the handoff is: B's save rejected, A's twenty preserved, description unchanged, and a reviewed resubmission still pending.

20 **Marco:** Yes. Don't label it resolved merely because we know v8. The condition protects the write; the client still has to prepare the right change.

**CHECK THE GAPS**

**1.** stale representation / **2.** If-Match precondition / **3.** lost update / **4.** strong ETag / **5.** HTTP 412 / **6.** conflict reconciliation

Why these answers fit, page 160

# A rejected edit is not a merged edit

Same fictional service. Current strong tag "v12" has quantity 9 and note "old". A stale PUT sends quantity 7, note "new", If-Match: "v11". It has not previously succeeded. The service returns 412 without changing data. No resubmission occurs.

Complete the exchange using each phrase once. Read both roles, then switch roles and repeat with the bank covered.

**WORD AND PHRASE BANK**

412 / 9 / old / stale

01 **Developer:** The returned status is [1] _____ .

02 **Reviewer:** The stored quantity remains [2] _____ .

03 **Developer:** The stored note is still [3] _____ .

04 **Reviewer:** Replacing only the tag would leave the payload [4] _____ .

**CHECK THE GAPS**

**1.** 412 / **2.** 9 / **3.** old / **4.** stale

Why these answers fit, page 160

**Keep the distinction audible**

Explain a rejected conditional update without silently overwriting another person's change.

# Language for the next case

## Fifty orders, fifty-one queries

### N+1 query pattern

One initial order query plus one line query for each of fifty orders produces fifty-one queries; the extra work grows with the number of orders.

*Amir: That matches the N+1 query pattern in this trace. Loading the orders once doesn't mean their related lines were loaded with them.*

### lazy loading

Lazy loading defers fetching related data until access; in this supplied trace each order's line collection triggers another query.

*Amir: That's lazy loading here. It isn't inherently wrong, but this access pattern produces one database call per order.*

### batched lookup

The proposed implementation uses one related-line query for the fifty selected order IDs, replacing fifty separate line queries while retaining the initial order query.

*Amir: Yes, the proposed batched lookup reduces this example from fifty-one to two. Check that each line is attached to its correct order afterward.*

### round-trip count

The new path has two serial round trips, contributing eight milliseconds in the supplied model; the count is not the complete application response time.

*Amir: And the new round-trip count contributes eight milliseconds under that model. That's a hundred ninety-six milliseconds less modeled network delay.*

### network component

Eight milliseconds accounts only for the two modeled network delays; database execution, transferred data, and application processing remain additional and unmeasured.

*Amir: Call it the network component, not the whole response. We haven't timed query execution, transferred rows, or object construction.*

### result set

The result set is the data returned by the queries; reducing query count does not establish its size, memory cost, or correct association with each order.

*Amir: Exactly. Measure the result set as well. An order with no lines must still appear if the page contract includes it.*

Read the phrases aloud. Then complete the conversation on the following pages using the word bank.

# Conversation

## Fifty orders, fifty-one queries

Fictional order page: one query loads 50 orders, then one separate query per order loads its lines. All 51 calls are serial; no cache or other queries. A proposed batch implementation uses one order query and one line query for those 50 IDs. For this simplified model, each database round trip contributes exactly 4 ms of network delay. Database work, data transfer, and application work are additional and unmeasured.

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

lazy loading / network component / round-trip count / batched lookup / result set / N+1 query pattern

01 **Jules:** The page has fifty orders, but the trace shows fifty-one queries. The first fetch looks normal; the extra calls appear while we render the lines.

02 **Amir:** That matches the [1] _____ in this trace. Loading the orders once doesn't mean their related lines were loaded with them.

03 **Jules:** Our object-relational mapper fetches the lines when the template first accesses each collection. I hadn't noticed that property access was issuing SQL.

04 **Amir:** That's [2] _____ here. It isn't inherently wrong, but this access pattern produces one database call per order.

05 **Jules:** The proposed version fetches the orders, collects their IDs, then fetches all matching lines together. That would be two queries for this page.

06 **Amir:** Yes, the proposed [3] _____ reduces this example from fifty-one to two. Check that each line is attached to its correct order afterward.

# Conversation / continued

07 **Jules:** At four milliseconds per serial round trip, the old path has two hundred four milliseconds of network delay before we add other work.

08 **Amir:** And the new [4] _____ contributes eight milliseconds under that model. That's a hundred ninety-six milliseconds less modeled network delay.

09 **Jules:** I'll put eight milliseconds in the response-time target, then. The arithmetic makes it look very fast.

10 **Amir:** Call it the [5] _____, not the whole response. We haven't timed query execution, transferred rows, or object construction.

Conversation continues on the next page.

# Fifty orders, fifty-one queries

**CONVERSATION / CONTINUED**

11 **Jules:** A larger batch could load more rows at once. Fewer queries don't automatically mean less memory or less database work.

12 **Amir:** Exactly. Measure the [6] _____ as well. An order with no lines must still appear if the page contract includes it.

13 **Jules:** I'll compare the rendered orders and line associations, not only the query counter. We could get two queries and still show the wrong data.

14 **Amir:** Good. Keep ordering and empty collections in that check. The optimization must preserve the page's specified result.

15 **Jules:** For a bigger page, is one giant query always the best answer? Some database parameter limits would affect a long list of IDs.

16 **Amir:** The batch size needs an implementation decision. Our fifty-ID example isn't a universal limit or a promise that all frameworks use that size.

17 **Jules:** Then I'll label fifty as this case's input, with two queries for the proposed version. We still need measurements on representative data.

18 **Amir:** Yes. Record query count, total time, and relevant resource use separately. A changed query count is strong evidence about calls, not everything else.

19 **Jules:** The review summary will say fifty-one versus two queries, modeled network delay two hundred four versus eight milliseconds, actual total latency not yet measured.

20 **Amir:** That is a defensible comparison. It explains the mechanism and arithmetic without turning a simplified calculation into a production guarantee.

**CHECK THE GAPS**

**1.** N+1 query pattern / **2.** lazy loading / **3.** batched lookup / **4.** round-trip count / **5.** network component / **6.** result set

Why these answers fit, page 162

# Batch size changes the query count

New fictional page: 200 orders, one initial order query, no cache. Old path makes one additional line query per order. New implementation explicitly groups IDs into batches of 50, one line query per batch. All calls are serial at 4 ms network delay each; other work remains unmeasured.

Complete the exchange using each phrase once. Read both roles, then switch roles and repeat with the bank covered.

**WORD AND PHRASE BANK**

20 / 201 / 5 / 784

01 **Developer:** The batched path makes [1] _____ total queries.

02 **Reviewer:** Its modeled network delay is [2] _____ milliseconds.

03 **Developer:** The old path makes [3] _____ total queries.

04 **Reviewer:** The modeled network-delay reduction is [4] _____ milliseconds.

**CHECK THE GAPS**

**1.** 5 / **2.** 20 / **3.** 201 / **4.** 784

Why these answers fit, page 162

### Keep the distinction audible

Explain query-count growth and a batching proposal without promising an unmeasured response time.

# Language for the next case

## One cent, two different contracts

### rounding stage

The agreed contract rounds each line's charge before summing; rounding once on the basket subtotal is a different calculation even with the same rate.

*Sol: Check the rounding stage. The contract rounds each line's charge before adding the charges together.*

### half-up rule

For these positive amounts, half-up rounds the exact half-cent tie upward to one cent; it must not be replaced with half-even or an unspecified default.

*Sol: Yes. The specified half-up rule sends 0.005 to 0.01. The fifteen-cent subtotal plus three cents gives eighteen.*

### basket-level calculation

The proposed calculation rounds 0.015 once to 0.02 and totals 0.17; it is internally consistent but does not implement the supplied line-level contract.

*Sol: That's a basket-level calculation. Its arithmetic isn't the agreed algorithm. Equal rates don't make the rounding stages interchangeable.*

### decimal arithmetic

Decimal arithmetic can preserve these exact decimal inputs and the requested rounding operation; merely formatting a binary result does not implement the missing line-level rule.

*Sol: Formatting doesn't establish decimal arithmetic. We need the calculation and its rounding decisions to match the contract first.*

### rounding mode

The rounding mode determines how ties are resolved; the contract explicitly requires half-up rather than relying on a library's possibly different default.

*Sol: And make the rounding mode explicit at the required operation. Don't assume the library default is half-up.*

### line-item boundary

The three separate line items each receive their own rounded charge; merging them into one line would change the supplied contract's calculation unit.

*Sol: Exactly. Preserve the line-item boundary. The test should catch a future optimization that sums first and rounds later.*

Read the phrases aloud. Then complete the conversation on the following pages using the word bank.

# Conversation

## One cent, two different contracts

Fictional checkout test, not tax advice. Three separate line items cost $0.05 each before a fictional 10% charge. The agreed contract calculates the charge on each line, rounds it to two decimal places using half-up, then sums the rounded charges. No discounts or other charges. A proposed implementation instead calculates 10% of the basket subtotal and rounds only that result. Use exact decimal inputs.

Complete the numbered gaps. Use each bank entry once; some entries contain several words.

**WORD AND PHRASE BANK**

rounding mode / half-up rule / decimal arithmetic / rounding stage / basket-level calculation / line-item boundary

01 **Rina:** My checkout total is seventeen cents, but the acceptance example says eighteen. Both calculations use ten percent, so I thought the example had a typo.

02 **Sol:** Check the [1] _____. The contract rounds each line's charge before adding the charges together.

03 **Rina:** Each five-cent line produces half a cent before rounding. With this rule, that becomes one cent per line, three cents in all.

04 **Sol:** Yes. The specified [2] _____ sends 0.005 to 0.01. The fifteen-cent subtotal plus three cents gives eighteen.

05 **Rina:** My version applies ten percent to fifteen cents, giving one and a half cents, then rounds to two cents. That explains seventeen.

06 **Sol:** That's a [3] _____. Its arithmetic isn't the agreed algorithm. Equal rates don't make the rounding stages interchangeable.

# Conversation / continued

07 **Rina:** Would changing the display to two decimal places fix it? The customer would still see a neatly formatted amount.

08 **Sol:** Formatting doesn't establish [4] _____. We need the calculation and its rounding decisions to match the contract first.

09 **Rina:** I'll construct the decimal values from the original decimal strings. Converting a binary floating-point approximation afterward can preserve the wrong intermediate value.

10 **Sol:** And make the [5] _____ explicit at the required operation. Don't assume the library default is half-up.

Conversation continues on the next page.

# One cent, two different contracts

**CONVERSATION / CONTINUED**

11 **Rina:** So the regression example must retain three separate five-cent lines. Combining them into one fifteen-cent line changes the meaning of the fixture.

12 **Sol:** Exactly. Preserve the [6] _____. The test should catch a future optimization that sums first and rounds later.

13 **Rina:** Could we store prices in integer cents instead? That would avoid binary approximations for the original prices.

14 **Sol:** It can represent those prices exactly, but the calculated charge still has fractional cents. Integer storage doesn't choose when or how to round them.

15 **Rina:** I also shouldn't throw away the half cent before applying the specified rounding. That would make each charge zero in this example.

16 **Sol:** Correct. Keep sufficient precision for the rule, then round at the agreed point. Truncating intermediate values would implement another algorithm.

17 **Rina:** I'll show the per-line charge, summed charge, subtotal, and total in the test evidence. Then the one-cent difference has a visible source.

18 **Sol:** Good. This contract describes our current implementation. We still need the approved financial rules for each jurisdiction before making a compliance claim.

19 **Rina:** The corrected expectation is three one-cent charges and an eighteen-cent total. My seventeen-cent result belongs to the rejected basket-level proposal.

20 **Sol:** That's the review conclusion. Fix the algorithm and verify this distinguishing example; don't just change the expected value to whatever the implementation currently returns.

**CHECK THE GAPS**

**1.** rounding stage / **2.** half-up rule / **3.** basket-level calculation / **4.** decimal arithmetic / **5.** rounding mode / **6.** line-item boundary

Why these answers fit, page 164

# Preserve four separate lines

Same fictional line-level contract and exact decimal inputs, now four separate $0.25 lines. Charge rate 10%; round each charge half-up to two decimal places, then sum. No other amounts. Compare with rounding 10% of the $1.00 subtotal only once.

Complete the exchange using each phrase once. Read both roles, then switch roles and repeat with the bank covered.

**WORD AND PHRASE BANK**

0.12 / 0.02 / 0.03 / 1.12

01 **Developer:** Each rounded line charge is [1] _____ dollars.

02 **Reviewer:** The sum of the four rounded charges is [2] _____ dollars.

03 **Developer:** The contract's final total is [3] _____ dollars.

04 **Reviewer:** That exceeds the basket-level total by [4] _____ dollars.

**CHECK THE GAPS**

**1.** 0.03 / **2.** 0.12 / **3.** 1.12 / **4.** 0.02

Why these answers fit, page 164

### Keep the distinction audible

Explain why rounding stage and rounding mode belong in an executable money calculation contract.

# 01 / Answers and explanations

## A / Read the situation

### A1. What does omitting nickname do under the current contract?

**B. Leaves the stored value unchanged**

The current contract explicitly preserves the value when nickname is omitted.

Return to the exercise, page 4

### A2. What does an empty string do?

**D. Clears the nickname**

The empty string has a defined clearing meaning distinct from omission.

Return to the exercise, page 4

### A3. What remains undefined?

**A. The meaning of an explicit null**

The proposed client sends null, whose meaning the contract has not specified.

Return to the exercise, page 4

## D / Response choices

### D1. Which comment identifies the real gap?

**C. The null case needs an agreed meaning**

Only null lacks a specified meaning in this particular contract.

Return to the exercise, page 8

### D2. Which pair already has different defined effects in this contract?

**B. Omission and an empty string**

Omission preserves the value, whereas an empty string clears it.

Return to the exercise, page 8

# 01 / Answers and explanations / continued

### D3. A colleague needs the term for this meaning: Field that may be absent under the contract. Which term fits?

**A. optional field**

**A Correct.** optional field: Field that may be absent under the contract. The usual combination is "omit an optional field".

**B Not here.** decimal arithmetic: Base-ten calculation with specified precision and rounding behavior. The usual combination is "use decimal arithmetic".

**C Not here.** stored state: Value or condition currently retained by the system. The usual combination is "preserve stored state".

**D Not here.** schema: Formal description of data structure and constraints. The usual combination is "revise the schema".

Return to the exercise, page 8

### D4. Rebuild the exchange from "Three nickname states, only two defined meanings". Maya: "For the existing clearing action, then, I'd send a string with zero characters. Leaving nickname out would preserve the old value instead." Arun: [missing response] Maya: "I'll stop calling null an alternative clearing command. We haven't agreed on what it means here, even if another API uses it that way." Which response belongs in the gap?

**D. Yes, an empty string clears the nickname. We should preserve that distinction in the examples so nobody silently changes the current behavior while updating the client.**

**A Not here.** This line belongs to turn 2, after Maya says: "The form currently sends null when a user removes a nickname. Before I ship that, which input does this service actually define as clearing?" It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 12, after Maya says: "I'll show the actual three wire examples separately: absent, empty string, and explicit null. Only the first two have defined outcomes so far." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 4, after Maya says: "I treated optional as permission to send null. But a missing property and a present property with a null value aren't the same payload." It addresses that point, not the missing turn in the supplied exchange.

**D Correct.** This is Arun's response in turn 6. It connects Maya's preceding point with Maya's follow-up. An empty string has the specified effect of clearing the nickname, unlike omission.

Return to the exercise, page 8

## E / Conversation

### 1. omitted field

**Turn 2.** An omitted field is absent and preserves the stored nickname under the stated contract.

# 01 / Answers and explanations / continued

### 2. optional field

**Turn 4.** Optional field describes allowed absence, not the meaning or acceptability of an explicit null.

### 3. empty string

**Turn 6.** An empty string has the specified effect of clearing the nickname, unlike omission.

### 4. interface contract

**Turn 8.** The interface contract is the agreement that must specify the missing null behavior before implementation.

### 5. request payload

**Turn 10.** Request payload identifies the actual submitted data, which may differ from an assumed client representation.

### 6. field presence

**Turn 12.** Field presence distinguishes an absent nickname property from a present property carrying a value.

### 7. validation rule

**Turn 14.** The validation rule must reflect an agreed input policy rather than preempt the unresolved decision.

### 8. contract test

**Turn 16.** A contract test checks the agreed interaction across the client-service boundary rather than only the visible form.

### 9. backward compatibility

**Turn 18.** Backward compatibility concerns preserving supported behavior for existing clients, including the established omission rule.

### 10. acceptance criterion

**Turn 20.** An acceptance criterion must name the agreed input and effect before implementation can be judged complete.

Return to the conversation, page 10

# 01 / Answers and explanations / continued

## F / Accuracy and scope

### F1. What would justify calling null a clearing command?

**D. An explicit agreed contract change**

A new interpretation needs agreement rather than an unsupported inference.

Return to the exercise, page 12

### F2. Which test plan matches the discussion?

**A. Check each defined input state and resolve null before asserting its effect**

Distinct input states require distinct expectations, with null still unresolved.

Return to the exercise, page 12

## F / Guided speaking

### 1. unchanged

**Developer:** An omitted nickname leaves the value unchanged.

**Turn 1.** Omission explicitly preserves the existing nickname under the current contract.

### 2. clears

**Reviewer:** An empty string clears the nickname.

**Turn 2.** The empty string is the defined clearing input in this interface.

### 3. unspecified

**Developer:** The meaning of null remains unspecified.

**Turn 3.** No agreed effect for null is supplied in the scenario.

### 4. nullable

**Reviewer:** Optional does not automatically mean nullable.

**Turn 4.** Optional concerns absence, while nullable concerns allowed explicit values.

Return to the conversation, page 13

# 01 / Answers and explanations / continued

## F / Rebuild the exchange

**1. Rebuild the exchange from "Three nickname states, only two defined meanings". Maya: "Would allowing null in the schema get ahead of the decision? I don't want that change to imply a stored-state rule nobody has approved." Arun: [missing response] Maya: "Once the rule is settled, I'll test the client-service interaction, not just whether the form looks empty on screen." Which response belongs in the gap?**

**A. Hold that change until the validation rule and intended effect are agreed. Allowing a value and defining its effect need to be consistent.**

**A Correct.** This is Arun's response in turn 14. It connects Maya's preceding point with Maya's follow-up. The validation rule must reflect an agreed input policy rather than preempt the unresolved decision.

**B Not here.** This line belongs to turn 18, after Maya says: "And I want the omission case protected. Solving the null question shouldn't silently turn an unchanged nickname into a cleared one for older clients." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 2, after Maya says: "The form currently sends null when a user removes a nickname. Before I ship that, which input does this service actually define as clearing?" It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 16, after Maya says: "Once the rule is settled, I'll test the client-service interaction, not just whether the form looks empty on screen." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 13

# 02 / Answers and explanations

## A / Read the situation

### A1. Which input fails?

**B. A three-row file containing an empty date**

Only the stated three-row example is established as failing.

Return to the exercise, page 14

### A2. What does the successful comparison change?

**B. Both row count and content**

Removing the row changes the number of rows and their content.

Return to the exercise, page 14

### A3. What does the stack trace establish?

**D. The observed failure path reaches date parsing**

The trace locates part of the failure path but does not isolate the cause.

Return to the exercise, page 14

## D / Response choices

### D1. Which conclusion is supported?

**B. The observed failing path reaches date parsing**

The trace supports a location statement, not the broader causal claims.

Return to the exercise, page 18

### D2. Which proposed comparison is strongest?

**B. Keep three rows and other content fixed while varying the suspect date**

Holding other factors fixed better isolates the condition being tested.

Return to the exercise, page 18

# 02 / Answers and explanations / continued

### D3. A colleague needs the term for this meaning: Component interpreting structured input. Which term fits?

**A. parser**

**A Correct.** parser: Component interpreting structured input. The usual combination is "inspect the parser".

**B Not here.** confounding variable: Additional changed factor obscuring a comparison. The usual combination is "remove a confounding variable".

**C Not here.** regression: Previously working behavior broken by a later change. The usual combination is "investigate a regression".

**D Not here.** batched lookup: Retrieving data for several keys in one grouped operation. The usual combination is "replace repeated calls with a batched lookup".

Return to the exercise, page 18

### D4. Rebuild the exchange from "The empty date is a suspect, not yet the cause". Nia: "I wrote that the empty date causes the error. You're asking me to narrow that to what the comparison actually shows." Ellis: [missing response] Nia: "Then I'll keep three rows and change only that date to a valid value, with the other data and settings unchanged." Which response belongs in the gap?

**B. Yes. Row count is a confounding variable in this comparison. We don't know whether it matters, but we haven't held it fixed.**

**A Not here.** This line belongs to turn 2, after Nia says: "The three-row import failed, and the trace reaches date parsing. I've attached the input, including the empty date in row two." It addresses that point, not the missing turn in the supplied exchange.

**B Correct.** This is Ellis's response in turn 6. It connects Nia's preceding point with Nia's follow-up. A confounding variable is another changed factor that prevents this comparison from isolating the date condition.

**C Not here.** This line belongs to turn 8, after Nia says: "Then I'll keep three rows and change only that date to a valid value, with the other data and settings unchanged." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 12, after Nia says: "The trace points into the parser. Could that still be downstream of a problem elsewhere in the import?" It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 18

## E / Conversation

### 1. reproducible example

**Turn 2.** A reproducible example provides the input and conditions needed to observe the reported failure again.

# 02 / Answers and explanations / continued

### 2. hypothesis

**Turn 4.** Hypothesis labels the empty-date explanation as plausible but not yet isolated by a controlled test.

### 3. confounding variable

**Turn 6.** A confounding variable is another changed factor that prevents this comparison from isolating the date condition.

### 4. controlled comparison

**Turn 8.** Controlled comparison holds the other relevant conditions fixed while varying the suspected date condition.

### 5. input fixture

**Turn 10.** Input fixture names the prepared test data, which must remain available to reproduce the original failure.

### 6. stack trace

**Turn 12.** The stack trace records the observed call path around the error but does not alone establish its root cause.

### 7. failure signature

**Turn 14.** Failure signature distinguishes the original error pattern from a different failure in the comparison run.

### 8. deterministic behavior

**Turn 16.** Deterministic behavior concerns consistent results under the same relevant conditions, which repeated checks can investigate.

### 9. nonproduction environment

**Turn 18.** A nonproduction environment supports the investigation without treating this dialogue as permission to test on live workloads.

### 10. causal claim

**Turn 20.** A causal claim needs stronger evidence than the current confounded comparison, so the proposed cause remains qualified.

Return to the conversation, page 20

# 02 / Answers and explanations / continued

## F / Accuracy and scope

### F1. Which phrase avoids overstating evidence?

**D. The empty date is a working hypothesis**

A working hypothesis accurately labels the explanation as still requiring evidence.

Return to the exercise, page 22

### F2. What should the handoff preserve?

**A. The failing fixture, conditions, and trace**

Reproducible evidence needs the original input and execution context.

Return to the exercise, page 22

## F / Guided speaking

### 1. three

**Developer:** The failing file contains three rows.

**Turn 1.** The supplied failing fixture has three rows, including one empty date.

### 2. two

**Reviewer:** The successful file contains two rows.

**Turn 2.** Removing the suspect row leaves the stated successful two-row file.

### 3. hypothesis

**Developer:** The empty date remains a hypothesis.

**Turn 3.** The current comparison has not isolated the date as the cause.

### 4. constant

**Reviewer:** Keep the other conditions constant in the next comparison.

**Turn 4.** Holding other conditions constant helps isolate the suspected date effect.

Return to the conversation, page 23

# 02 / Answers and explanations / continued

## F / Rebuild the exchange

**1. Rebuild the exchange from "The empty date is a suspect, not yet the cause". Nia: "I'll preserve the error type and location with the run details. If a later run fails differently, we shouldn't count it as the same result." Ellis: [missing response] Nia: "And I'll rerun each controlled input. A single failure followed by a single success doesn't establish repeatability." Which response belongs in the gap?**

**A. Exactly. Compare the failure signature as well as pass or fail. Otherwise a changed exception could hide a different defect.**

**A Correct.** This is Ellis's response in turn 14. It connects Nia's preceding point with Nia's follow-up. Failure signature distinguishes the original error pattern from a different failure in the comparison run.

**B Not here.** This line belongs to turn 12, after Nia says: "The trace points into the parser. Could that still be downstream of a problem elsewhere in the import?" It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 4, after Nia says: "I haven't established that yet. I removed row two and the import passed, which made me suspect the empty date." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 16, after Nia says: "And I'll rerun each controlled input. A single failure followed by a single success doesn't establish repeatability." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 23

# 03 / Answers and explanations

## A / Read the situation

### A1. What is the pull request titled?

**B. Rename export helper**

The stated title describes a helper rename, not a notification-policy change.

Return to the exercise, page 26

### A2. What additional change appears in the diff?

**B. Notification moves outside the success branch**

The changed call location permits notification after an unsuccessful export.

Return to the exercise, page 26

### A3. What part does Priya accept?

**D. The naming change**

Priya accepts the rename while objecting to the changed notification behavior.

Return to the exercise, page 26

## D / Response choices

### D1. Which comment is actionable?

**A. Restore the success-only notification and test that failure sends no completion notice**

The correct comment names the behavior, requested change, and verification.

Return to the exercise, page 30

### D2. Why is this not only a refactor?

**A. Observable notification behavior changes**

A behavior-preserving refactor would not add completion notices after failure.

Return to the exercise, page 30

# 03 / Answers and explanations / continued

**D3. A colleague needs the term for this meaning: Execution path entered when the relevant operation succeeds. Which term fits?**

**C. success branch**

**A Not here.** guard condition: Condition controlling whether an action may occur. The usual combination is "restore the guard condition".

**B Not here.** completion notification: Message indicating an operation has finished successfully here. The usual combination is "gate the completion notification".

**C Correct.** success branch: Execution path entered when the relevant operation succeeds. The usual combination is "keep the call in the success branch".

**D Not here.** blocking concern: Issue that must be resolved before approval under the review process. The usual combination is "explain the blocking concern".

Return to the exercise, page 30

**D4. Rebuild the exchange from "A rename that changed the failure path". Priya: "Keep the clearer name. Restore the guard around the notification; that's the behavior I'm blocking on, not my naming preference." Tomas: [missing response] Priya: "Please add a failing-export case too. It should verify that no completion notice is sent, not merely that export was attempted." Which response belongs in the gap?**

**A. Understood. The notification is a side effect we need to preserve correctly. I'll keep it dependent on a successful export.**

**A Correct.** This is Tomas's response in turn 6. It connects Priya's preceding point with Priya's follow-up. Side effect refers to the externally visible notification rather than the helper's new name.

**B Not here.** This line belongs to turn 14, after Priya says: "Would you agree the current version isn't behavior-preserving? The failed export and the completion notice contradict each other." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 12, after Priya says: "Also check the description against the final patch. Right now the title makes this sound like a purely mechanical change." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 16, after Priya says: "Once the guard and failure assertion are in place, send the updated change back with the relevant test result." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 30

## E / Conversation

### 1. rename

**Turn 2.** Rename identifies the accepted identifier change, which does not explain or justify the additional behavioral change.

# 03 / Answers and explanations / continued

### 2. control flow

**Turn 4.** Control flow determines whether a failed export can reach the notification call after it moves outside the branch.

### 3. side effect

**Turn 6.** Side effect refers to the externally visible notification rather than the helper's new name.

### 4. failure path

**Turn 8.** The failure path must be exercised to check that unsuccessful exports do not trigger a completion notice.

### 5. review comment

**Turn 10.** A review comment is actionable when it identifies the changed location, effect, and requested correction.

### 6. review scope

**Turn 12.** Review scope should reflect the actual remaining changes rather than rely solely on the original rename title.

### 7. false success

**Turn 14.** False success is the completion indication produced even though the export operation failed.

### 8. patch revision

**Turn 16.** A patch revision is an updated proposed change, which still needs review rather than automatic approval.

### 9. regression test

**Turn 18.** A regression test should detect recurrence of the known false-notification defect rather than merely execute the helper.

### 10. verification evidence

**Turn 20.** Verification evidence supports the revised behavior and is needed before the reviewer can resolve the blocking concern.

Return to the conversation, page 32

# 03 / Answers and explanations / continued

## F / Accuracy and scope

### F1. What should the failure test assert?

**B. No completion notice is sent**

The defect concerns a false success message after export failure.

Return to the exercise, page 34

### F2. Which statement preserves Priya's position?

**A. The rename is acceptable; the notification behavior needs correction**

Priya distinguishes an acceptable naming change from an unresolved behavioral defect.

Return to the exercise, page 34

## F / Guided speaking

### 1. name

**Reviewer:** The helper name is acceptable.

**Turn 1.** Priya explicitly accepts the naming change in the pull request.

### 2. success

**Author:** The notification must remain on the success path.

**Turn 2.** Completion notification is intended only for a successful export here.

### 3. failure

**Reviewer:** Add a test for the failure path.

**Turn 3.** The regression occurs when export fails yet a completion notice appears.

### 4. approval

**Author:** The current feedback is not yet approval.

**Turn 4.** The blocking behavior issue still requires correction and verification.

Return to the conversation, page 35

# 03 / Answers and explanations / continued

## F / Rebuild the exchange

**1. Rebuild the exchange from "A rename that changed the failure path". Priya: "Would you agree the current version isn't behavior-preserving? The failed export and the completion notice contradict each other." Tomas: [missing response] Priya: "Once the guard and failure assertion are in place, send the updated change back with the relevant test result." Which response belongs in the gap?**

**C. Yes. That false success is a defect, not a style disagreement. I won't describe the current patch as an approved refactor.**

**A Not here.** This line belongs to turn 6, after Priya says: "Keep the clearer name. Restore the guard around the notification; that's the behavior I'm blocking on, not my naming preference." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 12, after Priya says: "Also check the description against the final patch. Right now the title makes this sound like a purely mechanical change." It addresses that point, not the missing turn in the supplied exchange.

**C Correct.** This is Tomas's response in turn 14. It connects Priya's preceding point with Priya's follow-up. False success is the completion indication produced even though the export operation failed.

**D Not here.** This line belongs to turn 4, after Priya says: "The position. A failed export can now reach that call, so the user could receive a completion notice for an unsuccessful export." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 35

# 04 / Answers and explanations

## A / Read the situation

### A1. Which dependency does the application use directly?

**B. ParcelKit**

The application uses ParcelKit directly, while TextCore is introduced through ParcelKit.

Return to the exercise, page 36

### A2. What changes indirectly?

**C. The resolved TextCore version**

The proposed ParcelKit upgrade changes the TextCore version selected in the dependency resolution.

Return to the exercise, page 36

### A3. What does the note establish?

**A. A parsing entry point is deprecated**

Deprecation is stated, but usage, removal, and replacement compatibility remain unverified.

Return to the exercise, page 36

## D / Response choices

### D1. Which statement is accurate?

**A. TextCore is transitive through ParcelKit**

TextCore is brought into the application through its direct dependency ParcelKit.

Return to the exercise, page 40

### D2. What should be checked first about the selected version?

**B. The resolved dependency record**

The resolved record establishes the concrete TextCore version selected by this upgrade.

Return to the exercise, page 40

# 04 / Answers and explanations / continued

### D3. A colleague needs the term for this meaning: Specific version selected after dependency resolution. Which term fits?

**C. resolved version**

**A Not here.** compatibility: Ability to work with the relevant existing consumers and behavior. The usual combination is "verify compatibility".

**B Not here.** removal: Elimination of a previously available interface or behavior. The usual combination is "verify removal separately".

**C Correct.** resolved version: Specific version selected after dependency resolution. The usual combination is "record the resolved version".

**D Not here.** changelog: Maintained record of notable changes between versions. The usual combination is "compare the changelog".

Return to the exercise, page 40

### D4. Rebuild the exchange from "A direct upgrade with an indirect change". Leila: "The release notes mark a parsing entry point deprecated. I don't yet know whether our code or ParcelKit's internals call it." Ben: [missing response] Leila: "I'll remove my claim that the upgrade deletes a parser we use. Deprecation, removal, and our usage are three different questions." Which response belongs in the gap?

**D. Keep that as a deprecation finding, not a removal finding. The note deserves investigation, but its label alone does not tell us whether this application is affected.**

**A Not here.** This line belongs to turn 12, after Leila says: "The version looks familiar, but we haven't checked either package's versioning policy. I shouldn't infer its compatibility promise from the number alone." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 2, after Leila says: "The ParcelKit upgrade also selects a different TextCore version. The request title doesn't show that indirect change, but the resolved list does." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 4, after Leila says: "I can attach the package constraints and actual selected versions. Which record do you need to see what this build will really use?" It addresses that point, not the missing turn in the supplied exchange.

**D Correct.** This is Ben's response in turn 6. It connects Leila's preceding point with Leila's follow-up. Deprecation marks an interface for discouragement or transition but does not alone establish removal or current usage.

Return to the exercise, page 40

## E / Conversation

### 1. transitive dependency

**Turn 2.** TextCore is a transitive dependency because the application receives it through ParcelKit rather than using it directly here.

# 04 / Answers and explanations / continued

### 2. lockfile

**Turn 4.** The lockfile records resolved dependency selections and is more specific than a top-level version constraint alone.

### 3. deprecation

**Turn 6.** Deprecation marks an interface for discouragement or transition but does not alone establish removal or current usage.

### 4. call site

**Turn 8.** A call site shows where the affected parsing interface is invoked, which must be checked rather than assumed.

### 5. migration guide

**Turn 10.** The migration guide provides transition details that a similar replacement name cannot establish.

### 6. semantic versioning

**Turn 12.** Semantic versioning provides specific conventions only when the relevant project actually adopts and follows that policy.

### 7. dependency tree

**Turn 14.** The dependency tree makes the ParcelKit-to-TextCore relationship visible so indirect impact is included in the review.

### 8. public interface

**Turn 16.** The public interface is the exposed parsing surface whose supported behavior and affected callers need verification.

### 9. upgrade evidence

**Turn 18.** Upgrade evidence combines actual versions, affected usage, documentation, and relevant checks rather than a broad assertion of safety.

### 10. compatibility

**Turn 20.** Compatibility remains unverified until the relevant consumers and behavior have been checked against the proposed versions.

Return to the conversation, page 42

# 04 / Answers and explanations / continued

## F / Accuracy and scope

### F1. What does deprecated not establish?

**B. That the entry point has been removed**

Deprecation and removal are different statuses, and removal has not been established.

Return to the exercise, page 45

### F2. Which compatibility claim is justified now?

**D. Compatibility remains unverified**

Usage and replacement behavior have not yet been checked.

Return to the exercise, page 45

## F / Guided speaking

### 1. direct

**Developer:** ParcelKit is the direct dependency.

**Turn 1.** The application uses ParcelKit directly under the stated dependency relationship.

### 2. transitive

**Reviewer:** TextCore is the transitive dependency.

**Turn 2.** TextCore is introduced through ParcelKit rather than directly here.

### 3. deprecated

**Developer:** The parsing entry point is marked deprecated.

**Turn 3.** The change note establishes deprecation, not removal or usage.

### 4. unverified

**Reviewer:** Replacement compatibility remains unverified.

**Turn 4.** The relevant behavior and affected callers have not yet been checked.

Return to the conversation, page 46

# 04 / Answers and explanations / continued

## F / Rebuild the exchange

**1. Rebuild the exchange from "A direct upgrade with an indirect change". Leila: "I'll trace ParcelKit to TextCore in the review notes. That should explain why a package we don't call directly still matters to this upgrade." Ben: [missing response] Leila: "The existing suite passes, but I haven't shown that it exercises the affected parsing entry point. What evidence should the review request?" Which response belongs in the gap?**

**A. Good. The dependency tree will help explain why TextCore belongs in the discussion even though the request is named after ParcelKit.**

**A Correct.** This is Ben's response in turn 14. It connects Leila's preceding point with Leila's follow-up. The dependency tree makes the ParcelKit-to-TextCore relationship visible so indirect impact is included in the review.

**B Not here.** This line belongs to turn 2, after Leila says: "The ParcelKit upgrade also selects a different TextCore version. The request title doesn't show that indirect change, but the resolved list does." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 16, after Leila says: "The existing suite passes, but I haven't shown that it exercises the affected parsing entry point. What evidence should the review request?" It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 10, after Leila says: "The replacement has a similar name. Before I switch anything, where should we check whether its inputs and error behavior actually match?" It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 46

# 05 / Answers and explanations

## A / Read the situation

### A1. What does the storage double currently do?

**A. Always succeeds**

The supplied test double is configured only for successful storage.

Return to the exercise, page 49

### A2. What did the previous bug show?

**C. Success after storage rejected the save**

The prior defect was a false success indication after rejection.

Return to the exercise, page 49

### A3. What remains separately untested?

**C. The real storage adapter**

The scenario explicitly states that the real adapter has not been tested.

Return to the exercise, page 49

## D / Response choices

### D1. Why does the current test miss the prior bug?

**C. Its double never rejects**

The failing condition required to expose the bug is absent from the fixture.

Return to the exercise, page 53

### D2. Which addition targets the defect?

**C. Configure rejection and assert no success message**

The new case must trigger rejection and verify the missing user-visible boundary.

Return to the exercise, page 53

# 05 / Answers and explanations / continued

### D3. A colleague needs the term for this meaning: Test substitute that can record or check interactions. Which term fits?

**A. mock**

**A Correct.** mock: Test substitute that can record or check interactions. The usual combination is "inspect mock interactions".

**B Not here.** assertion: Explicit check of an expected test condition. The usual combination is "add a negative assertion".

**C Not here.** assert: Check the observed result against the expectation. The usual combination is "assert no false success".

**D Not here.** happy path: Expected successful execution path. The usual combination is "cover the happy path".

Return to the exercise, page 53

### D4. Rebuild the exchange from "A green test that never sees rejection". Owen: "I could assert that storage was called once. Would that distinguish the corrected application from the old one?" Farah: [missing response] Owen: "Right, the old version called it but treated the rejected save as completed. The call count wouldn't catch that." Which response belongs in the gap?

**B. Not by itself. Add a negative assertion that the success message is absent after rejection. Both versions might call storage once.**

**A Not here.** This line belongs to turn 8, after Owen says: "Right, the old version called it but treated the rejected save as completed. The call count wouldn't catch that." It addresses that point, not the missing turn in the supplied exchange.

**B Correct.** This is Farah's response in turn 6. It connects Owen's preceding point with Owen's follow-up. A negative assertion checks that the unwanted success message does not appear after the simulated rejection.

**C Not here.** This line belongs to turn 2, after Owen says: "The save test passes. I was about to mark the old false-success bug covered, but the storage substitute always returns success." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 4, after Owen says: "It doesn't. I have no rejection case in this fixture, so the green result can't tell us whether we still ignore rejection." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 53

## E / Conversation

### 1. happy path

**Turn 2.** The happy path is the successful execution route supplied by the always-successful storage double.

# 05 / Answers and explanations / continued

### 2. test double

**Turn 4.** The test double replaces real storage and must be configured to produce the rejection needed by this test.

### 3. negative assertion

**Turn 6.** A negative assertion checks that the unwanted success message does not appear after the simulated rejection.

### 4. Interaction verification

**Turn 8.** Interaction verification checks dependency calls but does not alone establish the correct user-visible result.

### 5. integration test

**Turn 10.** An integration test would exercise the relevant connected components, whereas this case still replaces the real storage adapter.

### 6. coverage gap

**Turn 12.** The coverage gap for the real adapter remains open even if the simulated rejection unit case passes.

### 7. arrange

**Turn 14.** Arrange is the preparation stage that configures the rejection condition before the save behavior is executed.

### 8. success response

**Turn 16.** Success response is the outcome configured in the existing test, so its evidence is limited to that path.

### 9. behavioral verification

**Turn 18.** Behavioral verification checks the observed application result, while retaining the boundary imposed by the substituted dependency.

### 10. unit test

**Turn 20.** A unit test can cover the application's rejection handling without proving the behavior of the real storage integration.

Return to the conversation, page 55

# 05 / Answers and explanations / continued

## F / Accuracy and scope

### F1. What does the rejection unit test not establish?

**A. Correct behavior of the real storage adapter**

The test still substitutes for the real adapter rather than exercising it.

Return to the exercise, page 57

### F2. Which report is precise?

**B. Success-path coverage exists; rejection and real-adapter coverage need work**

The report separates existing coverage from two distinct unresolved areas.

Return to the exercise, page 57

## F / Guided speaking

### 1. succeeds

**Developer:** The existing double always succeeds.

**Turn 1.** The current fixture supplies only successful storage outcomes to the save operation.

### 2. rejection

**Reviewer:** The new case must trigger rejection.

**Turn 2.** Rejection is the missing condition that exposed the previous false-success bug.

### 3. success

**Developer:** The assertion should prevent a false success message.

**Turn 3.** The known defect incorrectly displayed success after storage rejected the save.

### 4. untested

**Reviewer:** The real adapter remains untested.

**Turn 4.** Simulated rejection coverage does not exercise the real storage adapter.

Return to the conversation, page 58

# 05 / Answers and explanations / continued

## F / Rebuild the exchange

**1. Rebuild the exchange from "A green test that never sees rejection". Owen: "I'll prepare the rejected result, execute the save, then inspect the displayed message. The fixture should make the unsuccessful outcome obvious." Farah: [missing response] Owen: "I'll retain the original check separately. It confirms that the application shows the intended message when the substitute reports a successful save." Which response belongs in the gap?**

**B. Yes: arrange the rejection, act on the save, and assert no success indication. Reviewers should see that setup without hunting through defaults.**

**A Not here.** This line belongs to turn 10, after Owen says: "This still crosses the storage interface. Could I label it an integration check once the failure case is in place?" It addresses that point, not the missing turn in the supplied exchange.

**B Correct.** This is Farah's response in turn 14. It connects Owen's preceding point with Owen's follow-up. Arrange is the preparation stage that configures the rejection condition before the save behavior is executed.

**C Not here.** This line belongs to turn 16, after Owen says: "I'll retain the original check separately. It confirms that the application shows the intended message when the substitute reports a successful save." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 2, after Owen says: "The save test passes. I was about to mark the old false-success bug covered, but the storage substitute always returns success." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 58

# 06 / Answers and explanations

## A / Read the situation

### A1. Which conditions are matched?

**B. Input and machine**

The benchmark explicitly uses the same input and machine for both implementations.

Return to the exercise, page 59

### A2. How far is B over the memory budget?

**C. 40 megabytes**

B uses 160 megabytes against a budget of 120, exceeding it by forty.

Return to the exercise, page 59

### A3. What is unmeasured?

**B. Tail latency and production behavior**

Only the benchmark median and memory figures are supplied, not tail or production results.

Return to the exercise, page 59

## D / Response choices

### D1. What is B's median reduction relative to A?

**B. 37.5 percent**

Fifteen divided by the forty-millisecond baseline equals 0.375, or 37.5 percent.

Return to the exercise, page 63

### D2. Which budget statement is correct?

**C. A is 40 below and B is 40 above**

The 120-megabyte limit lies forty above A and forty below B.

Return to the exercise, page 63

# 06 / Answers and explanations / continued

### D3. A colleague needs the term for this meaning: Slower-end behavior in a latency distribution. Which term fits?

**B. tail latency**

**A Not here.** controlled conditions: Specified conditions held consistent during comparison. The usual combination is "preserve controlled conditions".

**B Correct.** tail latency: Slower-end behavior in a latency distribution. The usual combination is "measure tail latency".

**C Not here.** resource budget: Allowed limit for a specified resource. The usual combination is "respect the resource budget".

**D Not here.** tradeoff: Gain in one property accompanied by a cost in another. The usual combination is "explain the tradeoff".

Return to the exercise, page 63

### D4. Rebuild the exchange from "Fifteen milliseconds faster, forty megabytes over budget". Sora: "For the percentage, I subtracted twenty-five from forty and divided the fifteen-millisecond decrease by forty. That gives thirty-seven-point-five percent." Malik: [missing response] Sora: "Would faster be too loose in the headline? I measured latency, not how many requests per second either version handles." Which response belongs in the gap?

**C. Yes, that is the relative reduction relative to A. Keep the baseline explicit so nobody divides by twenty-five and reports a different quantity under the same label.**

**A Not here.** This line belongs to turn 2, after Sora says: "B's measured median is twenty-five milliseconds versus A's forty on the same input and machine. I'd like to propose B, but its memory figure complicates that." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 12, after Sora says: "A is forty megabytes below the same budget. I'll keep that separate from B's forty-megabyte excess rather than call both numbers spare memory." It addresses that point, not the missing turn in the supplied exchange.

**C Correct.** This is Malik's response in turn 6. It connects Sora's preceding point with Sora's follow-up. Relative reduction divides the fifteen-millisecond decrease by A's forty-millisecond baseline, yielding 37.5 percent.

**D Not here.** This line belongs to turn 10, after Sora says: "We didn't collect tail percentiles. A lower median could coexist with worse slow requests, so I can't claim improvement for every request." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 63

## E / Conversation

### 1. benchmark

**Turn 2.** Benchmark identifies the controlled measurement, whose conditions limit the scope of the comparison.

# 06 / Answers and explanations / continued

### 2. memory footprint

**Turn 4.** Memory footprint is the measured memory usage, which is 160 megabytes for B against a 120-megabyte budget.

### 3. relative reduction

**Turn 6.** Relative reduction divides the fifteen-millisecond decrease by A's forty-millisecond baseline, yielding 37.5 percent.

### 4. absolute difference

**Turn 8.** Absolute difference is the direct fifteen-millisecond reduction, which can be reported alongside the relative latency reduction.

### 5. Tail latency

**Turn 10.** Tail latency concerns slower-end observations and cannot be inferred from the supplied median alone.

### 6. headroom

**Turn 12.** Headroom is the forty-megabyte difference between A's eighty-megabyte usage and the 120-megabyte limit.

### 7. Production behavior

**Turn 14.** Production behavior remains unmeasured, so the controlled benchmark cannot establish live-system outcomes.

### 8. tradeoff

**Turn 16.** Tradeoff connects B's lower median latency with its higher memory use and failure to meet the stated budget.

### 9. budget compliance

**Turn 18.** Budget compliance is evaluated against the stated 120-megabyte limit, which B exceeds regardless of its latency gain.

### 10. controlled conditions

**Turn 20.** Controlled conditions specify the matched input and machine and prevent the result being generalized without further evidence.

Return to the conversation, page 65

# 06 / Answers and explanations / continued

## F / Accuracy and scope

### F1. Which claim exceeds the evidence?

**D. B improves every production request**

Production behavior and every-request outcomes have not been measured.

Return to the exercise, page 67

### F2. Which summary is balanced?

**D. B lowers median latency but exceeds the memory budget; tail and production remain unmeasured**

The summary combines the measured benefit, resource constraint, and remaining evidence limits.

Return to the exercise, page 67

## F / Guided speaking

### 1. 25

**Developer:** B's median is 25 milliseconds.

**Turn 1.** The measured median for implementation B is twenty-five milliseconds.

### 2. 15

**Reviewer:** The median decreases by 15 milliseconds relative to A.

**Turn 2.** Forty minus twenty-five gives a fifteen-millisecond absolute decrease.

### 3. 160

**Developer:** B uses 160 megabytes of memory.

**Turn 3.** The supplied memory measurement for implementation B is 160 megabytes.

### 4. 40

**Reviewer:** B exceeds the budget by 40 megabytes.

**Turn 4.** One hundred sixty minus the 120-megabyte budget leaves forty over.

Return to the conversation, page 68

# 06 / Answers and explanations / continued

## F / Rebuild the exchange

**1. Rebuild the exchange from "Fifteen milliseconds faster, forty megabytes over budget". Sora: "These are benchmark runs, not live measurements. Should I put that boundary alongside the numbers instead of burying it in a note?" Malik: [missing response] Sora: "Then the proposal needs both sides: fifteen milliseconds lower median, twice the memory, and B above the present limit." Which response belongs in the gap?**

**A. Yes. Production behavior is not established by these figures. Name that limit instead of relying on readers to infer it from the word benchmark.**

**A Correct.** This is Malik's response in turn 14. It connects Sora's preceding point with Sora's follow-up. Production behavior remains unmeasured, so the controlled benchmark cannot establish live-system outcomes.

**B Not here.** This line belongs to turn 20, after Sora says: "I'll report the matched conditions, latency decrease, memory figures, and unmeasured tail and live behavior. We still need to decide what to investigate next." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 18, after Sora says: "I could request a changed memory budget, but that wouldn't make this run compliant with the current one. It would be a separate decision." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 8, after Sora says: "Would faster be too loose in the headline? I measured latency, not how many requests per second either version handles." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 68

# 07 / Answers and explanations

## A / Read the situation

### A1. What did the client observe?

**B. No reply before its timeout**

A timeout establishes lack of a timely reply, not the service's final action.

Return to the exercise, page 71

### A2. What might the service have done?

**A. Created the reservation already**

The scenario explicitly leaves prior creation possible and unresolved.

Return to the exercise, page 71

### A3. What is unspecified?

**D. Duplicate-handling behavior for the proposed retry**

The service's treatment of duplicate requests is not defined in the supplied contract.

Return to the exercise, page 71

## D / Response choices

### D1. Which statement follows from the timeout?

**C. The client did not receive a timely reply**

A client timeout leaves the service-side result uncertain.

Return to the exercise, page 75

### D2. Why is a new reservation request risky?

**B. The first may already have created a reservation**

A repeated new operation may add another effect while the first outcome remains unknown.

Return to the exercise, page 75

# 07 / Answers and explanations / continued

### D3. A colleague needs the term for this meaning: Property allowing repeated equivalent requests without additional intended effects. Which term fits?

**B. idempotency**

**A Not here.** uncertain outcome: Situation where the operation's final result is not known. The usual combination is "report an uncertain outcome".

**B Correct.** idempotency: Property allowing repeated equivalent requests without additional intended effects. The usual combination is "specify idempotency semantics".

**C Not here.** correlation identifier: Identifier linking related events across components. The usual combination is "retain the correlation identifier".

**D Not here.** race condition: Behavior depending on timing among concurrent operations. The usual combination is "investigate a race condition".

Return to the exercise, page 75

### D4. Rebuild the exchange from "The timeout did not answer the business question". Imani: "If the service completed the first request, our new request could create another reservation. We don't have a documented rule that combines them." Leo: [missing response] Imani: "Could we use a key to repeat the original intent? I don't want to add a field and assume the service knows how to handle it." Which response belongs in the gap?

**C. Then deduplication must remain unspecified in the proposal. We cannot promise the second request will be merged or ignored just because the input looks similar.**

**A Not here.** This line belongs to turn 4, after Imani says: "I wrote request failed in the incident note. That's too strong: the only confirmed fact is that the client didn't get a response in time." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 2, after Imani says: "The reservation client stopped waiting before a reply arrived. My retry proposal creates a new request, and I'm worried the first reservation may already exist." It addresses that point, not the missing turn in the supplied exchange.

**C Correct.** This is Leo's response in turn 6. It connects Imani's preceding point with Imani's follow-up. Deduplication behavior is unspecified, so repeated intent cannot be assumed to produce only one reservation.

**D Not here.** This line belongs to turn 12, after Imani says: "Is there a supported way to check the first outcome? I can ask about a status operation, but I haven't established that one exists." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 75

## E / Conversation

### 1. uncertain outcome

**Turn 2.** Uncertain outcome means the first reservation may exist even though the client did not receive a timely response.

# 07 / Answers and explanations / continued

### 2. timeout

**Turn 4.** Timeout describes expiration of the client's wait, not a confirmed service-side rejection or absence of a reservation.

### 3. deduplication

**Turn 6.** Deduplication behavior is unspecified, so repeated intent cannot be assumed to produce only one reservation.

### 4. idempotency key

**Turn 8.** An idempotency key needs a defined service contract for repeated intent rather than merely being added to a request.

### 5. request identity

**Turn 10.** Request identity distinguishes a repeated attempt at the original operation from a new reservation with another intended effect.

### 6. status lookup

**Turn 12.** Status lookup is a possible reconciliation mechanism that must actually be supported and defined rather than invented.

### 7. Backoff

**Turn 14.** Backoff changes timing between attempts but does not itself establish the earlier outcome or prevent duplicate business effects.

### 8. authorization

**Turn 16.** Authorization is explicitly absent for production changes, so investigation must not be described as permission to enable the retry.

### 9. shared state

**Turn 18.** Shared state is the reservation state whose service-side result may differ from what the timed-out client knows.

### 10. reconciliation

**Turn 20.** Reconciliation establishes the actual first-request outcome and supports a defined retry decision rather than assuming failure from silence.

Return to the conversation, page 77

# 07 / Answers and explanations / continued

## F / Accuracy and scope

### F1. What does an idempotency key require?

**C. Agreed service semantics**

A key is useful only when the service contract defines how repeated intent is handled.

Return to the exercise, page 79

### F2. Which next step is appropriate?

**D. Clarify reconciliation and retry behavior, then test it outside production**

The contract and controlled evidence are missing, and production changes are not authorized.

Return to the exercise, page 79

## F / Guided speaking

### 1. timeout

**Developer:** The client observed a timeout.

**Turn 1.** The client stopped waiting before it received the service reply.

### 2. exist

**Reviewer:** The first reservation may already exist.

**Turn 2.** The service may have completed creation before the client timed out.

### 3. unspecified

**Developer:** Duplicate handling remains unspecified.

**Turn 3.** The service contract supplies no duplicate-handling behavior for this proposal.

### 4. authorized

**Reviewer:** No production change is authorized.

**Turn 4.** The scenario explicitly withholds permission to enable a live change.

Return to the conversation, page 80

# 07 / Answers and explanations / continued

## F / Rebuild the exchange

**1. Rebuild the exchange from "The timeout did not answer the business question". Imani: "Someone suggested waiting longer before retrying. That might reduce traffic, but wouldn't it leave the duplicate-reservation question exactly where it is?" Leo: [missing response] Imani: "I'll keep automatic retry disabled while those rules are unresolved. This investigation isn't permission to change live reservation behavior." Which response belongs in the gap?**

**D. Correct. Backoff is a timing policy, not a substitute for outcome and duplicate-handling semantics. A slower duplicate is still a possible duplicate.**

**A Not here.** This line belongs to turn 4, after Imani says: "I wrote request failed in the incident note. That's too strong: the only confirmed fact is that the client didn't get a response in time." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 18, after Imani says: "For the controlled test, I want the service to complete while its reply is delayed. Testing only an immediate rejection misses the uncertain-outcome case." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 16, after Imani says: "I'll keep automatic retry disabled while those rules are unresolved. This investigation isn't permission to change live reservation behavior." It addresses that point, not the missing turn in the supplied exchange.

**D Correct.** This is Leo's response in turn 14. It connects Imani's preceding point with Imani's follow-up. Backoff changes timing between attempts but does not itself establish the earlier outcome or prevent duplicate business effects.

Return to the exercise, page 80

# 08 / Answers and explanations

## A / Read the situation

### A1. Where is R18 deployed?

**D. Staging**

The scenario establishes a staging deployment only, not production deployment.

Return to the exercise, page 81

### A2. What is the new search feature's state?

**D. Disabled by a feature flag**

The search feature is present but disabled by its flag.

Return to the exercise, page 81

### A3. Which check is unresolved?

**A. Compatibility with the previous application version**

Compatibility with the prior application has not been tested.

Return to the exercise, page 81

## D / Response choices

### D1. Which status is accurate?

**C. R18 is in staging with search disabled**

Only the staging deployment and disabled search state are established.

Return to the exercise, page 85

### D2. Why does optional not settle compatibility?

**D. The previous application's behavior with migrated data is untested**

Optional describes a field rule, not evidence of the older application's behavior.

Return to the exercise, page 85

# 08 / Answers and explanations / continued

### D3. A colleague needs the term for this meaning: Live environment serving actual operational users or workloads. Which term fits?

**A. production**

**A Correct.** production: Live environment serving actual operational users or workloads. The usual combination is "authorize production release".

**B Not here.** optional field: Field that may be absent under the applicable rules. The usual combination is "add an optional field".

**C Not here.** environment boundary: Limit separating claims about different execution settings. The usual combination is "preserve the environment boundary".

**D Not here.** release gate: Required condition before advancing a release. The usual combination is "satisfy the release gate".

Return to the exercise, page 85

### D4. Rebuild the exchange from "R18 is deployed, but the release decision is not made". Chen: "The migration adds an optional field. I called it backward compatible in the draft, but we haven't tested the prior application against the migrated data." Lee: [missing response] Chen: "So I'll replace the claim with prior-version compatibility untested. I don't have evidence that the older reader tolerates the changed data." Which response belongs in the gap?

**D. Make that a known limitation instead. Optionality is a property of the field, not a result from running the previous application.**

**A Not here.** This line belongs to turn 18, after Chen says: "When we get the compatibility result, I'll attach the actual evidence. Until then the successful staging step stays separate from that open check." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 2, after Chen says: "R18 is deployed in staging. Search is still disabled, and I need your release decision after we address the compatibility gap." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 14, after Chen says: "I'll leave production pending. Sending you the handoff doesn't authorize me to turn search on or change the production environment." It addresses that point, not the missing turn in the supplied exchange.

**D Correct.** This is Lee's response in turn 6. It connects Chen's preceding point with Chen's follow-up. Known limitation identifies the untested previous-version compatibility rather than presenting optionality as proof.

Return to the exercise, page 85

## E / Conversation

### 1. environment boundary

**Turn 2.** Environment boundary limits the deployment claim to staging rather than implying a live production release.

# 08 / Answers and explanations / continued

### 2. feature flag

**Turn 4.** The feature flag controls whether new search is enabled, which is separate from the code being deployed.

### 3. known limitation

**Turn 6.** Known limitation identifies the untested previous-version compatibility rather than presenting optionality as proof.

### 4. Schema compatibility

**Turn 8.** Schema compatibility requires relevant evidence about components working with the changed structure, which is not supplied here.

### 5. rollback

**Turn 10.** Rollback is a return to an earlier state whose readiness depends on compatibility with the actual migrated data and system state.

### 6. data compatibility

**Turn 12.** Data compatibility concerns whether the earlier application can correctly use the actual migrated state, not whether an old executable exists.

### 7. go/no-go decision

**Turn 14.** The go/no-go decision is Lee's unresolved production decision, not an automatic consequence of receiving the handoff.

### 8. handoff

**Turn 16.** Handoff transfers specific state, evidence gaps, and responsibility so the reviewer does not have to infer readiness.

### 9. release gate

**Turn 18.** A release gate is a condition for advancing the release and cannot be treated as satisfied by an unrelated completed deployment.

### 10. readiness claim

**Turn 20.** A readiness claim must stay within the completed checks and unresolved decision rather than equating staging deployment with approval.

Return to the conversation, page 87

# 08 / Answers and explanations / continued

## F / Accuracy and scope

### F1. Which rollback claim is justified?

**D. Readiness remains unverified without the relevant compatibility evidence**

The earlier application's compatibility with the changed state has not been tested.

Return to the exercise, page 89

### F2. Who owns the unresolved production decision?

**C. Lee**

Lee is explicitly identified as the reviewer responsible for the production decision.

Return to the exercise, page 89

## F / Guided speaking

### 1. staging

**Developer:** R18 is deployed in staging.

**Turn 1.** The only established deployment environment in the scenario is staging.

### 2. disabled

**Reviewer:** New search remains disabled.

**Turn 2.** The feature flag keeps search disabled despite the completed deployment.

### 3. untested

**Developer:** Prior-version compatibility is untested.

**Turn 3.** No test of the previous application against the migrated state is established.

### 4. Lee

**Reviewer:** The production decision remains with Lee.

**Turn 4.** Lee owns the production review and has not made that decision.

Return to the conversation, page 90

# 08 / Answers and explanations / continued

## F / Rebuild the exchange

**1. Rebuild the exchange from "R18 is deployed, but the release decision is not made". Chen: "I'll leave production pending. Sending you the handoff doesn't authorize me to turn search on or change the production environment." Lee: [missing response] Chen: "The handoff will list R18, staging, search disabled, the optional-field change, and the missing prior-version check with you as decision owner." Which response belongs in the gap?**

**D. Correct. I own the go/no-go decision, and it hasn't been made. Please don't phrase my review as approval that's already expected.**

**A Not here.** This line belongs to turn 2, after Chen says: "R18 is deployed in staging. Search is still disabled, and I need your release decision after we address the compatibility gap." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 6, after Chen says: "The migration adds an optional field. I called it backward compatible in the draft, but we haven't tested the prior application against the migrated data." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 18, after Chen says: "When we get the compatibility result, I'll attach the actual evidence. Until then the successful staging step stays separate from that open check." It addresses that point, not the missing turn in the supplied exchange.

**D Correct.** This is Lee's response in turn 14. It connects Chen's preceding point with Chen's follow-up. The go/no-go decision is Lee's unresolved production decision, not an automatic consequence of receiving the handoff.

Return to the exercise, page 90

# Review 1 / Explanations

## Cumulative review

### 1. A colleague needs the term for this meaning: Data carried by a request. Which term fits?

**B. request payload**

**A Not here.** null value: Explicit marker whose meaning depends on the contract. The usual combination is "define the null value".

**B Correct.** request payload: Data carried by a request. The usual combination is "inspect the request payload".

**C Not here.** deserialization: Conversion from transported data into usable values. The usual combination is "verify deserialization behavior".

**D Not here.** producer: Component providing data or behavior through an interface. The usual combination is "align producer expectations".

Return to the exercise, page 24

### 2. A colleague needs the term for this meaning: Underlying cause established through investigation. Which term fits?

**C. root cause**

**A Not here.** reproduction: Repeatable demonstration of an observed failure. The usual combination is "obtain a reproduction".

**B Not here.** deterministic behavior: Behavior producing the same result under the same relevant conditions. The usual combination is "check deterministic behavior".

**C Correct.** root cause: Underlying cause established through investigation. The usual combination is "isolate the root cause".

**D Not here.** failure signature: Identifying pattern of an observed failure. The usual combination is "compare the failure signature".

Return to the exercise, page 24

# Review 1 / Explanations / continued

**3. Rebuild the exchange from "Three nickname states, only two defined meanings". Maya: "The client object shows null, but I haven't captured what its serializer sends. Could that conversion drop the property before the request leaves?" Arun: [missing response] Maya: "I'll show the actual three wire examples separately: absent, empty string, and explicit null. Only the first two have defined outcomes so far." Which response belongs in the gap?**

**B. Let us check the request payload in a controlled example. We need to distinguish the client's internal value from the representation delivered to the service.**

**A Not here.** This line belongs to turn 4, after Maya says: "I treated optional as permission to send null. But a missing property and a present property with a null value aren't the same payload." It addresses that point, not the missing turn in the supplied exchange.

**B Correct.** This is Arun's response in turn 10. It connects Maya's preceding point with Maya's follow-up. Request payload identifies the actual submitted data, which may differ from an assumed client representation.

**C Not here.** This line belongs to turn 2, after Maya says: "The form currently sends null when a user removes a nickname. Before I ship that, which input does this service actually define as clearing?" It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 20, after Maya says: "Then I'll report omission preserves, empty string clears, and null unresolved. My proposed null implementation stays pending until the interface decision." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 24

# Review 1 / Explanations / continued

**4. Rebuild the exchange from "The empty date is a suspect, not yet the cause". Nia: "I'll use the approved test environment, not try these files against a live customer import to get a quick answer." Ellis: [missing response] Nia: "My update will say the empty date is suspected, the successful comparison changed two factors, and a controlled repeat test is pending." Which response belongs in the gap?**

**B. Use the suitable nonproduction environment and keep its configuration in the record. We need evidence without changing production data.**

**A Not here.** This line belongs to turn 10, after Nia says: "Should I reduce the file further first? Two rows would be easier to send around, but that version currently passes." It addresses that point, not the missing turn in the supplied exchange.

**B Correct.** This is Ellis's response in turn 18. It connects Nia's preceding point with Nia's follow-up. A nonproduction environment supports the investigation without treating this dialogue as permission to test on live workloads.

**C Not here.** This line belongs to turn 4, after Nia says: "I haven't established that yet. I removed row two and the import passed, which made me suspect the empty date." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 12, after Nia says: "The trace points into the parser. Could that still be downstream of a problem elsewhere in the import?" It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 24

# Review 2 / Explanations

## Cumulative review

### 1. A colleague needs the term for this meaning: Feedback attached to a proposed change. Which term fits?

**B. review comment**

**A Not here.** false success: Indication of success when the operation did not succeed. The usual combination is "prevent false success".

**B Correct.** review comment: Feedback attached to a proposed change. The usual combination is "write a review comment".

**C Not here.** assertion: Test statement checking an expected condition. The usual combination is "strengthen the assertion".

**D Not here.** side effect: Observable effect beyond a function's returned value. The usual combination is "examine the side effect".

Return to the exercise, page 47

### 2. A colleague needs the term for this meaning: Notice that an interface is discouraged or scheduled for replacement. Which term fits?

**B. deprecation**

**A Not here.** patch version: Version component commonly used for fixes under a stated convention. The usual combination is "verify patch-version policy".

**B Correct.** deprecation: Notice that an interface is discouraged or scheduled for replacement. The usual combination is "investigate a deprecation".

**C Not here.** public interface: Supported surface exposed for other components to use. The usual combination is "compare the public interface".

**D Not here.** direct dependency: Package explicitly used or declared by the application here. The usual combination is "upgrade a direct dependency".

Return to the exercise, page 47

# Review 2 / Explanations / continued

**3. Rebuild the exchange from "The empty date is a suspect, not yet the cause". Nia: "Should I reduce the file further first? Two rows would be easier to send around, but that version currently passes." Ellis: [missing response] Nia: "The trace points into the parser. Could that still be downstream of a problem elsewhere in the import?" Which response belongs in the gap?**

**B. Keep the failing input fixture intact. A smaller passing file isn't a smaller reproduction of the failure we're investigating.**

**A Not here.** This line belongs to turn 12, after Nia says: "The trace points into the parser. Could that still be downstream of a problem elsewhere in the import?" It addresses that point, not the missing turn in the supplied exchange.

**B Correct.** This is Ellis's response in turn 10. It connects Nia's preceding point with Nia's follow-up. Input fixture names the prepared test data, which must remain available to reproduce the original failure.

**C Not here.** This line belongs to turn 18, after Nia says: "I'll use the approved test environment, not try these files against a live customer import to get a quick answer." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 4, after Nia says: "I haven't established that yet. I removed row two and the import passed, which made me suspect the empty date." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 47

# Review 2 / Explanations / continued

**4. Rebuild the exchange from "A direct upgrade with an indirect change". Leila: "I'll name that coverage gap instead of saying all tests passed therefore compatible. A green suite could simply miss the changed path." Ben: [missing response] Leila: "My revised summary will separate resolved versions, deprecation status, usage, and replacement behavior. The last two still need investigation." Which response belongs in the gap?**

**B. Exactly. We need upgrade evidence tied to this change. It should show what was checked and what remains unknown, not just that an upgrade command succeeded.**

**A Not here.** This line belongs to turn 10, after Leila says: "The replacement has a similar name. Before I switch anything, where should we check whether its inputs and error behavior actually match?" It addresses that point, not the missing turn in the supplied exchange.

**B Correct.** This is Ben's response in turn 18. It connects Leila's preceding point with Leila's follow-up. Upgrade evidence combines actual versions, affected usage, documentation, and relevant checks rather than a broad assertion of safety.

**C Not here.** This line belongs to turn 4, after Leila says: "I can attach the package constraints and actual selected versions. Which record do you need to see what this build will really use?" It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 2, after Leila says: "The ParcelKit upgrade also selects a different TextCore version. The request title doesn't show that indirect change, but the resolved list does." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 47

# Review 3 / Explanations

## Cumulative review

### 1. A colleague needs the term for this meaning: Test substitute supplying predetermined responses. Which term fits?

**C. stub**

**A Not here.** act: Execute the behavior being tested. The usual combination is "act on the save request".

**B Not here.** exception: Error signal raised during execution under the relevant model. The usual combination is "configure an exception".

**C Correct.** stub: Test substitute supplying predetermined responses. The usual combination is "use a success stub".

**D Not here.** branch coverage: Measure of which decision outcomes a test set executes. The usual combination is "examine branch coverage".

Return to the exercise, page 69

### 2. A colleague needs the term for this meaning: Amount of memory used under the stated conditions. Which term fits?

**A. memory footprint**

**A Correct.** memory footprint: Amount of memory used under the stated conditions. The usual combination is "compare the memory footprint".

**B Not here.** outlier: Observation unusually distant from most other values. The usual combination is "investigate an outlier".

**C Not here.** benchmark: Structured measurement under specified conditions. The usual combination is "run a benchmark".

**D Not here.** relative reduction: Decrease expressed as a fraction of the original value. The usual combination is "calculate the relative reduction".

Return to the exercise, page 69

# Review 3 / Explanations / continued

**3. Rebuild the exchange from "A direct upgrade with an indirect change". Leila: "The replacement has a similar name. Before I switch anything, where should we check whether its inputs and error behavior actually match?" Ben: [missing response] Leila: "The version looks familiar, but we haven't checked either package's versioning policy. I shouldn't infer its compatibility promise from the number alone." Which response belongs in the gap?**

**D. Consult the migration guide first. Similar names do not establish identical inputs, outputs, error behavior, or handling of edge cases. We need the relevant documented differences.**

**A Not here.** This line belongs to turn 18, after Leila says: "I'll name that coverage gap instead of saying all tests passed therefore compatible. A green suite could simply miss the changed path." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 2, after Leila says: "The ParcelKit upgrade also selects a different TextCore version. The request title doesn't show that indirect change, but the resolved list does." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 4, after Leila says: "I can attach the package constraints and actual selected versions. Which record do you need to see what this build will really use?" It addresses that point, not the missing turn in the supplied exchange.

**D Correct.** This is Ben's response in turn 10. It connects Leila's preceding point with Leila's follow-up. The migration guide provides transition details that a similar replacement name cannot establish.

Return to the exercise, page 69

# Review 3 / Explanations / continued

**4. Rebuild the exchange from "Fifteen milliseconds faster, forty megabytes over budget". Sora: "I could request a changed memory budget, but that wouldn't make this run compliant with the current one. It would be a separate decision." Malik: [missing response] Sora: "I'll report the matched conditions, latency decrease, memory figures, and unmeasured tail and live behavior. We still need to decide what to investigate next." Which response belongs in the gap?**

**A. Exactly. Current budget compliance is clear: A is below the limit and B is above it. A hypothetical future budget should not rewrite today's comparison.**

**A Correct.** This is Malik's response in turn 18. It connects Sora's preceding point with Sora's follow-up. Budget compliance is evaluated against the stated 120-megabyte limit, which B exceeds regardless of its latency gain.

**B Not here.** This line belongs to turn 10, after Sora says: "We didn't collect tail percentiles. A lower median could coexist with worse slow requests, so I can't claim improvement for every request." It addresses that point, not the missing turn in the supplied exchange.

**C Not here.** This line belongs to turn 2, after Sora says: "B's measured median is twenty-five milliseconds versus A's forty on the same input and machine. I'd like to propose B, but its memory figure complicates that." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 20, after Sora says: "I'll report the matched conditions, latency decrease, memory figures, and unmeasured tail and live behavior. We still need to decide what to investigate next." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 69

# Review 4 / Explanations

## Cumulative review

### 1. A colleague needs the term for this meaning: Identifier or meaning used to distinguish one operation from another. Which term fits?

**D. request identity**

**A Not here.** shared state: Data or condition used by multiple participants or operations. The usual combination is "reconcile shared state".

**B Not here.** deduplication: Recognition and handling of repeated requests or records. The usual combination is "define deduplication behavior".

**C Not here.** retry budget: Limit on permitted repeated attempts under a policy. The usual combination is "set a retry budget".

**D Correct.** request identity: Identifier or meaning used to distinguish one operation from another. The usual combination is "preserve request identity".

Return to the exercise, page 91

### 2. A colleague needs the term for this meaning: Change to stored data or its structure. Which term fits?

**D. migration**

**A Not here.** known limitation: Identified restriction or unresolved boundary. The usual combination is "list known limitations".

**B Not here.** release candidate: Version proposed for release pending the relevant checks. The usual combination is "review the release candidate".

**C Not here.** forward compatibility: Ability to work with relevant newer data or interfaces. The usual combination is "check forward compatibility".

**D Correct.** migration: Change to stored data or its structure. The usual combination is "review the migration".

Return to the exercise, page 91

# Review 4 / Explanations / continued

**3. Rebuild the exchange from "Fifteen milliseconds faster, forty megabytes over budget". Sora: "We didn't collect tail percentiles. A lower median could coexist with worse slow requests, so I can't claim improvement for every request." Malik: [missing response] Sora: "A is forty megabytes below the same budget. I'll keep that separate from B's forty-megabyte excess rather than call both numbers spare memory." Which response belongs in the gap?**

**B. Correct. Tail latency remains unmeasured. Do not turn a better middle value into a claim about every request or a specific percentile we did not record.**

**A Not here.** This line belongs to turn 6, after Sora says: "For the percentage, I subtracted twenty-five from forty and divided the fifteen-millisecond decrease by forty. That gives thirty-seven-point-five percent." It addresses that point, not the missing turn in the supplied exchange.

**B Correct.** This is Malik's response in turn 10. It connects Sora's preceding point with Sora's follow-up. Tail latency concerns slower-end observations and cannot be inferred from the supplied median alone.

**C Not here.** This line belongs to turn 2, after Sora says: "B's measured median is twenty-five milliseconds versus A's forty on the same input and machine. I'd like to propose B, but its memory figure complicates that." It addresses that point, not the missing turn in the supplied exchange.

**D Not here.** This line belongs to turn 12, after Sora says: "A is forty megabytes below the same budget. I'll keep that separate from B's forty-megabyte excess rather than call both numbers spare memory." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 91

# Review 4 / Explanations / continued

**4. Rebuild the exchange from "R18 is deployed, but the release decision is not made". Chen: "When we get the compatibility result, I'll attach the actual evidence. Until then the successful staging step stays separate from that open check." Lee: [missing response] Chen: "Final status: staging deployed, search disabled, compatibility pending, production decision with Lee. No statement that search is live." Which response belongs in the gap?**

**C. Good. A release gate isn't met simply because another step completed. We'll review the missing evidence before deciding whether to advance.**

**A Not here.** This line belongs to turn 2, after Chen says: "R18 is deployed in staging. Search is still disabled, and I need your release decision after we address the compatibility gap." It addresses that point, not the missing turn in the supplied exchange.

**B Not here.** This line belongs to turn 6, after Chen says: "The migration adds an optional field. I called it backward compatible in the draft, but we haven't tested the prior application against the migrated data." It addresses that point, not the missing turn in the supplied exchange.

**C Correct.** This is Lee's response in turn 18. It connects Chen's preceding point with Chen's follow-up. A release gate is a condition for advancing the release and cannot be treated as satisfied by an unrelated completed deployment.

**D Not here.** This line belongs to turn 14, after Chen says: "I'll leave production pending. Sending you the handoff doesn't authorize me to turn search on or change the production environment." It addresses that point, not the missing turn in the supplied exchange.

Return to the exercise, page 91

# Additional case 1 / Explanations

## Extended conversation

### 1. stale representation

**Turn 2.** B edited the older v7 representation containing eighteen dollars; the current representation is v8 with twenty dollars, so the submitted full record is stale.

### 2. If-Match precondition

**Turn 4.** If-Match requires the supplied entity tag to match the current representation using strong comparison; v7 does not match the current v8 here.

### 3. lost update

**Turn 6.** Sending the stale eighteen-dollar price with a current tag could overwrite A's twenty-dollar change; replacing only the tag does not reconcile the old body.

### 4. strong ETag

**Turn 8.** A strong ETag is the representation validator used by this conditional request; it is not permission to submit stale field values or proof of user intent.

### 5. HTTP 412

**Turn 10.** HTTP 412 is the specified precondition-failure response in this case; it does not establish a server outage, a successful save, or a generic authorization failure.

### 6. conflict reconciliation

**Turn 12.** Conflict reconciliation compares the current record with the intended edit before constructing another request; copying v8 onto the old body would bypass that reasoning.

Return to the conversation, page 94

## Transfer exchange

### 1. 412

**Developer:** The returned status is 412 .

**Turn 1.** The case explicitly specifies precondition failure; the stale v11 tag does not match v12.

# Additional case 1 / Explanations / continued

### 2. 9

**Reviewer:** The stored quantity remains 9 .

**Turn 2.** The rejected request makes no change, so the current quantity nine is preserved.

### 3. old

**Developer:** The stored note is still old .

**Turn 3.** The requested new note was not applied when the conditional request failed.

### 4. stale

**Reviewer:** Replacing only the tag would leave the payload stale .

**Turn 4.** The body would still contain seven rather than the current nine; a newer tag alone does not reconcile those values.

Return to the conversation, page 97

# Additional case 2 / Explanations

## Extended conversation

### 1. N+1 query pattern

**Turn 2.** One initial order query plus one line query for each of fifty orders produces fifty-one queries; the extra work grows with the number of orders.

### 2. lazy loading

**Turn 4.** Lazy loading defers fetching related data until access; in this supplied trace each order's line collection triggers another query.

### 3. batched lookup

**Turn 6.** The proposed implementation uses one related-line query for the fifty selected order IDs, replacing fifty separate line queries while retaining the initial order query.

### 4. round-trip count

**Turn 8.** The new path has two serial round trips, contributing eight milliseconds in the supplied model; the count is not the complete application response time.

### 5. network component

**Turn 10.** Eight milliseconds accounts only for the two modeled network delays; database execution, transferred data, and application processing remain additional and unmeasured.

### 6. result set

**Turn 12.** The result set is the data returned by the queries; reducing query count does not establish its size, memory cost, or correct association with each order.

Return to the conversation, page 99

## Transfer exchange

### 1. 5

**Developer:** The batched path makes 5 total queries.

**Turn 1.** Four groups of fifty IDs require four line queries plus one initial order query, totaling five.

# Additional case 2 / Explanations / continued

### 2. 20

**Reviewer:** Its modeled network delay is 20 milliseconds.

**Turn 2.** Five serial round trips multiplied by four milliseconds equals twenty milliseconds, excluding all other work.

### 3. 201

**Developer:** The old path makes 201 total queries.

**Turn 3.** One order query plus two hundred individual line queries equals two hundred one.

### 4. 784

**Reviewer:** The modeled network-delay reduction is 784 milliseconds.

**Turn 4.** The old component is 201 times 4, or 804 milliseconds; subtract the new 20 to obtain 784, not a measured total-response improvement.

Return to the conversation, page 102

# Additional case 3 / Explanations

## Extended conversation

### 1. rounding stage

**Turn 2.** The agreed contract rounds each line's charge before summing; rounding once on the basket subtotal is a different calculation even with the same rate.

### 2. half-up rule

**Turn 4.** For these positive amounts, half-up rounds the exact half-cent tie upward to one cent; it must not be replaced with half-even or an unspecified default.

### 3. basket-level calculation

**Turn 6.** The proposed calculation rounds 0.015 once to 0.02 and totals 0.17; it is internally consistent but does not implement the supplied line-level contract.

### 4. decimal arithmetic

**Turn 8.** Decimal arithmetic can preserve these exact decimal inputs and the requested rounding operation; merely formatting a binary result does not implement the missing line-level rule.

### 5. rounding mode

**Turn 10.** The rounding mode determines how ties are resolved; the contract explicitly requires half-up rather than relying on a library's possibly different default.

### 6. line-item boundary

**Turn 12.** The three separate line items each receive their own rounded charge; merging them into one line would change the supplied contract's calculation unit.

Return to the conversation, page 104

## Transfer exchange

### 1. 0.03

**Developer:** Each rounded line charge is 0.03 dollars.

**Turn 1.** Ten percent of 0.25 is exactly 0.025; half-up to two decimal places yields 0.03.

# Additional case 3 / Explanations / continued

### 2. 0.12

**Reviewer:** The sum of the four rounded charges is 0.12 dollars.

**Turn 2.** Four separate charges of 0.03 total 0.12; do not round the basket instead.

### 3. 1.12

**Developer:** The contract's final total is 1.12 dollars.

**Turn 3.** The four prices total 1.00, to which the line-level charge total of 0.12 is added.

### 4. 0.02

**Reviewer:** That exceeds the basket-level total by 0.02 dollars.

**Turn 4.** The basket method gives 1.00 plus 0.10, or 1.10; the contract total 1.12 is two cents higher.

Return to the conversation, page 107

# Find a phrase by purpose

## Clarifying interface contracts

**Name the boundary:** I want to confirm the profile interface contract.

**Describe absence:** An omitted nickname leaves the stored value unchanged.

**Describe clearing:** An empty string clears the nickname.

Full phrase bank, page 7

## Debugging from reproducible evidence

**Report the failing input:** The three-row file fails during import.

**Report the distinguishing detail:** One row contains an empty date.

**Name the observation:** The stack trace reaches date parsing.

Full phrase bank, page 17

## Giving actionable code-review feedback

**Acknowledge the acceptable part:** The helper name is clear.

**Locate the concern:** This notification call has moved outside the success branch.

**Describe the consequence:** A failed export can now send a completion notice.

Full phrase bank, page 29

## Requesting dependency evidence

**Identify the direct dependency:** The application uses ParcelKit directly.

**Identify the indirect dependency:** TextCore comes through ParcelKit.

**Name the additional change:** The resolved TextCore version also changes.

Full phrase bank, page 39

# Find a phrase by purpose / continued

## Explaining what a test actually exercises

**Describe the current setup:** The storage double always succeeds.

**Name the tested path:** This test exercises the happy path.

**Recall the defect:** The bug showed success after a rejected save.

Full phrase bank, page 52

## Comparing performance tradeoffs fairly

**Name the conditions:** Both runs use the same input and machine.

**State A:** A has a forty-millisecond median and uses eighty megabytes.

**State B:** B has a twenty-five-millisecond median and uses one hundred sixty megabytes.

Full phrase bank, page 62

## Discussing retries and shared state

**State the observation:** The client timed out before receiving a reply.

**Preserve uncertainty:** The service may already have created the reservation.

**Avoid a false conclusion:** No reply does not prove no reservation.

Full phrase bank, page 74

## Handing over a release with explicit limits

**Identify the release:** This handoff concerns R18.

**Name the environment:** R18 is deployed in staging.

**Name the feature state:** New search remains disabled by its flag.

Full phrase bank, page 84

# Vocabulary index

**absolute difference**
Lesson 06, p. 61

**acceptance criterion**
Lesson 01, p. 6

**acknowledgment**
Lesson 07, p. 72

**act**
Lesson 05, p. 51

**adapter**
Lesson 05, p. 51

**application programming interface**
Lesson 01, p. 5

**approval**
Lesson 03, p. 28

**arrange**
Lesson 05, p. 51

**assert**
Lesson 05, p. 51

**assertion**
Lesson 03, p. 28, Lesson 05, p. 50

**at-least-once delivery**
Lesson 07, p. 72

**at-most-once delivery**
Lesson 07, p. 73

**atomic operation**
Lesson 07, p. 73

**authorization**
Lesson 07, p. 73

**backoff**
Lesson 07, p. 73

**backward compatibility**
Lesson 01, p. 6, Lesson 08, p. 83

**baseline**
Lesson 06, p. 60

**basket-level calculation**
Case 3, p. 103

**batched lookup**
Lesson 02, p. 16, Case 2, p. 98

**behavior change**
Lesson 03, p. 27

**behavioral verification**
Lesson 05, p. 51

**benchmark**
Lesson 06, p. 60

**blocking concern**
Lesson 03, p. 28

**bottleneck**
Lesson 06, p. 61

**branch coverage**
Lesson 05, p. 51

**breaking change**
Lesson 04, p. 38

**budget compliance**
Lesson 06, p. 61

**call site**
Lesson 03, p. 27, Lesson 04, p. 37

**causal claim**
Lesson 02, p. 16

**changelog**
Lesson 04, p. 38

**compatibility**
Lesson 04, p. 38

**completion notification**
Lesson 03, p. 27

**concurrency**
Lesson 07, p. 73

**conflict reconciliation**
Case 1, p. 93

**confounding variable**
Lesson 02, p. 15

**consumer**
Lesson 01, p. 6

**contract test**
Lesson 01, p. 6

**control flow**
Lesson 03, p. 27

**controlled comparison**
Lesson 02, p. 15

**controlled conditions**
Lesson 06, p. 61

**correlation identifier**
Lesson 07, p. 73

**coverage gap**
Lesson 05, p. 51

**data compatibility**
Lesson 08, p. 83

**date parsing**
Lesson 02, p. 15

# Vocabulary index / continued

**decimal arithmetic**
Lesson 01, p. 6, Case 3, p. 103

**default value**
Lesson 01, p. 5

**deployment**
Lesson 08, p. 82

**deserialization**
Lesson 01, p. 6

**diff**
Lesson 03, p. 27

**disabled state**
Lesson 08, p. 82

**duplicate request**
Lesson 07, p. 72

**empty field**
Lesson 02, p. 16

**entry point**
Lesson 04, p. 37

**environment parity**
Lesson 02, p. 16

**exception**
Lesson 02, p. 16, Lesson 05, p. 50

**failure signature**
Lesson 02, p. 16

**false success**
Lesson 03, p. 28

**field presence**
Lesson 01, p. 6

**forward compatibility**
Lesson 08, p. 83

**guard condition**
Lesson 03, p. 28

**handoff**
Lesson 08, p. 83

**headroom**
Lesson 06, p. 60

**hypothesis**
Lesson 02, p. 15

**idempotency key**
Lesson 07, p. 72

**input fixture**
Lesson 02, p. 15

**integration test**
Lesson 05, p. 50

**deduplication**
Lesson 07, p. 73

**dependency tree**
Lesson 04, p. 37

**deprecation**
Lesson 04, p. 37

**deterministic behavior**
Lesson 02, p. 16

**direct dependency**
Lesson 04, p. 37

**distribution**
Lesson 06, p. 61

**duplicate side effect**
Lesson 07, p. 72

**empty string**
Lesson 01, p. 5

**environment boundary**
Lesson 08, p. 83

**error path**
Lesson 05, p. 51

**failure path**
Lesson 03, p. 27

**false negative**
Lesson 05, p. 51

**feature flag**
Lesson 08, p. 82

**fixture**
Lesson 05, p. 50

**go/no-go decision**
Lesson 08, p. 83

**half-up rule**
Case 3, p. 103

**happy path**
Lesson 05, p. 51

**HTTP 412**
Case 1, p. 93

**idempotency**
Lesson 07, p. 72

**If-Match precondition**
Case 1, p. 93

**instrumentation**
Lesson 02, p. 16

**intent**
Lesson 03, p. 28

# Vocabulary index / continued

**interaction verification**
Lesson 05, p. 51

**interface contract**
Lesson 01, p. 5

**known limitation**
Lesson 08, p. 83

**latency**
Lesson 06, p. 60

**lazy loading**
Case 2, p. 98

**line-item boundary**
Case 3, p. 103

**lockfile**
Lesson 04, p. 37

**lost update**
Case 1, p. 93

**major version**
Lesson 04, p. 38

**malformed input**
Lesson 02, p. 16

**measurement noise**
Lesson 06, p. 61

**median**
Lesson 06, p. 60

**memory footprint**
Lesson 06, p. 60

**migration**
Lesson 08, p. 82

**migration guide**
Lesson 04, p. 37

**minor version**
Lesson 04, p. 38

**mock**
Lesson 05, p. 50

**N+1 query pattern**
Lesson 02, p. 16, Case 2, p. 98

**negative assertion**
Lesson 05, p. 50

**network component**
Case 2, p. 98

**nonblocking suggestion**
Lesson 03, p. 28

**nonproduction environment**
Lesson 02, p. 16

**null value**
Lesson 01, p. 5

**nullable field**
Lesson 01, p. 5

**observable behavior**
Lesson 03, p. 28

**omitted field**
Lesson 01, p. 5

**optimistic concurrency**
Lesson 02, p. 15

**optional field**
Lesson 01, p. 5, Lesson 08, p. 82

**outlier**
Lesson 06, p. 61

**overrun**
Lesson 06, p. 60

**p95**
Lesson 06, p. 60

**parser**
Lesson 02, p. 15

**partial update**
Lesson 01, p. 5

**patch revision**
Lesson 03, p. 28

**patch version**
Lesson 04, p. 38

**percentile**
Lesson 06, p. 60

**pin**
Lesson 04, p. 38

**previous version**
Lesson 08, p. 82

**producer**
Lesson 01, p. 6

**production**
Lesson 08, p. 82

**production behavior**
Lesson 06, p. 61

**public interface**
Lesson 04, p. 38

**pull request**
Lesson 03, p. 27

**race condition**
Lesson 07, p. 73

# Vocabulary index / continued

**readiness claim**
Lesson 08, p. 83

**reconciliation**
Lesson 07, p. 72

**refactor**
Lesson 03, p. 27

**regression**
Lesson 02, p. 16

**regression test**
Lesson 03, p. 28

**rejection**
Lesson 05, p. 50

**relative reduction**
Lesson 06, p. 61

**release candidate**
Lesson 08, p. 82

**release gate**
Lesson 08, p. 83

**release note**
Lesson 04, p. 37

**release owner**
Lesson 08, p. 83

**removal**
Lesson 04, p. 37

**rename**
Lesson 03, p. 27

**replacement interface**
Lesson 04, p. 38

**reproducible example**
Lesson 02, p. 15

**reproduction**
Lesson 02, p. 15

**request identity**
Lesson 07, p. 72

**request payload**
Lesson 01, p. 5

**resolution**
Lesson 04, p. 38

**resolved version**
Lesson 04, p. 37

**resource budget**
Lesson 06, p. 60

**result set**
Case 2, p. 98

**retry**
Lesson 07, p. 72

**retry budget**
Lesson 07, p. 73

**review comment**
Lesson 03, p. 27

**review scope**
Lesson 03, p. 28

**rollback**
Lesson 08, p. 82

**rollout**
Lesson 08, p. 82

**root cause**
Lesson 02, p. 15

**round-trip count**
Case 2, p. 98

**rounding mode**
Lesson 02, p. 15, Case 3, p. 103

**rounding stage**
Case 3, p. 103

**sample**
Lesson 06, p. 61

**sandbox**
Lesson 07, p. 73

**schema**
Lesson 01, p. 6

**schema compatibility**
Lesson 08, p. 82

**semantic versioning**
Lesson 04, p. 38

**serialization**
Lesson 01, p. 6

**shared state**
Lesson 07, p. 72

**side effect**
Lesson 03, p. 27

**sign-off**
Lesson 08, p. 83

**stack trace**
Lesson 02, p. 15

**staging**
Lesson 08, p. 82

**stale representation**
Case 1, p. 93

# Vocabulary index / continued

**status lookup**
Lesson 07, p. 73

**strong ETag**
Lesson 01, p. 6, Case 1, p. 93

**success branch**
Lesson 03, p. 27

**tail latency**
Lesson 06, p. 60

**test isolation**
Lesson 05, p. 50

**throughput**
Lesson 06, p. 60

**tradeoff**
Lesson 06, p. 61

**transitive dependency**
Lesson 04, p. 37

**unit test**
Lesson 05, p. 50

**upgrade evidence**
Lesson 04, p. 38

**verification evidence**
Lesson 03, p. 28

**version constraint**
Lesson 04, p. 37

**stored state**
Lesson 01, p. 5

**stub**
Lesson 05, p. 50

**success response**
Lesson 05, p. 50

**test double**
Lesson 05, p. 50

**test oracle**
Lesson 05, p. 51

**timeout**
Lesson 07, p. 72

**transaction boundary**
Lesson 07, p. 73

**uncertain outcome**
Lesson 07, p. 72

**unspecified interface behavior**
Lesson 01, p. 6

**validation rule**
Lesson 01, p. 5

**verification matrix**
Lesson 08, p. 83

**workload**
Lesson 06, p. 61

# References and publication notes

All cases, organizations, figures, and conversations are fictional teaching material, not transcripts. Communication practice does not replace qualified advice or current workplace procedures.


### [1] Google Engineering Practices. What to Look for in a Code Review.

Context for reviewing functionality, tests, naming, and changed behavior. All pull requests and conversations here are original. Source checked 10 October 2026.

https://google.github.io/eng-practices/review/reviewer/looking-for.html

### [2] Amazon Builders Library. Making Retries Safe with Idempotent APIs.

Background on uncertain outcomes and retry contracts. The fictional reservation interface is not an AWS service or implementation guide. Source checked 10 October 2026.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

### [3] Python Documentation. unittest.mock.

Reference for configured test doubles, return values, and raised exceptions. Exercises distinguish simulated behavior from actual dependency coverage. Source checked 10 October 2026.

https://docs.python.org/3/library/unittest.mock.html

### [4] Semantic Versioning 2.0.0.

Terminology for public interfaces, compatibility, and deprecation. The fictional packages are not assumed to follow this versioning convention. Source checked 10 October 2026.

https://semver.org/


> **Companion course**
>
> The website offers the eight-lesson course and related practice. This book can be used independently. No audio access or external account is needed for its exercises.
>
> https://englishladder.com/efsp-software-developers.html

# Make the behavior clear. Make the limits explicit.

Eight engineering conversations follow work from an unclear interface contract to a release handoff. Practice requesting evidence, challenging assumptions, explaining tradeoffs, and distinguishing a tested result from an unverified expectation.

### Clarifying interface contracts

Optional and nullable answer different questions. Optional concerns presence; nullable concerns allowed values. The stated contract defines omission and an empty string, but it does not authorize treating null as either one.

### Debugging from reproducible evidence

The two-row success is useful evidence, but it changes two things at once. It does not distinguish an empty-date effect from other differences associated with the removed row or the changed row count.

### Giving actionable code-review feedback

A rename can be acceptable while the same diff contains a blocking defect. Moving the notification outside the success branch changes observable behavior, so it is not merely a naming preference or a behavior-preserving refactor.

### Requesting dependency evidence

Direct describes the application-to-ParcelKit relationship. Transitive describes the application-to-TextCore relationship through ParcelKit. The dependency actually selected matters, not just the version written in the top-level upgrade request.

---

## Phil Stilwell

© 2026 Phil Stilwell. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior written permission from the author, except as permitted by law.

English Ladder / English for Work / Revised 2026 edition

This publication includes AI-generated text and cover illustration, revised and checked during production. Brand artwork remains the English Ladder mark. This edition makes no claim of independent professional certification.

Typeface: Bitstream Vera, embedded under its accompanying license. Illustration created with OpenAI's native image generator for this edition. The copyright notice is not a claim to ownership of third-party frameworks or references.